

BernatLab

Sensors, dades i visualització

Raspberry Pi · Docker · IA · LoRa · Hort Osona

Mòdul 2

Bernat — 8 de juliol del 2026

Document tècnic de consulta personal

Sobre aquest manual

Aquest és el segon mòdul del manual tècnic del BernatLab. Cobreix tota la cadena de sensors i visualització: el protocol MQTT, el broker Mosquitto, l'esquema de publicació dels sensors, la base de dades InfluxDB, l'agent Telegraf, la programació visual amb Node-RED, la visualització amb Grafana, l'API REST amb FastAPI, la integració amb la web pública Hort Osona i l'operativa del dia a dia.

Com es llegeix

En ordre, si parteixes de zero. Per capítols, si ja tens una base i vols aprofundir. Cada capítol segueix la mateixa estructura: teoria, aplicació al BernatLab, esquemes, comandes útils, errors habituals, exercicis pràctics i resum final.

Índex de capítols

- 11. Del Mòdul 1 al M2: què construïm
- 12. MQTT des de zero
- 13. Mosquitto al BernatLab
- 14. Publicar dades: els sensors
- 15. InfluxDB: base de dades de sèries temporals
- 16. Telegraf: el pont
- 17. Node-RED: programació visual
- 18. Fluxos pràctics
- 19. Grafana: visualitzar les dades
- 20. API pública: servir les dades al món
- 21. Integració amb Hort Osona
- 22. Operativa: còpies, alertes, escalat

Capítol 11 — Del Mòdul 1 al M2: què construïm

"Quan un homelab comença a fer coses útils, deixa de ser un joguina i es converteix en una eina. Aquest mòdul és el pas d'una cosa a l'altra."

11.1 On érem

Al final del Mòdul 1 teníem:

- Un servidor Raspberry Pi 4 que funciona 24/7.
- Tailscale, que ens permet accedir-hi des de qualsevol lloc.
- Portainer, Uptime Kuma, Homepage com a serveis principals.
- Una estructura de carpetes clara a `/home/bernat/home lab/`.
- Un sistema de documentació i versionat amb Git.
- Una full de ruta amb deu o quinze idees pendents.

Però el sistema, ara com ara, **no fa res productiu**. Vigila els seus propis serveis, ensenya un panell bonic, i poca cosa més. Això és perfecte per aprendre, però arriba un punt en què volem que la màquina treballi per a nosaltres, no pas que només existeixi.

Aquest mòdul és el punt d'inflexió. A partir d'ara, cada peça que afegim al BernatLab tindrà una finalitat concreta dins del projecte Hort Osona: rebre dades de sensors, guardar-les, transformar-les, visualitzar-les, alertar-nos, i finalment servir-les a una web pública que qualsevol pugui consultar.

11.2 Què és Hort Osona

Hort Osona és un projecte personal que combina agricultura familiar amb monitorització electrònica. La idea és senzilla: a 245 metres de casa hi ha un hort amb tomaqueres, enciams, pebrots, herbes aromàtiques i fruiters. Volem saber, en temps real o quasi-real:

- La temperatura ambient i la humitat relativa.
- La temperatura del sòl a diferents profunditats.
- La humitat del sòl a diferents zones de l'hort.
- La il·luminació (per saber si les plantes tenen prou llum).
- Potser, en el futur, la pluja acumulada, la velocitat del vent, la radiació solar.

Aquestes dades han de servir per:

- **Decidir quan regar.** Si la humitat del sòl baixa del 30%, és hora de regar.
- **Detectar gelades.** Si la temperatura ambient baixa de 2 °C a la nit, volem una alerta immediata al mòbil.
- **Avaluar l'evolució de la temporada.** Comparar la temperatura mitjana del juny d'enguany amb la del juny de l'any passat.

- **Documentar l'experiment.** Tenir un registre històric de com ha anat cada cultiu.
- **Compartir el projecte.** La web pública Hort Osona permet que altres persones interessades en l'hort urbà vegin què estem fent.

11.3 Per què una cadena completa

Podríem fer les coses de manera més simple: connectar els sensors directament a una base de dades, fer una web que hi connecti, i llest. Però al llarg dels anys, la indústria ha après que per a projectes IoT reals, hi ha una cadena d'eines que funciona millor que les alternatives:

- **MQTT** per rebre dades: protocol lleuger, dissenyat per a xarxes inestables i dispositius de baix consum.
- **InfluxDB** per guardar-les: base de dades optimitzada per a sèries temporals, molt més eficient que una base de dades relacional per a aquest cas.
- **Telegraf** per moure-les: agent lleuger que recull dades de moltes fonts i les escriu a molts destins.
- **Node-RED** per transformar-les: eina visual que ens permet netejar, agregar i reaccionar a les dades sense escriure codi complicat.
- **Grafana** per visualitzar-les: l'eina de gràfiques més potent del món del codi obert.
- **API REST** per servir-les: perquè la web pública les pugui consumir sense accedir directament a la base de dades.

Cadascuna d'aquestes peces fa una sola cosa i la fa bé. Combinades, formen un sistema modular: podem canviar una peça sense tocar les altres, podem entendre cada peça per separat, podem créixer gradualment.

11.4 Arquitectura del sistema

Aquí tenim el diagrama general del que construirem en aquest mòdul:

Esquema Mermaid (text):

```
graph TB
    subgraph Terreny["Hort Osona (al terreny)"]
        S1["Sensor temperatura<br/>(zona tomateres)"]
        S2["Sensor humitat sòl<br/>(zona enciams)"]
        S3["Sensor llum<br/>(zona general)"]
        SN["... altres sensors"]
    end

    subgraph Radio["Transmissió"]
        LORA["LoRa 868 MHz<br/>o Wi-Fi"]
    end

    subgraph Servidor["BernatLab (Raspberry Pi 4)"]
        MOSQ["Mosquitto<br/>(broker MQTT:1883)"]
        TELEG["Telegraf<br/>(recol·lector)"]
        INFLUX["InfluxDB<br/>(base de dades)"]
        NR["Node-RED<br/>(processament)"]
        GRAF["Grafana<br/>(visualització)"]
        API["API FastAPI<br/>(:8000)"]
    end

    subgraph Consums["Consumidors"]
        WEB["Web Hort Osona<br/>(PWA pública)"]
        MOB["Mòbil Bernat<br/>(alertes Telegram)"]
    end
```

```

    GRAFANA_UI["Dashboard Grafana<br/>(:3000 intern)"]
end

S1 --> LORA
S2 --> LORA
S3 --> LORA
SN --> LORA

LORA --> MOSQ
MOSQ --> TELEG
TELEG --> INFLUX
INFLUX --> NR
INFLUX --> GRAF
NR -->|alertes| MOB
INFLUX --> API
API --> WEB
GRAF --> GRAFANA_UI

```

Aquesta arquitectura té moltes virtuts. Vegem-les:

1. **Separació de responsabilitats.** Cada component té un paper clar. Si un falla, els altres poden continuar treballant (amb degradació).
2. **Escalabilitat.** Si volem afegir més sensors, només cal que parlin MQTT i el sistema els acull sense canvis.
3. **Observabilitat.** Grafana ensenya l'estat del sistema; Uptime Kuma vigila que estigui tot dret; els logs de cada servei expliquen què ha passat.
4. **Testabilitat.** Podem simular sensors publicant amb `mosquitto_pub` i veure com es comporta tot el sistema sense tenir el hardware al camp.
5. **Independència del transport.** Avui els sensors poden parlar per Wi-Fi; demà per LoRa; passat demà per 4G. Mentre parlin MQTT, el sistema els acull.

11.5 Què aprendrem en aquest mòdul

En capítols posteriors, aprendrem a:

- **Capítol 12:** entendre el protocol MQTT a fons, des dels conceptes fins a les particularitats que el fan ideal per a IoT.
- **Capítol 13:** instal·lar i configurar Mosquitto al BernatLab, amb autenticació, ACLs i un esquema de topics clar.
- **Capítol 14:** dissenyar l'esquema de publicació dels sensors, amb exemples de codi Python i, opcionalment, C++ per a microcontroladors.
- **Capítol 15:** instal·lar InfluxDB 2.x, entendre els seus conceptes (orgs, buckets, tokens, series) i practicar el llenguatge de consultes Flux.
- **Capítol 16:** configurar Telegraf per rebre dades de MQTT i escriure-les a InfluxDB de forma eficient.
- **Capítol 17:** instal·lar Node-RED i entendre com es programa visualment.
- **Capítol 18:** veure fluxos reals: netejar dades, agregar, detectar anomalies, enviar alertes a Telegram.
- **Capítol 19:** instal·lar Grafana, connectar-lo a InfluxDB, crear dashboards útils, configurar alertes visuals.

- **Capítol 20:** construir una API REST amb FastAPI que serveixi les dades a la web pública, amb autenticació, documentació OpenAPI i bones pràctiques.
- **Capítol 21:** modificar la web Hort Osona perquè consumeixi l'API i mostri gràfiques en temps real (o quasi-real).
- **Capítol 22:** tot el que envolta el sistema un cop està en marxa: còpies de seguretat d'InfluxDB, retenció, alerting avançat, quan caldrà pujar de hardware.

11.6 Què NO farem en aquest mòdul

Igual d'important que la llista de coses que farem és la llista de coses que no farem (encara):

- **LoRa SX1262 en detall.** Això serà el **Mòdul 3**, perquè la part de ràdio té massa detalls per tractar-los de passada. Aquí, però, ja preveurem que la cadena de dades és compatible amb sensors LoRa, deixant la interfície MQTT com a punt d'entrada.
- **IA local amb Ollama i RAG sobre les dades.** Això serà el **Mòdul 4**, quan tinguem prou dades per entrenar res útil i la Raspberry estigui en marxa amb prou solvència.
- **Balanceig de càrrega, alta disponibilitat, Kubernetes.** No tenim dues Raspberry i no les tindrem. Com al Mòdul 1, acceptem que el sistema pot caure i ens preparem per recuperar-lo.
- **Criptografia avançada, TLS per a MQTT, certificats propis.** En el Mòdul 1 ja vam comentar que la xarxa Tailscale ja xifra tot el tràfic, de manera que afegir TLS a MQTT seria redundar. Si mai traïem el servidor de Tailscale, caldria reconsiderar.

11.7 Requisits previs

Per treure profit d'aquest mòdul, has de tenir:

- El Mòdul 1 complet i el BernatLab funcionant.
- Coneixements bàsics de Docker i Docker Compose.
- Coneixements bàsics de Python (per als capítols 14 i 20).
- Pacència: configurar tota aquesta cadena porta dies, no hores.

També has de tenir clar que **la Raspberry Pi 4 encara no ha arribat** (juliol 2026). Això vol dir que tot el que es descriu en aquest mòdul s'ha preparat per fer-se tan bon punt la màquina estigui disponible, però no s'ha pogut provar amb el sistema en producció. Les configuracions s'han validat conceptualment, comparant-les amb la documentació oficial i amb exemples de la comunitat. Quan la RPi arribi, serà el moment d'ajustar els detalls que inevitablement caldrà ajustar (ports, IPs, volums, etc.).

Això, però, no ens ha d'aturar. Aprendre la teoria ara, quan la màquina no hi és, ens permetrà fer les coses més de pressa i amb més criteri quan sí que hi sigui.

11.8 Estat de cada component abans de començar

Fem un repàs de quin és l'estat previst de cada eina al començar aquest mòdul:

Component	Estat inicial	El que farem
Mosquitto	No instal·lat	Instal·lar, configurar, securitzar
InfluxDB	No instal·lat	Instal·lar, crear org i bucket, definir retenció
Telegraf	No instal·lat	Instal·lar, configurar inputs i outputs
Node-RED	No instal·lat	Instal·lar, configurar paleta MQTT i InfluxDB
Grafana	No instal·lat	Instal·lar, connectar a InfluxDB, primer dashboard
API FastAPI	No instal·lat	Desenvolupar, dockeritzar, documentar
Web Hort Osona	Versió actual a GitHub Pages	Modificar per consumir l'API

Això és el "to-do list" d'aquest mòdul. Al final, haurem afegit **sis serveis nous** al BernatLab i haurem modificat la web pública.

11.9 Com afecta al BernatLab

A nivell pràctic, afegir aquesta cadena implica:

- **Més contenidors Docker.** Passarem de 3 serveis principals a 9 o 10.
- **Més consum de RAM.** InfluxDB i Grafana, especialment, mengen RAM. Hauríem d'estar atents als 4 GB totals.
- **Més volums de dades.** InfluxDB pot créixer ràpidament si no controlem la retenció.
- **Més complexitat operacional.** Cadascun d'aquests serveis té la seva pròpia configuració, els seus propis logs, les seves pròpies alertes.
- **Més valor real.** Per contra, tindrem un sistema que ens avisa quan l'hort es gel·la, ensenya gràfiques boniques, i comparteix informació amb el món.

L'arquitectura triada és **la més comuna a la indústria** per a sistemes IoT petits i mitjans. Si mai cal migrar a una plataforma professional (AWS IoT, Azure IoT Hub, Google Cloud IoT), el coneixement serà directament transferible.

11.10 Filosofia d'aquest mòdul

Com al Mòdul 1, ens regim per uns quants principis:

1. **Construir pas a pas.** Afegirem un servei, l'entendrem, el configurarem, el validarem, i només després passarem al següent.
2. **Provar sense hardware.** Durant el desenvolupament, simularem sensors amb `mosquitto_pub` i petits scripts Python. Això ens permet avançar sense dependre del terreny.
3. **Documentar cada decisió.** Cada configuració, cada error, cada solució, va al `CHANGELOG.md` del BernatLab.
4. **Mesurar constantment.** Uptime Kuma vigila que els serveis estiguin vius. Grafana ens dirà quantes dades entren, quantes en surten, quantes es perden.

5. **No obsessionar-se amb la perfecció.** Un sistema que funciona al 80 % durant un any és millor que un sistema que funciona al 100 % durant una setmana i després s'abandona perquè era massa complicat de mantenir.

11.11 Esquema de la cadena

Esquema Mermaid (text):

```
sequenceDiagram
    participant S as Sensor
    participant M as Mosquitto
    participant T as Telegraf
    participant I as InfluxDB
    participant N as Node-RED
    participant G as Grafana
    participant A as API
    participant W as Web

    S->>M: PUBLISH sensors/zona1/temp 23.5
    M->>T: rep el missatge
    T->>I: escriu el punt (mesura, tag, timestamp)
    I-->>N: consulta periòdica (cada 5 min)
    N-->>N: neteja, agrega, decideix
    N-->>Telegram: alerta si cal
    I-->>G: consulta per a gràfiques
    G-->>G: renderitza dashboard
    I-->>A: consulta agregada
    A-->>W: JSON amb últimes mesures
    W-->>W: dibuixa gràfica al client
```

Aquesta seqüència és el que passa des que el sensor fa una lectura fins que la gràfica apareix a la web. Pot trigar des d'un segon (si tot va bé) fins a uns quants minuts (si hi ha cua o problemes de xarxa). Aquest retard és acceptable per al nostre cas: no estem controlant una central nuclear, estem mirant tomateres.

11.12 Resum

En aquest capítol hem vist què construirem en els pròxims dotze capítols: la cadena completa que porta les dades des dels sensors de l'hort fins a la pantalla del mòbil i la web pública. Hem après per què cada eina és necessària, com es connecten entre elles, quin és l'estat inicial i quin serà el final. Hem recordat que la Raspberry encara no ha arribat i que tot el que es descriu està preparat per implementar tan bon punt estigui disponible. En el proper capítol començarem pel primer element de la cadena: el protocol MQTT.

11.13 Exercicis pràctics

1. Fes una llista dels components que tens al BernatLab ara i compara-la amb la que tindràs al final d'aquest mòdul.
2. Dibuixa, a mà, el diagrama de la secció 11.4, amb els teus propis noms per als components.
3. Inventa un cas d'ús nou: una cosa que no sigui Hort Osona, que podries monitorar amb aquesta cadena. Explica-ho en cinc línies.
4. Mira el `docker-compose.yml` actual del BernatLab. Quants serveis té? Quanta RAM consumeixen en total? Tens prou recursos per afegir-ne sis més?

5. Obre Grafana o una eina equivalent i mira els gràfics de l'ús de recursos de la teva màquina actual. Anota els valors en repòs i sota càrrega, si pots.

Comandes útils:

```
docker ps
docker stats --no-stream
free -h
df -h
```

Paraules clau: **arquitectura, MQTT, InfluxDB, Telegraf, Node-RED, Grafana, API, Hort Osona, cadena IoT, sensors, sèries temporals, full de ruta, Mòdul 2, Mòdul 3, Mòdul 4, Tailscale.**

Capítol 12 — MQTT des de zero

"MQTT és la mena de tecnologia que, un cop l'entens, et sembla òbvia. Abans, sembla màgia."

12.1 Per què MQTT i no HTTP

Si volem que un sensor envii dades a un servidor, la solució més immediata seria fer una petició HTTP. Un sensor connectat a la Wi-Fi, fent un `POST` a una URL, amb les dades en format JSON. Això funciona, i molts sistemes petits ho fan servir. Però per a sistemes amb molts sensors, durant molt de temps, en xarxes poc fiables, MQTT és molt millor. Vegem per què.

HTTP: el model de petició-resposta

HTTP està dissenyat per al patró "un client demana, un servidor respon". Cada interacció és independent: el client obre una connexió, envia la petició, rep la resposta, tanca. Si volem que el sensor envii dades periòdicament, el sensor ha de:

1. Obrir una connexió TCP (o reutilitzar-ne una).
2. Enviar una petició HTTP.
3. Esperar la resposta.
4. Tancar la connexió.
5. Tornar a començar d'aquí 30 segons.

Això és car en termes d'energia, ample de banda, i complexitat. Per a un sensor que ha de funcionar amb piles durant mesos, és inviable.

MQTT: el model de publicació-subscripció

MQTT (Message Queuing Telemetry Transport) va ser inventat el 1999 per Andy Stanford-Clark (IBM) i Arlen Nipper (Eurotech). L'objectiu era clar: un protocol que permeti a sensors amb poca energia, poca capacitat de càlcul i connexions intermitents enviar dades a un servidor de manera eficient.

MQTT funciona amb un model de **publicació-subscripció** (pub/sub):

- Hi ha un **broker** central, que és un programa que rep tots els missatges i els distribueix.
- Hi ha **publishers** (publicadors), que envien missatges amb un **topic** (tema).
- Hi ha **subscribers** (subscriptors), que diuen al broker "vull rebre tots els missatges amb aquest patró de topic".

La gràcia és que **els publicadors i els subscriptors no es coneixen entre ells**. El sensor que publica la temperatura no sap qui l'escolta. Pot ser cap, pot ser un, poden ser mil. Això fa el sistema molt flexible.

Comparació pràctica

Característica	HTTP	MQTT
Model	Petició-resposta	Publicació-subscripció
Connexió	Nova per a cada petició (o Keep-Alive)	Persistent, sempre oberta
Quantitat de dades en una transmissió	Centenars de bytes (capçaleres HTTP)	2-100 bytes
Consum d'energia	Alt	Molt baix
Latència	Variable	Molt baixa
Fiabilitat	Depèn del codi de l'aplicació	QoS configurable
Ús típic	Pàgines web, APIs REST	IoT, missatgeria, telemetria

Per a un sensor de temperatura que envia una lectura cada 5 minuts, MQTT és com un cartutx de diana: precís, econòmic, fiable.

12.2 Conceptes fonamentals

El broker

El **broker** és el cor del sistema MQTT. És un programa que escolta en un port TCP (per defecte, 1883 per a MQTT sense xifrar, 8883 per a MQTT sobre TLS) i que rep, distribueix i desa missatges segons les regles que rep dels clients.

Al BernatLab farem servir **Mosquitto**, el broker de referència de la comunitat de codi obert. El coneixerem a fons al Capítol 13.

Els clients

Un **client** MQTT és qualsevol programa que es connecta al broker. Pot ser:

- Un sensor que publica dades (publisher).
- Un consumidor de dades (subscriber).
- Tots dos alhora.

Els clients poden ser:

- Aplicacions en un llenguatge qualsevol (Python, JavaScript, Go, C++, etc.).
- Dispositius encastats (ESP32, ESP8266, Arduino amb Wi-Fi, Raspberry Pi Pico W).
- Altres brokers (per connectar dos brokers entre ells, el que s'anomena "bridge").

Els topics

Un **topic** és una cadena de text organitzada jeràrquicament amb barres (/) com a separadors.

Exemples:

```
sensors/hort/zona-tomateres/temperatura
sensors/hort/zona-enciams/humitat-solo
actuadors/hort/reg/zona-1
casa/temp/sala
```

Els topics no cal que estiguin predefinitos. El broker els accepta tots. El que importa és que tots els participants del sistema segueixin el mateix esquema.

Wildcards

Quan un client vol subscriure's a molts topics, pot fer servir **wildcards**:

- ****+****: un sol nivell. Per exemple, `sensors/+/temperatura` selecciona tots els sensors de temperatura independentment de la zona.
- ****#****: múltiples nivells (ha d'estar al final). Per exemple, `sensors/#` selecciona tot el que comenci per `sensors/`.

Els wildcards només serveixen per subscriure's, mai per publicar.

QoS (Quality of Service)

MQTT defineix tres nivells de qualitat de servei:

- **QoS 0 — "at most once"**. El broker fa el que pot per entregar el missatge, però si alguna cosa falla, el missatge es perd. És el mode més eficient. Adequat per a dades que es renovent constantment (temperatura actual: si en perdem una, la següent ja la substitueix).
- **QoS 1 — "at least once"**. El broker garanteix que el missatge arriba almenys un cop. Pot arribar duplicat. Adequat per a dades que no es poden perdre, com ara una comanda d'actuador.
- **QoS 2 — "exactly once"**. El broker garanteix que el missatge arriba exactament un cop. És el mode més car, però l'únic que garanteix la univocitat.

Per a sensors de temperatura o humitat, **QoS 0** és perfectament adequat. Per a comandes d'actuadors (obrir una vàlvula de reg), **QoS 1 o 2** són convenients.

Retained messages

Un **retained message** és un missatge que el broker desa i l'entrega immediatament a qualsevol subscriptor nou. Pensa en això: si subscrius un client a `sensors/zona1/temperatura` i mai s'ha publicat res, el client no rebrà res. Però si el darrer missatge publicat s'ha marcat com a "retained", el client el rebrà immediatament en connectar-se.

Això és útil per a l'últim valor conegut d'un sensor. No cal esperar el proper cicle de publicació per tenir-ne dades.

Last Will and Testament (LWT)

Quan un client es connecta al broker, pot registrar un **testament**: un missatge que el broker publicarà automàticament quan detecti que el client s'ha desconnectat de manera no esperada (per exemple, perquè s'ha quedat sense bateria).

Això ens permet saber, sense haver de fer pings constantment, si un sensor ha mort. Patró típic: el sensor es connecta, registra el seu testament a `sensors/zona1/status` amb valor `online`, i quan es desconnecta, el broker publica automàticament `offline` al mateix topic.

Clean session

Quan un client es connecta, pot especificar si vol una **clean session** o no:

- **Clean session = true:** el broker no desa cap estat del client. Si es desconnecta, quan torni, no rebrà cap missatge que s'hagi publicat durant la seva absència.
- **Clean session = false:** el broker desa les subscripcions i els missatges pendents. Quan el client torni, rebrà tot el que s'hagi publicat durant la seva absència (segons QoS).

Per a sensors que es desconnecten sovint (per estalviar energia), **clean session = true** és el normal.

12.3 Anatomia d'un missatge MQTT

Un missatge MQTT és molt senzill. Consta de:

- **Topic:** la cadena que identifica el tema.
- **Payload:** les dades (pot ser text, números, JSON, binari).
- **QoS:** el nivell de servei desitjat.
- **Retain:** si és un retained message o no.
- **Packet ID:** identificador intern (per a QoS > 0).

Exemple d'un missatge publicat amb `mosquitto_pub`:

```
mosquitto_pub -h broker.local -t sensors/zona1/temperatura \
-m '{"valor": 23.5, "unitat": "graus", "ts": 1717823400}' \
-q 0 -r
```

Aquí estem publicant:

- Al topic `sensors/zona1/temperatura`.
- Amb un payload JSON de 60 bytes aproximadament.
- Amb QoS 0.
- Marcat com a retained (`-r`).

Això és tot. La simplicitat és el que fa MQTT tan eficient.

12.4 Seguretat: autenticació i xifrat

MQTT en la seva forma més bàsica és **obert**: qualsevol pot subscriure's o publicar a qualsevol topic. Això, evidentment, no és acceptable per a un sistema en producció.

MQTT suporta:

- **Autenticació per usuari i contrasenya.** El client envia un nom d'usuari i una contrasenya en connectar-se. El broker els valida i accepta o rebutja la connexió.
- **Xifrat TLS/SSL.** Tot el tràfic viatja xifrat, com en HTTPS. Això és important si el broker està exposat a una xarxa que no controlem.
- **Control d'accés (ACL).** El broker pot tenir una llista que diu quin usuari pot subscriure's o publicar a quin topic. Per exemple, el sensor `temp-zona1` pot publicar a `sensors/zona1/#` però no a `sensors/zona2/#`.

Al BernatLab, **no activarem TLS per a MQTT perquè la xarxa Tailscale ja xifra tot el tràfic**. Això ens estalvia la complexitat de gestionar certificats. Si mai traiem el servidor de Tailscale, caldria reconsiderar.

Sí que activarem **autenticació i ACLs**, que són la base de la seguretat lògica: cada sensor té el seu usuari i només pot parlar dels seus topics.

12.5 Com es comparen les adreces i els topics

Una analogia útil: els topics MQTT són com les adreces de correu electrònic, però jeràrquiques i sense registre central. Pots inventar-te un topic nou en qualsevol moment i el broker l'acceptarà. La convenció és seguir un esquema consistent a tot el sistema.

Un esquema típic per a un sistema de sensors és:

```
{sistema}/{zona}/{tipus_sensor}/{identificador}
```

Per exemple:

- hort/zona1/temperatura/aire
- hort/zona1/humitat/sol
- hort/zona1/lluminositat/par
- hort/zona1/estat
- casa/sala/temperatura
- casa/sala/humitat

A l'Hort Osona farem servir un esquema semblant, tot i que el detall el veurem al Capítol 14.

12.6 Exemple pràctic: conversa MQTT

Imaginem un termòmetre que vol enviar la temperatura a un servidor. La conversa seria:

Termòmetre → **Broker** (en connectar-se):

```
CONNECT
  client_id: "term-hort-zona1"
  username: "sensor-temp-zona1"
  password: "*****"
  clean_session: true
  will_topic: "hort/zona1/estat"
  will_message: "offline"
  will_qos: 1
  will_retain: true
```

Termòmetre → **Broker** (cada 30 segons):

```
PUBLISH
  topic: "hort/zona1/temperatura/aire"
  payload: "23.5"
  qos: 0
  retain: true
```

Termòmetre → **Broker** (en connectar-se, després d'establir subscripcions buides):

```
SUBSCRIBE
  topic: "" (cap subscripció)
```

Això és tot. La informació viatja en 30-50 bytes per cicle, perfectament assumible per a una xarxa LoRa de baixa capacitat o per a una Wi-Fi domèstica.

12.7 Avantatges i inconvenients de MQTT

Avantatges

- **Lleuger**: payload de pocs bytes, molt poc overhead.
- **Eficient en energia**: ideal per a dispositius amb piles.
- **Desacoblat**: publishers i subscriptors no es coneixen.
- **Escalable**: un sol broker pot gestionar milers de clients.
- **QoS configurable**: podem triar entre velocitat i fiabilitat.
- **Comunitat gran**: hi ha llibreries per a tots els llenguatges i plataformes.
- **Estàndard obert**: especificació pública, implementacions múltiples.

Inconvenients

- **No és HTTP**: cal aprendre un protocol nou.
- **El broker és un punt únic de fallada**: si cau, tot el sistema deixa de funcionar. Solució: brokers redundants.
- **No té esquema de dades**: cal definir el format del payload (típicament JSON).
- **Compleixitat afegida**: en sistemes molt petits, pot ser excessiu.

Al BernatLab, els avantatges superen de llarg els inconvenients. MQTT és l'estàndard de facto per a IoT i mereix la pena aprendre'l.

12.8 Clients MQTT: eines habituals

Per treballar amb MQTT al BernatLab, necessitem clients. Els més habituals són:

mosquitto-clients

El paquet estàndard que ve amb Mosquitto. Inclou dues eines de línia d'ordres:

- **mosquitto_pub**: per publicar missatges.
- **mosquitto_sub**: per subscriure's i rebre missatges.

Exemple:

```
# Publicar
mosquitto_pub -h 100.115.134.76 -t test -m "hola"

# Subscriure
mosquitto_sub -h 100.115.134.76 -t "sensors/#" -v
```

paho-mqtt (Python)

Llibreria Python de referència per a MQTT. Suporta Python 3.6+ i té versions síncrones i asíncrones.

```
import paho.mqtt.client as mqtt
```

```
client = mqtt.Client()
client.connect("100.115.134.76", 1883, 60)
client.publish("test", "hola des de Python")
client.loop_start()
```

mqtt.js (JavaScript)

Llibreria MQTT per a Node.js i navegadors. Molt popular per a clients web.

Llibreries per a microcontroladors

Per a ESP32, ESP8266, Arduino, Raspberry Pi Pico W, hi ha llibreries MQTT específiques. Les més populars són `PubSubClient` per a Arduino, i les integracions natives de frameworks com ESPHome, Tasmota o PlatformIO.

12.9 Exemple complet: simulant un sensor

Vegem un exemple de Python que simula un sensor de temperatura. És el que farem servir per provar tot el sistema sense tenir el hardware real:

```
"""
simula_sensor.py
Simula un sensor de temperatura que publica cada 5 segons.
"""

import json
import random
import time
import paho.mqtt.client as mqtt

BROKER = "100.115.134.76"
PORT = 1883
USERNAME = "sensor-temp-zona1"
PASSWORD = "elmeupassword"
TOPIC = "hort/zona1/temperatura/aire"
LWT_TOPIC = "hort/zona1/estat"

client = mqtt.Client(client_id="simula-temp-zona1", clean_session=True)
client.username_pw_set(USERNAME, PASSWORD)
client.will_set(LWT_TOPIC, payload="offline", qos=1, retain=True)

def publicar_temperatura():
    valor = 20 + random.gauss(0, 3)
    payload = json.dumps({
        "valor": round(valor, 2),
        "unitat": "graus",
    })
    client.publish(TOPIC, payload, qos=0, retain=True)
    print(f"Publicat: {payload}")

def on_connect(client, userdata, flags, rc, properties=None):
    if rc == 0:
        print("Connectat al broker")
        client.publish(LWT_TOPIC, "online", qos=1, retain=True)
    else:
        print(f"Error de connexió: {rc}")

client.on_connect = on_connect
client.connect(BROKER, PORT, 60)
client.loop_start()

try:
```



```

while True:
    publicar_temperatura()
    time.sleep(5)
except KeyboardInterrupt:
    print("Aturant...")
    client.publish(LWT_TOPIC, "offline", qos=1, retain=True)
    client.loop_stop()
    client.disconnect()

```

Aquest script, executat des de qualsevol màquina amb Python i paho-mqtt, ens permet provar tot el sistema sense sensors reals. Quan la RPi arribi, l'executarem des d'allà mateix com a test d'integració.

12.10 Esquema de la conversa MQTT

Esquema Mermaid (text):

```

sequenceDiagram
    participant S as Sensor
    participant B as Broker Mosquitto
    participant T as Telegraf
    participant N as Node-RED
    participant G as Grafana

    S->>B: CONNECT (amb LWT)
    B-->>S: CONNACK
    S->>B: PUBLISH hort/zona1/temp 23.5 (retained)
    S->>B: PUBLISH hort/zona1/estat online (retained)
    B->>T: rep tots els missatges publicats
    B->>N: subscripció a hort/#
    N->>N: processa
    G->>B: subscripció a hort/zona1/temp
    B->>G: rep el valor retained

```

12.11 Errors habituals

Error 1: oblidar el username/password. Síntoma: el client no es connecta. Solució: configurar l'autenticació correctament tant al client com al broker.

Error 2: subscriure's a un topic massa ampli. Síntoma: el client rep tants missatges que satura la xarxa. Solució: subscriure's només al que necessitem, fer servir wildcards amb compte.

Error 3: usar QoS 2 per a tot. Síntoma: el sistema va lent, el broker es satura. Solució: QoS 0 o 1 per a sensors, QoS 1 o 2 només per a comandes.

Error 4: no marcar els missatges de "estat" com a retained. Síntoma: quan un client es connecta, no rep l'estat actual del sensor. Solució: usar retained per a l'últim valor conegut.

Error 5: dissenyar un esquema de topics inconsistent. Síntoma: alguns sensors publiquen a un esquema, d'altres a un altre, i Grafana mostra gràfiques incompletes. Solució: documentar l'esquema al README del projecte i fer-lo complir per tothom.

12.12 Bones pràctiques

1. **Esquema de topic clar i consistent.** Documentar-lo al README.
2. **Retained per a l'últim valor.** Especialment útil per a sensors.
3. **LWT per saber l'estat.** Permet detectar sensors morts.

4. **QoS adequat.** QoS 0 per a telemetria periòdica, QoS 1 o 2 per a comandes.
5. **Autenticació i ACLs.** Encara que la xarxa sigui privada.
6. **Payload compacte.** JSON és còmode, però podem optimitzar amb formats binaris (CBOR, MessagePack) si cal.
7. **Logs al broker.** Mosquitto pot guardar tots els missatges; activem-ho només quan calgui depurar.

12.13 Resum

Hem après què és MQTT, per què s'usa a IoT en comptes d'HTTP, com funciona el model publicació-subscripció, què són els topics, els wildcards, el QoS, els retained messages i el Last Will and Testament. Hem vist un exemple complet de sensor simulat en Python, i hem après les bones pràctiques i els errors habituals. En el proper capítol instal·larem Mosquitto al BernatLab i el configurarem amb autenticació i ACLs.

12.14 Exercicis pràctics

1. Instal·la `mosquitto-clients` al teu PC: `apt install mosquitto-clients` o `brew install mosquitto`.
2. Connecta't a un broker MQTT públic de prova (com `test.mosquitto.org`) i publica un missatge: `mosquitto_pub -h test.mosquitto.org -t bernatlab/test -m "hola"`.
3. En una altra terminal, subscriu-te al mateix topic: `mosquitto_sub -h test.mosquitto.org -t "bernatlab/#" -v`. Publica un altre missatge des de la primera terminal i observa'l.
4. Escriu un petit script Python amb `paho-mqtt` que publiqui un missatge cada 3 segons durant 30 segons.
5. Experimenta amb wildcards: subscriu-te a `sensors/+temperature` i a `sensors/#`, i mira la diferència.
6. Llegeix l'especificació oficial de MQTT (3.1.1 o 5.0) si vols aprofundir. Comença per la secció "Architecture".

Comandes útils:

```
# Instal·lar clients
apt install mosquitto-clients

# Publicar
mosquitto_pub -h BROKER -t TOPIC -m MISSATGE

# Subscriure
mosquitto_sub -h BROKER -t TOPIC -v

# Amb QoS i retain
mosquitto_pub -h BROKER -t TOPIC -m MISSATGE -q 1 -r
```

Paraules clau: **MQTT, broker, publisher, subscriber, topic, wildcard, QoS, retained, LWT, clean session, pub/sub, Mosquitto, paho-mqtt, IoT, telemetria, sensors, payload.**

Capítol 13 — Mosquitto al BernatLab

****"Un bon broker és un broker que no es nota. Si tot funciona, és que està fent la seva feina."***

13.1 Què és Mosquitto

Eclipse Mosquitto és un broker MQTT de codi obert, lleuger, ràpid i àmpliament utilitzat. Va ser creat per Roger Light el 2010, mantingut per la Eclipse Foundation, i és una de les implementacions de referència del protocol MQTT.

Mosquitto destaca per:

- **Lleugeresa:** el binari del broker ocupa pocs megabytes i consumeix poca RAM.
- **Velocitat:** pot gestionar desenes de milers de missatges per segon en màquines modestes.
- **Compliment del protocol:** suporta MQTT 3.1.1 i MQTT 5.0.
- **Seguretat:** autenticació per usuari/contrasenya, ACLs, TLS opcional.
- **Bridge:** pot connectar-se a altres brokers, permetent arquitectures distribuïdes.
- **Comunitat:** una de les comunitats més actives del món MQTT.

Al BernatLab és la opció natural: és oficial, està mantingut, i té una imatge Docker oficial que ens permet desplegar-lo fàcilment.

13.2 Instal·lació al BernatLab

Mosquitto es desplega com un contenidor Docker amb la imatge oficial `eclipse-mosquitto`. La configuració la farem amb tres fitxers:

- `mosquitto.conf`: configuració principal del broker.
- `passwordfile`: base de dades d'usuaris i contrasenyes.
- `aclfile`: llistes de control d'accés.

Estructura de carpetes

A `/home/bernat/homelab/`:

```
homelab/
├── docker-compose.yml
├── stacks/
│   └── iot/
│       ├── docker-compose.yml
│       ├── mosquitto.conf
│       ├── passwordfile
│       └── aclfile
└── data/
    └── mosquitto/      (volum persistent)
```

Definició al docker-compose.yml

```
services:
  mosquitto:
    image: eclipse-mosquitto:2.0
    container_name: mosquitto
    restart: unless-stopped
    ports:
      - "1883:1883"      # MQTT
      - "9001:9001"      # WebSockets (opcional)
    volumes:
      - ./mosquitto.conf:/mosquitto/config/mosquitto.conf:ro
      - ./passwordfile:/mosquitto/config/passwordfile:ro
      - ./aclfile:/mosquitto/config/aclfile:ro
      - /home/bernat/homelab/data/mosquitto:/mosquitto/data
      - /home/bernat/homelab/data/mosquitto/log:/mosquitto/log
```

Primera configuració

El fitxer `mosquitto.conf` bàsic:

```
# Configuració del broker Mosquitto per al BernatLab

# Escoltem a totes les interfícies (Tailscale ho xifra tot)
listener 1883
allow_anonymous false

# Persistència
persistence true
persistence_location /mosquitto/data/

# Logs
log_dest file /mosquitto/log/mosquitto.log
log_type error
log_type warning
log_type notice
log_type information
connection_messages true
log_timestamp true

# Autenticació
password_file /mosquitto/config/passwordfile
acl_file /mosquitto/config/aclfile

# WebSockets (opcional, útil per a clients web)
listener 9001
protocol websockets
```

Detalls a notar:

- **`listener 1883`**: el broker escolta al port estàndard MQTT.
- **`allow_anonymous false`**: cap connexió sense usuari/contrasenya.
- **`persistence true`**: el broker desa els missatges retained i les subscripcions, de manera que si es reinicia, no es perd res.
- **`log_dest file`**: els logs van a un fitxer, accessible al bind mount.
- **`password_file` i `acl_file`**: apunten als fitxers de seguretat.

13.3 Crear usuaris i contrasenyes

Mosquitto emmagatzema usuaris i contrasenyes en un format propi, amb hashing. La comanda per crear-los és:

```
mosquitto_passwd -c passwordfile USUARI
```

-c crea el fitxer nou. Ens demanarà la contrasenya dues vegades. Per afegir un altre usuari al fitxer existent:

```
mosquitto_passwd -b passwordfile USUARI CONTRASENYA
```

-b ens permet passar la contrasenya com a argument (útil en scripts). Compte amb la seguretat: la contrasenya quedarà a la història de la shell.

Al BernatLab crearem aquests usuaris inicials:

Usuari	Contrasenya	Propòsit
`bernat`	(força)	Administrador, pot fer tot
`sensor-temp-zona1`	(força)	Publica a `hort/zona1/temp/*`
`sensor-hum-zona1`	(força)	Publica a `hort/zona1/humitat/*`
`telegraf`	(força)	Llegeix tot per a InfluxDB
`nodered`	(força)	Llegeix i publica alertes
`grafana`	(força)	Llegeix alguns topics per a alertes

Totes les contrasenyes les guardarem al fitxer `.env` del projecte, que és al `.gitignore` (Capítol 9 del Mòdul 1). Al `passwordfile` hi aniran les contrasenyes amb hash.

13.4 Llistes de control d'accés (ACLs)

Les ACLs defineixen, per a cada usuari, a quins topics pot subscriure's i a quins pot publicar. La sintaxi és:

```
# Comentaris
user USUARI
topic [read|write|readwrite] PATRÓ

# 0 per defecte, aplicable a tothom
topic [read|write|readwrite] PATRÓ
```

L'ordre és important: les primeres línies tenen prioritats. Si un usuari té múltiples regles, s'aplica la primera que coincideix.

Exemple d'`aclfile` per al BernatLab:

```
# =====
# ACLs per al BernatLab
# =====

# L'administrador bernal té accés total
user bernal
topic readwrite #

# Els sensors de temperatura publiquen a la seva zona
```

```

user sensor-temp-zona1
topic write hort/zona1/temperatura/+
topic write hort/zona1/estat
topic read hort/zona1/control/+

# Els sensors d'humitat publiquen a la seva zona
user sensor-hum-zona1
topic write hort/zona1/humitat/+
topic write hort/zona1/estat
topic read hort/zona1/control/+

# Telegraf llegeix tot el que hi ha a hort/
user telegraf
topic read hort/#

# Node-RED llegeix tot i pot publicar alertes
user nodered
topic read #
topic write alertes/#
topic write hort/control/#

# Grafana llegeix alguns topics per alertes
user grafana
topic read hort/zona1/+
topic read alertes/#

```

Aquest esquema garanteix que cada usuari només pot fer el que li toca. Si un sensor és compromès, l'atacant només pot publicar als seus topics, no pas a tot el sistema.

Bona pràctica: noms d'usuari únics per dispositiu

Cada sensor físic ha de tenir el seu propi usuari i contrasenya. No compartir credencials entre dispositius. Això ens permet revocar l'accés d'un sensor concret sense afectar els altres, i ens permet rastrejar quin sensor ha publicat cada missatge als logs.

13.5 Posar en marxa

Un cop creats els fitxers, podem aixecar el servei:

```

cd /home/bernat/homelab/stacks/iot
docker compose up -d mosquito
docker compose logs mosquito

```

Si tot ha anat bé, veurem línies com:

```

mosquitto | 1717823400: mosquitto version 2.0.18 starting
mosquitto | 1717823400: Config loaded from /mosquitto/config/mosquitto.conf
mosquitto | 1717823400: Opening ipv4 listen socket on port 1883
mosquitto | 1717823400: Opening ipv6 listen socket on port 1883
mosquitto | 1717823400: mosquitto ready to accept connections

```

Per comprovar que escolta:

```

docker ps
# Ha de mostrar el contenidor mosquito amb status "Up"
ss -tulnp | grep 1883
# Ha de mostrar el port 1883 escoltant

```

13.6 Provar el broker

Un cop en marxa, podem provar-lo des de qualsevol màquina de la xarxa Tailscale.

Provar amb mosquitto_sub

En una terminal, ens subscriurem a tot:

```
mosquitto_sub -h 100.115.134.76 -t "#" -v \  
-u bernat -P LACONTRASENYA
```

Hem de veure els missatges que es publiquin, amb el format `topic payload`.

Provar amb mosquitto_pub

En una altra terminal, publiquem un missatge de prova:

```
mosquitto_pub -h 100.115.134.76 \  
-t "test/bernatlab" \  
-m "Hola des del BernatLab" \  
-u bernat -P LACONTRASENYA \  
-r
```

Si hem fet les coses bé, a la primera terminal veurem:

```
test/bernatlab Hola des del BernatLab
```

Provar amb retenció

Com que hem marcat el missatge amb `-r` (retained), si ens connectem ara sense publicar res, rebré el missatge de seguida:

```
mosquitto_sub -h 100.115.134.76 -t "test/bernatlab" -v \  
-u bernat -P LACONTRASENYA
```

Hauríem de rebre el missatge immediatament en connectar.

Provar ACLs

Iniciem sessió amb l'usuari del sensor i intentem accedir a un topic no permès:

```
# Això hauria de funcionar (el sensor pot publicar a la seva zona)  
mosquitto_pub -h 100.115.134.76 \  
-t "hort/zona1/temperatura/aire" \  
-m "23.5" \  
-u sensor-temp-zona1 -P LACONTRASENYADELSENSOR
```

```
# Això hauria de FALLAR (el sensor no pot accedir a una altra zona)  
mosquitto_pub -h 100.115.134.76 \  
-t "hort/zona2/temperatura/aire" \  
-m "20.0" \  
-u sensor-temp-zona1 -P LACONTRASENYADELSENSOR
```

Si el broker està ben configurat, la primera ordre funciona i la segona retorna un error. Si tot funciona, tenim un broker segur.

13.7 Monitoratge

Mosquitto pot ser monitorat per Uptime Kuma afegint un monitor de tipus **TCP Port** al port 1883. Si el broker cau, Uptime Kuma ens avisarà per Telegram.

A més, podem afegir un monitor més avançat amb un script que envii un missatge MQTT i comprovi que arriba. Però això és opcional.

Pel que fa a mètriques internes, Mosquitto ofereix un sistema de `$$SYS` topics que contenen estadístiques del broker en temps real. Per accedir-hi:

```
mosquitto_sub -h 100.115.134.76 -t '$SYS/#' -v -u bernat -P LACONTRASENYA
```

Veurem coses com:

```
$SYS/broker/version mosquitto version 2.0.18
$SYS/broker/uptime 1234 seconds
$SYS/broker/clients/connected 5
$SYS/broker/messages/received 1234
$SYS/broker/messages/sent 1234
$SYS/broker/load/messages/received/5min 2.5
```

Això és or pur per a depurar i entendre com es comporta el sistema. Si volem, podem connectar Grafana a InfluxDB i visualitzar aquestes mètriques, però això és opcional.

13.8 WebSockets: per a clients web

El port 9001 (WebSockets) ens permet que aplicacions web es connectin directament al broker MQTT sense necessitat d'un servidor intermedi. Això és útil si volem que la web Hort Osona consumeixi directament dades en temps real.

Tanmateix, **al BernatLab, la web Hort Osona consumirà les dades a través de l'API REST** (Capítols 20 i 21), no pas directament per MQTT. Per tant, podem desactivar WebSockets si volem, o deixar-lo activat per si el necessitem més endavant.

Per desactivar-lo, eliminem les dues línies de `listener 9001` i `protocol websockets` del `mosquitto.conf`.

13.9 Bridge entre brokers (avançat)

Mosquitto pot actuar com a **bridge** entre dos brokers. Això ens permetria, per exemple:

- Un broker a la Raspberry del BernatLab.
- Un altre broker a la Raspberry del Camp (quan el sistema creixi).

El bridge replica missatges entre ells. Per configurar-lo, afegim al `mosquitto.conf`:

```
connection camp
address camp.bernatlab.local:1883
username bernat
password CONTRASENYA
topic hort/camp/# both 0
topic # out 0
```

Això replica tots els missatges d'`hort/camp/#` entre el broker del BernatLab i el del camp. Però això és una optimització que farem molt més endavant, quan tinguem el sistema bàsic funcionant.

13.10 Configuració avançada: limitació de recursos

Mosquitto permet limitar l'ús de recursos per evitar que un client maliciós o un error de programació sature el broker:

```
# Màxim nombre de connexions
max_connections 100

# Mida màxima dels payloads (per defecte, sense límit)
max_packet_size 1024

# Màxim de missatges en cua per a un client amb QoS > 0
```



```
max_queued_messages 1000
```

```
# Temps de vida dels missatges en cua  
max_queued_bytes 65536
```

Al BernatLab, com que tenim pocs sensors i tot és en xarxa privada, podem deixar els valors per defecte. Però és bo saber que existeixen.

13.11 Logs: què mirar quan alguna cosa falla

Els logs de Mosquitto són la primera pista quan alguna cosa no va bé. Per defecte, els tenim a `/home/bernat/homelab/data/mosquitto/log/mosquitto.log`. Cada línia porta un timestamp i un missatge.

Tipus de missatges habituals:

- **Error**: alguna cosa ha fallat (per exemple, un client ha intentat accedir a un topic prohibit).
- **Warning**: possible problema, però el sistema continua.
- **Notice**: informació general (connexions, desconnexions).
- **Information**: detalls addicionals.

Per depurar un problema concret, podem augmentar el nivell de log afegint al `mosquitto.conf`:

```
log_type all  
log_priority debug
```

I reiniciar el contenidor. Compte: això genera molt de volum. Per a ús normal, `log_type error warning notice information` és suficient.

13.12 Còpies de seguretat

Què hem de copiar de Mosquitto?

- El fitxer `mosquitto.conf`.
- El fitxer `passwordfile`.
- El fitxer `aclfile`.
- Opcionalment, els logs (per a anàlisi posterior).

La carpeta `/mosquitto/data` (on es guarden els missatges retained i les subscripcions) és regenerable: si la perdem, els sensors tornaran a publicar i el sistema es referà. Per tant, no cal copiar-la estrictament, però és bona pràctica fer-ho per si de cas.

Al BernatLab, tots aquests fitxers viuen dins de `/home/bernat/homelab/stacks/iot/`, que ja està versionat amb Git. Les còpies periòdiques es fan seguint el procediment del Capítol 9 del Mòdul 1.

13.13 Actualització

Per actualitzar Mosquitto a una nova versió:

```
cd /home/bernat/homelab/stacks/iot  
docker compose pull mosquitto  
docker compose up -d mosquitto
```

Els fitxers de configuració solen ser compatibles entre versions 2.x, però sempre cal revisar les notes de versió a mosquitto.org/documentation.

13.14 Integració amb la resta del BernatLab

Un cop tenim Mosquitto funcionant, podem afegir serveis que el consumeixin:

- **Telegraf** (Capítol 16): s'hi subscriu i escriu les dades a InfluxDB.
- **Node-RED** (Capítols 17 i 18): s'hi subscriu, processa, i publica alertes.
- **Grafana** (Capítol 19): s'hi pot subscriure directament per a mètriques en temps real, tot i que és més eficient fer-ho via InfluxDB.
- **Scripts Python** puntuals: podem subscriure'ns des d'un script per depurar o fer experiments.

Tots aquests serveis s'autenticaran al broker amb els seus propis usuaris, cadascun amb els seus permisos d'ACL.

13.15 Esquema d'integració

Esquema Mermaid (text):

```
graph TB
    subgraph Sensors["Sensors (terreny)"]
        S1["Sensor temp"]
        S2["Sensor hum"]
        S3["Sensor llum"]
    end

    subgraph Mosquitto["Mosquitto (broker)"]
        M["Port 1883<br/>Autenticació<br/>ACLs"]
    end

    subgraph Consumidors["Consumidors"]
        T["Telegraf<br/>(llegeix hort/#)"]
        N["Node-RED<br/>(llegeix tot, escriu alertes)"]
        G["Grafana<br/>(llegeix hort/zona1/+)"]
        SCR["Scripts debug<br/>(llegeixen ad-hoc)"]
    end

    S1 --> M
    S2 --> M
    S3 --> M
    M --> T
    M --> N
    M --> G
    M --> SCR
```

13.16 Errors habituals

****Error 1: oblidar `allow_anonymous false`**.** Síntoma: qualsevol es pot connectar sense autenticació. Solució: posar `allow_anonymous false` i configurar usuaris.

Error 2: ACL massa permissiva. Síntoma: qualsevol pot accedir a qualsevol topic. Solució: aplicar el principi de mínim privilegi, un usuari per dispositiu.

Error 3: contrasenyes en text pla al passwordfile. Síntoma: si el fitxer es filtra, tothom veu les contrasenyes. Solució: `mosquitto_passwd` ja les emmagatzema amb hash (per defecte, SHA-512). Mai no editar-lo a mà.

Error 4: el port 1883 exposat a Internet. Síntoma: intents de connexió massius des d'arreu del món. Solució: mantenir el port 1883 només a la xarxa Tailscale. Si cal accedir des de fora, fer-ho sempre a través de Tailscale.

Error 5: no persistir la configuració. Síntoma: en actualitzar el contenidor, es perd la configuració. Solució: bind mounts per a `mosquitto.conf`, `passwordfile`, `aclfile`.

Error 6: només un usuari per a tots els sensors. Síntoma: no podem saber quin sensor ha fallat, i revocar-ne un afecta tots. Solució: un usuari per dispositiu.

13.17 Bones pràctiques

1. **Un usuari per dispositiu.** Això ens permet rastreig i revocació granular.
2. **ACLs estrictes.** Cada usuari només pot fer el que li toca.
3. **Contrasenyes llargues.** Mínim 16 caràcters, millor 20+.
4. **Bind mounts per a la configuració.** Que pugui ser versionada amb Git.
5. **Logs a disc.** Per poder analitzar incidents.
6. **Monitoratge amb Uptime Kuma.** Un monitor TCP al port 1883.
7. **TLS opcional, no necessari amb Tailscale.** Si mai traiem el broker de Tailscale, caldrà reconsiderar.
8. **Documentar l'esquema de topics.** Al README del projecte.
9. **Testejant les ACLs.** Assegurar-nos que els sensors no poden accedir a topics aliens.
10. **Auditar periòdicament.** Qui s'ha connectat, què ha publicat, quantes vegades.

13.18 Resum

Hem après a instal·lar i configurar Mosquitto al BernatLab: la configuració bàsica, la creació d'usuaris amb `mosquitto_passwd`, les ACLs per aplicar el principi de mínim privilegi, com provar que tot funciona, com monitorar el servei, i com resoldre els problemes habituals. En el proper capítol dissenyarem l'esquema de publicació dels sensors i veurem exemples de codi per simular i capturar dades.

13.19 Exercicis pràctics

1. Desplega Mosquitto al BernatLab amb la configuració que hem vist.
2. Crea un usuari administrador (`bernat`) i un usuari sensor (`sensor-test`).
3. Configura les ACLs perquè `sensor-test` només pugui publicar a `test/bernatlab/#`.
4. Prova de subscriure't a `test/#` amb l'usuari `bernat`.
5. Publica un missatge de prova i comprova que arriba.
6. Intenta publicar amb `sensor-test` a `altres/topic` i comprova que el broker rebutja la connexió/publicació.
7. Comprova els `$SYS` topics per veure les estadístiques del broker.

8. Afegeix un monitor a Uptime Kuma per al port 1883.

9. Mira els logs de Mosquitto i explica què signifiquen les primeres 10 línies.

Comandes útils:

```
# Crear usuaris
mosquitto_passwd -c passwordfile bernat
mosquitto_passwd -b passwordfile sensor-test password

# Provar
mosquitto_sub -h 100.115.134.76 -t "#" -v -u bernat -P CONTRASENYA
mosquitto_pub -h 100.115.134.76 -t test/bernatlab -m "hola" -u bernat -P CONTRASENYA -r

# Estadístiques
mosquitto_sub -h 100.115.134.76 -t '$SYS/#' -v -u bernat -P CONTRASENYA

# Logs
docker compose logs mosquitto
tail -f /home/bernat/homelab/data/mosquitto/log/mosquitto.log
```

Paraules clau: **Mosquitto, broker, eclipse-mosquitto, passwordfile, ACL, mosquitto_passwd, listener, allow_anonymous, persistence, retained, \$SYS, QoS, Tailscale, autenticació, principi de mínim privilegi, un-usuari-per-dispositiu.**

Capítol 14 — Publicar dades: els sensors

""Un sensor ben dissenyat és un sensor que sabem què està fent, des d'on, i amb quin propòsit. La clau no és el sensor, sinó l'esquema.""

14.1 El punt de vista del sensor

En el capítol anterior hem vist el broker des del costat del servidor. Ara posem-nos al costat del sensor.

Un sensor, en el context del BernatLab, és un petit dispositiu electrònic que mesura alguna propietat física (temperatura, humitat, llum, etc.) i l'envia al broker MQTT. Pot ser:

- Un **microcontrolador** (ESP32, ESP8266, Arduino Nano Connect, Raspberry Pi Pico W) connectat a un sensor físic (DHT22, BME280, SHT31, etc.).
- Una **Raspberry Pi** amb un sensor connectat per GPIO o USB.
- Un **dispositiu comercial** que ja parla MQTT (com els sensors Xiaomi Mi Flora que ja tenim al projecte Hort Osona).
- Un **script Python** en execució al servidor, simulant un sensor per a proves.

Tots ells, des del punt de vista del protocol, són iguals: clients MQTT que es connecten al broker, es subscriuen (opcionalment) i publiquen dades.

14.2 Què hem de tenir clar abans de connectar un sensor

Tres coses:

1. **L'esquema de topics.** Com es diran els topics? Quin patró seguir?
2. **El format del payload.** Com es codifiquen les dades? JSON? Binari? Text pla?
3. **La freqüència de publicació.** Cada quan publiquem? Massa sovint, saturarem; massa poc, perdrem detall.

Aquestes tres decisions es prenen una sola vegada, al principi, i tots els sensors les han de respectar. Per això és fonamental documentar-les al README del projecte.

14.3 L'esquema de topics de Hort Osona

Per a Hort Osona, proposem el següent esquema:

```
hort/{zona}/{tipus_sensor}/{identificador}
hort/{zona}/estat
hort/control/{zona}/{actuador}
alertes/{tipus}
```

Exemples concrets:

- `hort/zona-tomateres/temperatura/aire`: temperatura ambient a la zona de les tomateres.
- `hort/zona-tomateres/temperatura/sol-10cm`: temperatura del sol a 10 cm de profunditat.
- `hort/zona-enciams/humitat/sol-20cm`: humitat del sol a 20 cm a la zona dels enciams.

- `hort/zona-enciams/lluminositat/par`: lluminositat PAR (radiació fotosintèticament activa).
- `hort/zona-tomateres/estat`: estat del node sensor (online/offline).
- `hort/control/zona-enciams/reg`: comanda per obrir/tancar el reg dels enciams.
- `alertes/gelada`: alerta de gelada imminent.
- `alertes/sol-sec`: alerta de sol massa sec.

Aquest esquema és extensible: quan afegim un nou sensor, triem una zona, un tipus, i un identificador. Si el sensor té múltiples lectures (com un BME280, que mesura temperatura, humitat i pressió), publica a tres topics:

```
hort/zona-x/temperatura/aire
hort/zona-x/humitat/relativa
hort/zona-x/pressio/atmosferica
```

14.4 El format del payload

Tenim diverses opcions. Vegem-les:

Opció A: text pla (un sol valor)

```
mosquitto_pub -t hort/zona1/temperatura/aire -m "23.5"
```

Avantatges: molt simple, ocupació mínima. Inconvenients: no podem afegir metadades (unitats, qualitat, etc.).

Opció B: JSON

```
{
  "valor": 23.5,
  "unitat": "graus_C",
  "ts": 1717823400,
  "qualitat": 95
}
```

Avantatges: extensible, llegible, autocontingut. Inconvenients: ocupa més bytes (entre 30 i 100 bytes típicament).

Opció C: JSON minimalista

```
{"v": 23.5, "u": "C", "t": 1717823400}
```

Avantatges: ocupació reduïda, encara extensible. Inconvenients: menys llegible.

Opció D: binari (CBOR, MessagePack)

Avantatges: molt compacte. Inconvenients: no llegible, cal una biblioteca per codificar/decodificar.

Recomanació per al BernatLab

Per a sensors amb comunicació Wi-Fi (que tenen ample de banda de sobres), **JSON complet** és la millor opció. Per a sensors amb LoRa (que tenen limitacions d'ample de banda), JSON minimalista o fins i tot CBOR pot ser necessari.

Al BernatLab, començarem amb **JSON complet** perquè la claredat supera l'estalvi de bytes:

```
{
  "valor": 23.5,
```

```
"unitat": "graus_C",
"ts": 1717823400
}
```

El camp `ts` (timestamp en segons Unix) ens permet correlacionar les dades encara que el sensor no estigui sincronitzat amb l'hora del servidor. Alternativament, podem ometre'l i deixar que InfluxDB assigni el timestamp quan rep el missatge (que és el que farem servir per defecte, per simplicitat).

14.5 Freqüència de publicació

Cada quan ha de publicar un sensor? Depèn del que mesurem:

Tipus de mesura	Freqüència recomanada
Temperatura ambient	1 minut
Humitat relativa	1 minut
Temperatura del sòl	5 minuts
Humitat del sòl	5-10 minuts
Lluminositat	5-15 minuts (pot variar ràpid)
Pluja	immediat (esdeveniment)
Vent	1-5 minuts

A l'Hort Osona, cada zona tindrà un node sensor que publicarà:

- Temperatura ambient: cada 60 segons.
- Humitat relativa: cada 60 segons.
- Temperatura del sòl: cada 300 segons (5 min).
- Humitat del sòl: cada 300 segons.
- Lluminositat: cada 300 segons.
- Estat del node: cada 60 segons (canvia a "online"/"offline").

14.6 Exemple: simulant un sensor en Python

Aquest script és el que farem servir per provar tot el sistema sense tenir el hardware. Simula un node sensor BME280 (temperatura, humitat, pressió) amb valors aleatoris realistes:

```
"""
simula_bme280.py
Simula un node sensor BME280 que publica a MQTT.
"""

import json
import random
import time
import paho.mqtt.client as mqtt

BROKER = "100.115.134.76"
PORT = 1883
USERNAME = "sensor-bme-zona1"
PASSWORD = "elmeupassword"
```

```

ZONA = "zona-tomateres"
LWT_TOPIC = f"hort/{ZONA}/estat"

client = mqtt.Client(client_id=f"simula-bme-{ZONA}", clean_session=True)
client.username_pw_set(USERNAME, PASSWORD)
client.will_set(LWT_TOPIC, payload="offline", qos=1, retain=True)

def publicar(topic, valor, unitat):
    payload = json.dumps({
        "valor": round(valor, 2),
        "unitat": unitat,
        "ts": int(time.time()),
    })
    client.publish(topic, payload, qos=0, retain=True)
    print(f" → {topic}: {payload}")

def on_connect(client, userdata, flags, rc, properties=None):
    if rc == 0:
        print(f"[{ZONA}] Connectat al broker")
        client.publish(LWT_TOPIC, "online", qos=1, retain=True)
    else:
        print(f"[{ZONA}] Error de connexió: {rc}")

client.on_connect = on_connect
client.connect(BROKER, PORT, 60)
client.loop_start()

temperatura_base = 22.0
humitat_base = 60.0
pressio_base = 1013.0

try:
    while True:
        # Simulació amb soroll gaussià
        temperatura = temperatura_base + random.gauss(0, 0.5)
        humitat = max(20, min(100, humitat_base + random.gauss(0, 2)))
        pressio = pressio_base + random.gauss(0, 0.3)

        publicar(f"hort/{ZONA}/temperatura/aire", temperatura, "graus_C")
        publicar(f"hort/{ZONA}/humitat/relativa", humitat, "%")
        publicar(f"hort/{ZONA}/pressio/atmosferica", pressio, "hPa")

        time.sleep(60)
except KeyboardInterrupt:
    print(f"[{ZONA}] Aturant...")
    client.publish(LWT_TOPIC, "offline", qos=1, retain=True)
    client.loop_stop()
    client.disconnect()

```

Aquest script, executat al servidor, ens permet:

- Validar que el broker accepta connexions.
- Veure com es comporten les ACLs.
- Generar dades per provar Telegraf, InfluxDB, Node-RED i Grafana.
- Estalviar-nos el viatge al camp mentre posem a punt el sistema.

14.7 Exemple: ESP32 amb MicroPython

Per als sensors reals al terreny, un microcontrolador com l'ESP32 és la opció natural. Pot funcionar amb piles durant setmanes, té Wi-Fi integrat, i adafruit/MicroPython fan que programar-lo sigui relativament fàcil.

Exemple de codi MicroPython per a un ESP32 amb un sensor DHT22:

```
"""
ESP32 + DHT22 + MQTT
Llegeix temperatura i humitat cada 60 segons i publica via MQTT.
"""

import network
import time
import dht
from machine import Pin
from umqtt.simple import MQTTClient

WIFI_SSID = "elmeuSSID"
WIFI_PASSWORD = "elmeupassword"

BROKER = "100.115.134.76"
PORT = 1883
CLIENT_ID = "esp32-dht-zona1"
USERNAME = "sensor-dht-zona1"
PASSWORD = "elmeupassword"

ZONA = "zona-tomateres"
LWT_TOPIC = f"hort/{ZONA}/estat"

# Connexió Wi-Fi
wlan = network.WLAN(network.STA_IF)
wlan.active(True)
wlan.connect(WIFI_SSID, WIFI_PASSWORD)
while not wlan.isconnected():
    print("Connectant a Wi-Fi...")
    time.sleep(1)
print("Wi-Fi connectat:", wlan.ifconfig())

# Sensor DHT22 al pin 4
sensor = dht.DHT22(Pin(4))

# Client MQTT
def publicar(client, topic, valor, unitat):
    import ujson
    payload = ujson.dumps({
        "valor": round(valor, 2),
        "unitat": unitat,
    })
    client.publish(topic, payload, retain=True)

client = MQTTClient(CLIENT_ID, BROKER, PORT, USERNAME, PASSWORD)
client.set_last_will(LWT_TOPIC, "offline", retain=True, qos=1)
client.connect()
client.publish(LWT_TOPIC, "online", retain=True, qos=1)
print("MQTT connectat")

try:
    while True:
        try:
            sensor.measure()
            temperatura = sensor.temperature()
            humitat = sensor.humidity()
```

```

        publicar(client, f"hort/{ZONA}/temperatura/aire", temperatura, "graus_C")
        publicar(client, f"hort/{ZONA}/humitat/relativa", humitat, "%")
        print(f"Temp: {temperatura} °C, Hum: {humitat} %")
    except OSError as e:
        print("Error de lectura:", e)
        time.sleep(60)
except KeyboardInterrupt:
    client.publish(LWT_TOPIC, "offline", retain=True, qos=1)
    client.disconnect()

```

Aquest codi és un punt de partida. En el Mòdul 3 (LoRa), veurem variants que usen SX1262 en lloc de Wi-Fi.

14.8 Exemple: integració amb sensors Xiaomi Mi Flora

Hort Osona ja té sensors **Xiaomi Mi Flora** (o "Flower Care"), que mesuren temperatura, humitat del sòl, lluminositat, fertilitat i salinitat. Aquests sensors es comuniquen per Bluetooth, no pas per Wi-Fi. Per integrar-los al BernatLab, tenim dues opcions:

Opció 1: Home Assistant + bridge MQTT

Home Assistant té una integració nativa per a Mi Flora, i pot publicar les dades a un broker MQTT. Però afegir Home Assistant al BernatLab seria excessiu per a un sol tipus de sensor.

Opció 2: miflora-mqtt-daemon

Hi ha un projecte de codi obert, **miflora-mqtt-daemon**, que escolta els sensors Mi Flora per Bluetooth i publica les dades directament a un broker MQTT. És la solució més neta.

El projecte es pot trobar a GitHub: github.com/ThomDietrich/miflora-mqtt-daemon.

Configuració típica:

```

# /etc/miflora-mqtt-daemon/config.yaml
mqtt:
  host: 100.115.134.76
  port: 1883
  username: miflora
  password: elmeupassword
  topic_prefix: hort/miflora
  availability_topic: hort/miflora/estat

daemon:
  poll_interval: 300 # 5 minuts
  period: 600
  bluetooth_scan_timeout: 20

miflora:
  cache: /tmp/miflora-cache
  enabled: true
  report_unknown: false
  sensors:
    - mac: C4:7C:8D:6A:XX:XX
      name: zona-tomateres
    - mac: C4:7C:8D:6A:YY:YY
      name: zona-enciams

```

Un cop configurat, el daemon:

1. Busca els sensors per Bluetooth cada 5 minuts.

2. Llegeix les dades.

3. Publica a MQTT amb el patró:

```
hort/miflora/zona-tomateres/temperature 23.5
hort/miflora/zona-tomateres/moisture 45
hort/miflora/zona-tomateres/light 12000
hort/miflora/zona-tomateres/conductivity 800
hort/miflora/zona-tomateres/battery 85
```

Això és exactament el que volem: les dades dels sensors al broker MQTT, llestes per ser processades per Telegraf i emmagatzemades a InfluxDB.

14.9 Patrons de publicació

Hi ha tres patrons habituals:

Patró 1: un missatge per mesura

El sensor publica cada mesura a un topic separat:

```
hort/zona1/temperatura/aire → 23.5
hort/zona1/humitat/relativa → 60
hort/zona1/pressio/atmosferica → 1013
```

Avantatges: fàcil de subscriure individualment, fàcil d'indexar. **Inconvenients:** més missatges, més overhead.

Patró 2: un missatge amb múltiples mesures

El sensor publica un sol missatge amb totes les mesures:

```
hort/zona1/measure → {"temperatura": 23.5, "humitat": 60, "pressio": 1013}
```

Avantatges: menys missatges, menys overhead. **Inconvenients:** cal parsejar JSON, no es pot subscriure a una sola mesura.

Patró 3: híbrid

El sensor publica cada mesura individualment, però en un sol "cicle" amb múltiples publicacions. Per exemple, un node BME280 publica:

```
hort/zona1/temperatura/aire → 23.5
hort/zona1/humitat/relativa → 60
hort/zona1/pressio/atmosferica → 1013
hort/zona1/estat → online
```

En quatre publicacions separades, dins d'un bucle. Aquest és el patró que farem servir al BernatLab perquè combina les avantatges dels altres dos.

14.10 Estructura del payload en detall

Per a cada topic, el payload serà un objecte JSON amb tres camps:

```
{
  "valor": 23.5,
  "unitat": "graus_C",
  "ts": 1717823400
}
```

- **valor**: el valor numèric de la mesura. Pot ser un enter o un decimal.
- **unitat**: una cadena curta que indica la unitat (graus_C, %, hPa, etc.).
- **ts**: el timestamp Unix en segons. Opcional, però recomanable.

Si volem afegir camps extra (qualitat de la mesura, identificador del sensor, etc.), ho podem fer sense trencar el patró:

```
{
  "valor": 23.5,
  "unitat": "graus_C",
  "ts": 1717823400,
  "sensor_id": "bme280-zona1",
  "qualitat": 95
}
```

Telegraf i InfluxDB ignoraran els camps que no coneguin, de manera que el sistema és tolerant a extensions.

14.11 Ús de retained

Quins missatges han de ser retained?

Tots els que representen l'últim valor conegut d'un sensor. Això inclou:

- Temperatura, humitat, pressió, lluminositat, etc.
- L'estat del node (online/offline).

No han de ser retained:

- Les alertes (ja que volem que es processin, no que es quedin al broker).
- Els missatges de control (comandes a actuadors).

14.12 Ús del Last Will and Testament

Cada node sensor ha de registrar un LWT al topic d'estat:

```
client.will_set(
    topic=f"hort/{ZONA}/estat",
    payload="offline",
    qos=1,
    retain=True
)
```

Això garanteix que, si el node es desconnecta de manera no esperada (per pèrdua de Wi-Fi, bateria esgotada, etc.), el broker publicarà automàticament "offline" al topic d'estat. Si el node es desconnecta correctament (per exemple, amb `client.disconnect()`), hauria de publicar "offline" explícitament abans de desconnectar.

Node-RED pot subscriure's a aquests topics i generar una alerta si un sensor porta més de X minuts sense publicar.

14.13 Consideracions de seguretat

Cada node sensor ha de tenir:

- El seu propi **usuari i contrasenya** a Mosquitto.
- Les seves pròpies **ACLs** que limiten els topics als quals pot accedir.
- Connexió per **Tailscale** (si escau) o per una xarxa aïllada.

Si un node és compromès (per exemple, per un atac físic), l'atacant només podrà:

- Publicar als topics permesos.
- Rebre els topics als quals està subscrit.

Per tant, és fonamental que les ACLs siguin estrictes.

14.14 Proves d'integració

Un cop tenim el sensor (real o simulat) publicant, hem de poder verificar que:

1. El broker accepta les connexions.
2. Les dades arriben al topic correcte.
3. El payload té el format esperat.
4. El LWT funciona correctament (disconnectem el node i veiem com canvia l'estat).

Per fer aquestes proves, podem fer servir:

```
# Subscriure's a tot el que publica el node
mosquitto_sub -h 100.115.134.76 \
  -t "hort/zona1/#" -v \
  -u bernat -P CONTRASENYA
```

```
# En una altra terminal, executar el sensor
```

```
# En una tercera, comprovar $SYS
mosquitto_sub -h 100.115.134.76 \
  -t '$SYS/broker/clients/#' -v \
  -u bernat -P CONTRASENYA
```

Si tot és correcte, veurem un flux continu de missatges.

14.15 Quan falla un sensor: com reaccionar

Els sensors, al camp, fallen. La bateria s'esgota, la Wi-Fi es perd, l'electrònica es rovella. Hem d'estar preparats.

Detecció de fallada

Node-RED, al Capítol 18, aprendrà a:

- Detectar quan un sensor no ha publicat en X minuts.
- Enviar una alerta a Telegram.
- Publicar l'estat al topic `hort/zona1/estat` com a "stale".

Recuperació

Quan el sensor torna, simplement:

- Es reconnecta al broker.
- Publica "online" al LWT topic.
- Comença a enviar dades de nou.

Node-RED detecta la recuperació i ens avisa.

14.16 Exemple complet: node amb múltiples sensors

Un node sensor pot tenir múltiples sensors connectats. Per exemple, un node ESP32 amb un BME280 (temperatura, humitat, pressió) i un sensor de sòl capacitiu:

```
# Pseudocodi
import time
import bme280
import soil_sensor
import paho.mqtt.client as mqtt

client = mqtt.Client()
client.connect("100.115.134.76", 1883, 60)
client.will_set("hort/zona1/estat", "offline", qos=1, retain=True)
client.loop_start()

bme = bme280.BME280(i2c_bus=1, address=0x76)
soil = soil_sensor.Capacitive(pin=34)

while True:
    # BME280
    t, p, h = bme.read_compensated_data()
    publicar("hort/zona1/temperatura/aire", t/100, "graus_C")
    publicar("hort/zona1/humitat/relativa", h/1024, "%")
    publicar("hort/zona1/pressio/atmosferica", p/25600, "hPa")

    # Sensor de sòl
    hum_sol = soil.read()
    publicar("hort/zona1/humitat/sol-20cm", hum_sol, "%")

    # Estat
    client.publish("hort/zona1/estat", "online", qos=1, retain=True)

    time.sleep(60)
```

Aquest és el patró que farem servir a tots els nodes: lectures periòdiques, publicació a múltiples topics, manteniment de l'estat.

14.17 Esquema conceptual

Esquema Mermaid (text):

```
graph TB
    subgraph Node["Node sensor (ESP32)"]
        BME["BME280<br/>(T, H, P)"]
        SOL["Sensor sòl<br/>(humitat)"]
        LLUM["Sensor llum<br/>(PAR)"]
        MCU["Microcontrolador"]
    end

    subgraph Radio["Transmissió"]
        WIFI["Wi-Fi o LoRa"]
    end

    subgraph Broker["Broker Mosquitto"]
```

```

        M["1883<br/>ACLs"]
    end

    BME --> MCU
    SÒL --> MCU
    LLUM --> MCU
    MCU --> WIFI
    WIFI --> M

    M --> T["Topic T"]
    M --> H["Topic H"]
    M --> P["Topic P"]
    M --> SH["Topic hum. sòl"]
    M --> L["Topic llum"]
    M --> E["Topic estat"]

```

14.18 Errors habituals

Error 1: esquema de topics inconsistent. Síntoma: alguns sensors publiquen a un esquema, d'altres a un altre, Grafana mostra gràfiques incompletes. Solució: documentar l'esquema al README i fer-lo complir.

Error 2: payload en format inconsistent. Síntoma: Telegraf no sap parsejar les dades, InfluxDB rebutja les insercions. Solució: validar el format amb JSON Schema o, com a mínim, amb un test.

Error 3: publicar massa sovint. Síntoma: el broker es satura, la xarxa es congestiona, la bateria dels sensors s'esgota ràpidament. Solució: ajustar la freqüència a la necessitat real.

Error 4: oblidar el LWT. Síntoma: no sabem quan un sensor ha caigut. Solució: configurar el LWT a tots els clients.

Error 5: no testear el payload. Síntoma: arriba un missatge buit, o amb un valor impossible (temperatura de 200 °C, humitat del 500%). Solució: validar les dades al sensor abans de publicar-les.

Error 6: compartir credencials entre sensors. Síntoma: un sensor compromès permet accedir a tots els altres. Solució: un usuari per dispositiu, com ja hem vist al Capítol 13.

14.19 Bones pràctiques

1. **Esquema de topics documentat al README.** Tots els participants l'han de conèixer.
2. **Format de payload consistent.** Validar-lo.
3. **Freqüència de publicació adequada.** Ni massa sovint, ni massa poc.
4. **LWT configurat a tots els clients.**
5. **Un usuari per dispositiu a Mosquitto.**
6. **ACLs estrictes.**
7. **Validació al sensor.** Si una lectura és impossible, no la publiquem.
8. **Logs locals al sensor.** Per depurar problemes al camp.
9. **Mode de test abans de desplegar.** Publicar a un topic de proves abans de passar a producció.
10. **Monitoratge extern.** Node-RED ha de poder detectar sensors que no publiquen.

14.20 Resum

Hem après com es connecten els sensors al broker MQTT, quin esquema de topics i quin format de payload són adequats per a Hort Osona, com simular sensors amb Python per desenvolupar sense hardware, i com integrar sensors comercials com els Xiaomi Mi Flora. Hem vist exemples reals per a ESP32 amb MicroPython, i hem après les bones pràctiques i els errors habituals. En el proper capítol veurem on i com es guarden aquestes dades: a InfluxDB.

14.21 Exercicis pràctics

1. Dibuixa, a mà, l'esquema de topics que faries servir per al teu hort o un altre sistema de sensors.
2. Escriu un script Python que simuli un node sensor amb tres lectures (temperatura, humitat, llum) i publiqui cada 10 segons.
3. Connecta't al broker del BernatLab i subscriu-te a [hort/#](#). Fes que el teu script publiqui durant 2 minuts i comprova que reps tots els missatges.
4. Afegeix una alerta al teu script: si la temperatura baixa de 5 °C, publica una alerta a [alertes/gelada](#).
5. Investiga com connectar un sensor Xiaomi Mi Flora al teu PC amb Linux i Bluetooth. Prova de llegir-ne les dades amb [gatttool](#) o [bluetoothctl](#).
6. Documenta l'esquema de topics al README del teu projecte.

Comandes útils:

```
# Subscriure's a tots els topics d'una zona
mosquitto_sub -h 100.115.134.76 -t "hort/zona1/#" -v -u bernat -P CONTRASENYA
```

```
# Publicar amb LWT
python3 -c "
import paho.mqtt.client as mqtt
c = mqtt.Client()
c.username_pw_set('bernat', 'CONTRASENYA')
c.will_set('hort/zona1/estat', 'offline', qos=1, retain=True)
c.connect('100.115.134.76', 1883, 60)
c.publish('hort/zona1/estat', 'online', qos=1, retain=True)
c.publish('hort/zona1/temperatura/aire', '{"valor": 23.5}', retain=True)
c.loop_forever()
"
```

Paraules clau: **sensor**, **ESP32**, **DHT22**, **BME280**, **Mi Flora**, **miflora-mqtt-daemon**, **payload JSON**, **LWT**, **retained**, **esquema de topics**, **simulació**, **ACL**, **un-usuari-per-dispositiu**, **MicroPython**, **paho-mqtt**, **integració de sensors**, **Hort Osona**, **validació de dades**.

Capítol 15 — InfluxDB: base de dades de sèries temporals

"Guardar dades d'un sensor és fàcil. Guardar-ne milions i poder-hi fer consultes ràpides és el que diferencia una base de dades de sèries temporals d'una base de dades normal."

15.1 Què és una base de dades de sèries temporals

Una **base de dades de sèries temporals** (TSDB, Time Series Database) és un tipus especialitzat de base de dades dissenyat per emmagatzemar i consultar **dades associades a marques de temps**. Exemples clars:

- Lectures de sensors cada minut.
- Preus d'una acció cada segon.
- Mètriques d'un servidor cada 5 segons.
- Temperatura d'una ciutat cada hora.

Aquestes dades tenen tres propietats que les fan diferents de les dades "normals" d'una base de dades relacional:

1. **Sempre tenen un timestamp.** Sense excepció.
2. **Arriben en ordre cronològic.** Normalment no es reescriuen valors antics.
3. **El volum és enorme.** Un sistema amb 100 sensors publicant cada minut genera $100 \times 60 \times 24 = 144.000$ punts per dia.

Una base de dades relacional com PostgreSQL pot guardar aquestes dades, però no està optimitzada per fer-ho. Les consultes sobre intervals temporals grans es tornen lentes. La compressió de dades és pobre. La gestió d'índexos temporals és complicada.

Una TSDB, en canvi, està dissenyada des de zero per a aquest cas:

- **Emmagatzematge column-oriented:** les dades de cada sèrie temporal s'emmagatzemen en columnes, la qual cosa permet una compressió excel·lent.
- **Índexs temporals natius:** les consultes per rangs temporals són ràpides per disseny.
- **Agregacions natives:** promitjos, màxims, mínims per finestres temporals.
- **Retenció configurable:** podem dir "guarda les dades a resolució original durant 30 dies, i les dades agregades a resolució horària durant 5 anys".

InfluxDB és una de les TSDB de codi obert més populars, juntament amb TimescaleDB (que és una extensió de PostgreSQL), Prometheus (orientada a mètriques), i OpenTSDB. Al BernatLab farem servir InfluxDB per la seva senzillesa i potència.

15.2 InfluxDB: versions i edicions

InfluxDB té dues línies principals:

- **InfluxDB 1.x**: la versió clàssica, amb el seu propi llenguatge de consultes (InfluxQL), una interfície web (Chronograf), i un agent de recollida (Telegraf, compartit amb 2.x). Encara àmpliament usada.
- **InfluxDB 2.x**: la nova versió, que unifica la base de dades, la interfície web, i el sistema d'autenticació. Té el seu propi llenguatge de consultes (Flux), una interfície web moderna, i una API REST.

Al BernatLab farem servir **InfluxDB 2.x** perquè:

- La interfície web està integrada.
- L'autenticació basada en tokens és moderna i còmoda.
- Té millor suport per a nous llenguatges de consulta.
- InfluxData l'està desenvolupant activament.

15.3 Conceptes clau d'InfluxDB 2.x

InfluxDB 2.x introdueix una sèrie de conceptes nous respecte a 1.x. Val la pena entendre'ls bé abans de posar-nos a instal·lar.

Organització (Organization)

Una **organització** (org) és un contenidor d'alt nivell. Tots els recursos (buckets, tasques, tokens, usuaris) viuen dins d'una organització. Al BernatLab només en tindrem una, anomenada `bernatlab`.

Bucket

Un **bucket** és l'equivalent d'una base de dades o una taula en el món relacional. Totes les dades s'emmagatzemen en un bucket. Cada bucket té:

- Un **nom** (per exemple, `hort-osona`).
- Una **política de retenció**: quant de temps es guarden les dades.
- Un **període de retenció** opcional.

InfluxDB 2.x ja no distingeix entre "base de dades" i "política de retenció" com feia 1.x. Tot es configura al bucket.

Al BernatLab, crearem un bucket `hort-osona` amb una retenció prou llarga (per exemple, 1 any) i un bucket `hort-osona-downsampled` amb dades agregades a resolució horària per a visualitzacions a llarg termini.

Mesures, tags i camps

Una **mesura** (measurement) és l'equivalent d'una taula. El seu nom és arbitrari, normalment coincideix amb el tipus de dada (per exemple, `temperatura`, `humitat`).

Cada punt dins d'una mesura té:

- **Tag**: una metadada indexada (per exemple, `zona="zona-tomateres"`). Els tags s'indexen, de manera que les consultes que filtren per tag són ràpides.
- **Camp** (field): el valor numèric (per exemple, `valor=23.5`). Els camps no s'indexen.

La regla d'or:

- Si volem **filtrar** per una propietat, és un **tag**.
- Si volem **agregar** (sumar, fer mitjana) una propietat, és un **camp**.

Al BernatLab:

- Tag: **zona** (zona-tomateres, zona-enciams, ...), **sensor** (bme280, dht22, ...).
- Camp: **valor** (la mesura en si), **qualitat** (opcional, % de fiabilitat).

Series

Una **series** és la combinació d'una mesura + un conjunt de tags. Per exemple:

temperatura, zona=zona-tomateres, sensor=bme280

Aquesta és una sèrie. Tots els punts amb aquesta combinació formen una seqüència temporal.

Punt (point)

Un **punt** és l'equivalent d'una fila en una taula. Té:

- Una mesura.
- Un o més tags.
- Un o més camps.
- Un timestamp.

Exemple de punt en Line Protocol (veure més avall):

temperatura, zona=zona-tomateres, sensor=bme280 valor=23.5,qualitat=95 1717823400000000000

Token

Un **token** és una cadena que autentica una aplicació davant InfluxDB. Al BernatLab crearem tokens específics per a cada consumidor:

- **telegraf-token**: per a Telegraf.
- **nodered-token**: per a Node-RED.
- **grafana-token**: per a Grafana.
- **api-token**: per a la nostra API FastAPI.

Cada token té els seus permisos (lectura, escriptura, lectura+escriptura) sobre els seus buckets.

15.4 Instal·lació al BernatLab

InfluxDB 2.x es desplega amb Docker. La imatge oficial és `influxdb:2.7` (o la darrera versió estable).

Definició al docker-compose.yml

```
services:
  influxdb:
    image: influxdb:2.7
    container_name: influxdb
    restart: unless-stopped
    ports:
      - "8086:8086"      # API
    volumes:
      - /home/bernat/homelab/data/influxdb:/var/lib/influxdb2
      - /home/bernat/homelab/data/influxdb/config:/etc/influxdb2
    environment:
      - DOCKER_INFLUXDB_INIT_MODE=setup
      - DOCKER_INFLUXDB_INIT_USERNAME=bernat
      - DOCKER_INFLUXDB_INIT_PASSWORD=ELMEUPASSWORD
      - DOCKER_INFLUXDB_INIT_ORG=bernatlab
      - DOCKER_INFLUXDB_INIT_BUCKET=hort-osona
      - DOCKER_INFLUXDB_INIT_RETENTION=8760h # 1 any
      - DOCKER_INFLUXDB_INIT_ADMIN_TOKEN=ELMEUTOKENINICIAL
```

Aquesta configuració aprofita la inicialització automàtica de la imatge Docker: quan es crea el contenidor per primera vegada, InfluxDB es configura sol amb l'usuari, organització, bucket i token inicial que li hem passat.

Primer accés

Un cop en marxa, podem accedir a la interfície web a <http://100.115.134.76:8086>. Ens demanarà l'usuari ([bernat](#)) i la contrasenya que hem definit.

La interfície web d'InfluxDB 2.x és molt completa:

- **Data Explorer:** per explorar les dades i construir consultes.
- **Dashboards:** per crear gràfiques senzilles (per a les complexes, farem servir Grafana).
- **Tasks:** per programar consultes periòdiques (per exemple, agregacions).
- **Buckets:** per gestionar els contenidors de dades.
- **Tokens:** per crear i gestionar tokens d'accés.
- **Sources:** per configurar Telegraf o altres clients.
- **Settings:** per configurar l'organització i altres paràmetres.

Crear els tokens

Un cop dins, anirem a **Load Data** → **Tokens** i crearem els tokens específics per a cada servei:

1. **telegraf-token:** permisos d'escriptura sobre [hort-osona](#).
2. **nodered-token:** permisos de lectura i escriptura sobre [hort-osona](#).
3. **grafana-token:** permisos de lectura sobre [hort-osona](#).
4. **api-token:** permisos de lectura sobre [hort-osona](#).

Cada token es mostra una sola vegada en crear-lo. ****Cal guardar-los al .env immediatament**** (no pas al codi, no pas a Git).

15.5 Line Protocol: el llenguatge de InfluxDB

InfluxDB accepta dades en un format anomenat **Line Protocol**, que és molt compacte i eficient. La sintaxi és:

```
measurement,tag1=value1,tag2=value2 field1=value1,field2=value2 timestamp
```

Exemple:

```
temperatura, zona=zona-tomateres, sensor=bme280 valor=23.5,qualitat=95 1717823400000000000
```

Detalls a notar:

- El **timestamp** és en nanosegons des de l'època Unix (1 de gener de 1970). Si no es proporciona, InfluxDB assigna el temps del servidor.
- Els **tags** sempre són strings, per la qual cosa els valors van sense cometes.
- Els **camp**s poden ser floats, enters, strings, o booleans. Per a floats, podem posar el sufix **i** per a enters.
- Les **cadena**s **string** als camps van entre cometes dobles i escapen les cometes interiors amb ****.

Exemple complet:

```
temperatura, zona=zona-tomateres valor=23.5 1717823400000000000
humitat, zona=zona-tomateres valor=60 1717823400000000000
pressio, zona=zona-tomateres valor=1013 1717823400000000000
```

Cadascuna d'aquestes línies és un punt. Podem enviar-les una per una, o en bloc (una línia per punt, separades per **\n**).

15.6 Escriure dades: la API d'escriptura

InfluxDB ofereix diverses maneres d'escriure dades:

- **HTTP POST** a `/api/v2/write`, amb el Line Protocol al body.
- **Client libraries** (Python, JavaScript, Go, etc.) que encapsulen l'API.
- **Telegraf**, que és l'opció que farem servir al Capítol 16.

Exemple d'escriptura amb curl:

```
curl -i -XPOST "http://100.115.134.76:8086/api/v2/write?org=bernatlab&bucket=hort-osona" \
  --header "Authorization: Token ELMEUTOKEN" \
  --data-raw "temperatura, zona=zona-tomateres valor=23.5"
```

Exemple amb Python:

```
from influxdb_client import InfluxDBClient
from influxdb_client.client.write_api import SYNCHRONOUS

client = InfluxDBClient(
    url="http://100.115.134.76:8086",
    token="ELMEUTOKEN",
    org="bernatlab"
)
write_api = client.write_api(write_options=SYNCHRONOUS)

punt = [
    {
```

```

    "measurement": "temperatura",
    "tags": {"zona": "zona-tomateres"},
    "fields": {"valor": 23.5},
    "time": 1717823400
  }
]
write_api.write(bucket="hort-osona", record=punt)

```

15.7 Llegir dades: Flux, el llenguatge de consultes

InfluxDB 2.x introdueix **Flux**, un llenguatge de consultes funcional inspirat en JavaScript. No és SQL, però té una estructura lògica:

```

from(bucket: "hort-osona")
  |> range(start: -1h)
  |> filter(fn: (r) => r._measurement == "temperatura")
  |> filter(fn: (r) => r.zona == "zona-tomateres")
  |> mean()

```

Aquesta consulta:

1. Agafa totes les dades del bucket `hort-osona` de l'última hora.
2. Filtra per mesura `temperatura`.
3. Filtra per zona `zona-tomateres`.
4. Calcula la mitjana.

Exemples habituals:

Últim valor:

```

from(bucket: "hort-osona")
  |> range(start: -1h)
  |> filter(fn: (r) => r._measurement == "temperatura")
  |> last()

```

Mitjana per hora:

```

from(bucket: "hort-osona")
  |> range(start: -24h)
  |> filter(fn: (r) => r._measurement == "temperatura")
  |> aggregateWindow(every: 1h, fn: mean, createEmpty: false)

```

Màxim del dia:

```

from(bucket: "hort-osona")
  |> range(start: -1d)
  |> filter(fn: (r) => r._measurement == "temperatura")
  |> max()

```

Aprendrem més sobre Flux a mesura que l'usem, però aquests exemples ja donen una idea.

15.8 InfluxDB UI: Data Explorer

El **Data Explorer** de la interfície web ens permet construir consultes visualment, sense escriure Flux a mà. Podem:

1. Triar el bucket.
2. Triar la mesura.

3. Triar els camps.
4. Aplicar filtres (tags, rangs temporals).
5. Aplicar agregacions.
6. Visualitzar el resultat en una taula o gràfica.

És una eina molt útil per aprendre i per depurar consultes abans de posar-les a Grafana o a la API.

15.9 Retenció de dades

La **retenció** és la política que determina quant de temps es guarden les dades. Al bucket `hort-osona` hem definit 1 any (8760 hores). Això vol dir que, passat un any, InfluxDB esborrarà les dades antigues.

Però no cal que esperem un any per tenir dades agregades. Podem crear **Tasks** (tasques) que periòdicament agreguin dades a resolucions més baixes i les desin en un altre bucket amb retenció més llarga:

- `hort-osona`: dades originals, retenció 1 any.
- `hort-osona-1h`: dades agregades a resolució horària, retenció 5 anys.
- `hort-osona-1d`: dades agregades a resolució diària, retenció 10 anys.

Exemple de task per agregar dades a resolució horària:

```
option task = {
  name: "agrega-horaria",
  every: 1h
}

from(bucket: "hort-osona")
  |> range(start: -1h)
  |> filter(fn: (r) => r._measurement == "temperatura")
  |> aggregateWindow(every: 1h, fn: mean, createEmpty: false)
  |> to(bucket: "hort-osona-1h")
```

Aquesta tasca s'executa cada hora, agafa les dades de l'última hora, n'agrega la mitjana, i les desa al bucket d'agregats. Així, podem consultar el comportament de la temperatura al llarg de 5 anys sense saturar el bucket principal.

15.10 Consultes útils per al dia a dia

A la interfície web o des de la CLI `influx`, podem fer consultes com:

Quantes mesures tenim al bucket?

```
import "influxdata/influxdb/schema"

schema.measurements(bucket: "hort-osona")
```

Quines zones tenim registrades?

```
import "influxdata/influxdb/schema"

schema.tagValues(bucket: "hort-osona", tag: "zona")
```

Quin és el rànkung de zones per temperatura màxima les últimes 24 h?

```
from(bucket: "hort-osona")
  |> range(start: -24h)
  |> filter(fn: (r) => r._measurement == "temperatura")
  |> max()
  |> group(columns: ["zona"])
  |> top(n: 5)
```

Aquestes consultes les explorarem a mesura que creem els dashboards de Grafana.

15.11 Rendiment i recursos

InfluxDB 2.x és bastant eficient, però en una Raspberry Pi 4 amb 4 GB de RAM, hem de ser curosos:

- **Bucket per defecte:** el que hem creat (`hort-osona`) és prou.
- **Memòria cache:** InfluxDB fa servir memòria per cachejar consultes. Amb 4 GB totals, no podem permetre que InfluxDB en consumeixi més de 500 MB.
- **Compactació periòdica:** InfluxDB compacta les dades periòdicament. Això consumeix CPU temporalment.

Si veiem que el sistema va lent o que InfluxDB consumeix massa RAM, podem ajustar:

```
# Al fitxer de configuració (configurat per variables d'entorn)
INFLUXD_BOLT_PATH=/var/lib/influxdb2/influxd.bolt
INFLUXD_ENGINE_PATH=/var/lib/influxdb2/engine
INFLUXD_QUERY_MEMORY_BYTES=536870912 # 512 MB
INFLUXD_QUERY_MAX_BUCKETS=20
```

Al BernatLab, amb pocs sensors i freqüències de publicació raonables, InfluxDB no hauria de ser un problema. Però estarem atents.

15.12 Còpies de seguretat

InfluxDB permet fer còpies de seguretat de tota la base de dades amb la comanda `influx backup`:

```
influx backup /path/al/backup \
  --bucket hort-osona \
  --org bernatlab \
  --token ELMEUTOKEN
```

Això crea una còpia completa del bucket, que podem restaurar amb:

```
influx restore /path/al/backup \
  --bucket hort-osona \
  --org bernatlab \
  --token ELMEUTOKEN
```

Important: les còpies s'han de fer amb el servei **aturat** o amb la còpia "online" (que InfluxDB 2.x sí que permet). Millor parar el servei per seguretat:

```
docker compose stop influxdb
influx backup /home/bernat/homelab/backup/influxdb-$(date +%F)
docker compose start influxdb
```

15.13 Integració amb la resta del BernatLab

Un cop tenim InfluxDB funcionant, podem connectar-hi:

- **Telegraf** (Capítol 16): subscriu's a MQTT i escriu punts a InfluxDB.

- **Node-RED** (Capítols 17 i 18): llegeix dades per prendre decisions.
- **Grafana** (Capítol 19): llegeix dades per visualitzar.
- **API FastAPI** (Capítol 20): llegeix dades agregades per servir a la web.

Tots aquests serveis s'autenticaran amb el seu propi token, amb els permisos adequats.

15.14 InfluxDB CLI

A més de la interfície web, InfluxDB ofereix una CLI (*influx*) que ens permet gestionar-ho tot des de la consola:

```
# Llistar buckets
influx bucket list --org bernatlab --token ELMEUTOKEN

# Crear un bucket
influx bucket create --name hort-osona-downsampled \
  --org bernatlab --retention 17520h --token ELMEUTOKEN

# Llistar tokens
influx auth list --org bernatlab --token ELMEUTOKEN

# Escriure un punt
influx write --bucket hort-osona \
  --org bernatlab --token ELMEUTOKEN \
  "temperatura, zona=zona-tomateres valor=23.5"
```

Aquesta CLI és molt útil per a scripts i per a la integració amb eines externes.

15.15 Esquema d'integració

Esquema Mermaid (text):

```
graph LR
    subgraph Fonts ["Fonts de dades"]
        TELE["Telegraf"]
        NR["Node-RED"]
        API["API FastAPI"]
    end

    subgraph InfluxDB ["InfluxDB 2.x"]
        B1["Bucket hort-osona<br/>(1 any)"]
        B2["Bucket hort-osona-1h<br/>(5 anys)"]
        T1["Task agrega horària"]
        T2["Task agrega diària"]
    end

    subgraph Consums ["Consumidors"]
        GRAF["Grafana"]
        APP["Web Hort Osona"]
        NR2["Node-RED<br/>(alertes)"]
    end

    TELE --> B1
    NR --> B1
    API --> B1
    B1 --> T1
    T1 --> B2
    B2 --> T2
    B1 --> GRAF
    B1 --> NR2
    B2 --> GRAF
    B1 --> API
    API --> APP
```

15.16 Errors habituals

Error 1: no protegir els tokens. Síntoma: si un token es filtra, algú pot accedir a les dades o esborrar-les. Solució: tokens al `.env`, no pas al codi.

Error 2: no configurar la retenció. Síntoma: el bucket creix sense límit, la microSD s'omple. Solució: retenció adequada, monitorejar l'ús de disc.

Error 3: confondre tags i camps. Síntoma: consultes lentes o impossibles. Solució: Llegir bé la regla d'or: filtre → tag, agrega → camp.

Error 4: timestamp en segons en lloc de nanosegons. Síntoma: les dades apareixen amb dates incorrectes. Solució: recordar que Line Protocol vol nanosegons.

Error 5: no testear les consultes abans de posar-les a Grafana. Síntoma: gràfiques buides o errònies. Solució: usar el Data Explorer per validar.

Error 6: no fer còpies de seguretat. Síntoma: quan el sistema falla, perdem tot l'històric. Solució: backup periòdic, com ja hem vist.

15.17 Bones pràctiques

1. **Un token per servei.** Amb permisos mínims.
2. **Retenció definida des del primer moment.**
3. **Tasks d'agregació configurades** per a llarg termini.
4. **Buckets separats per a dades originals i agregades.**
5. **Backups periòdics.**
6. **Testejar consultes al Data Explorer** abans de posar-les a Grafana.
7. **Monitoratge amb Uptime Kuma** del port 8086.
8. **Documentar l'esquema** (quins tags, quins camps) al README.
9. **Limitar l'ús de memòria** amb les variables d'entorn adequades.
10. **Auditar l'accés** revisant els logs periòdicament.

15.18 Resum

Hem après què és una base de dades de sèries temporals i per què InfluxDB és una bona elecció. Hem vist els conceptes clau: organització, bucket, mesura, tag, camp, series, punt, token. Hem après a instal·lar InfluxDB 2.x al BernatLab, a crear tokens, a escriure dades amb Line Protocol i Python, i a consultar-les amb Flux. Hem vist com configurar la retenció i les tasques d'agregació. En el proper capítol aprendrem a connectar MQTT a InfluxDB amb Telegraf, l'agent que farà de pont entre els sensors i la base de dades.

15.19 Exercicis pràctics

1. Desplega InfluxDB 2.x al BernatLab amb la configuració que hem vist.
2. Accedeix a la interfície web i crea un bucket `hort-osona` amb retenció d'1 any.

3. Crea un token de lectura-escriptura per a Telegraf. Guarda'l al `.env`.
4. Escriu un punt de prova amb la CLI: `influx write --bucket hort-osona --org bernatlab --token TOKEN "temperatura, zona=test valor=23.5"`.
5. Llegeix el punt amb el Data Explorer.
6. Escriu un script Python que escrigui 100 punts aleatoris a InfluxDB, amb 5 segons d'interval entre ells.
7. Crea una task que agregui les dades a resolució horària.
8. Fes una còpia de seguretat del bucket amb `influx backup`.

Comandes útils:

```
# CLI
influx bucket list --org bernatlab --token TOKEN
influx bucket create --name hort-osona --org bernatlab --retention 8760h --token TOKEN
influx auth list --org bernatlab --token TOKEN
influx write --bucket hort-osona --org bernatlab --token TOKEN "mesura,tag=valor camp=123"
```

```
# Python
python3 -c "
from influxdb_client import InfluxDBClient
c = InfluxDBClient(url='http://100.115.134.76:8086', token='TOKEN', org='bernatlab')
print('Buckets:', c.buckets_api().find_buckets().buckets)
"
```

```
# Backup
docker compose stop influxdb
influx backup /home/bernat/homelab/backup/influxdb-$(date +%F) --token TOKEN
docker compose start influxdb
```

Paraules clau: **InfluxDB, TSDB, sèrie temporal, bucket, token, Line Protocol, Flux, tag, camp, mesura, organització, retenció, task, agregació, còpia de seguretat, Data Explorer, Telegraf, Node-RED, Grafana, API, sensors, Hort Osona, nanosegons, schemaless.**

Capítol 16 — Telegraf: el pont

"Telegraf és el treballador silenciós. No es nota, però sense ell, les dades no arriben mai a bon port."

16.1 Què és Telegraf

Telegraf és un agent de codi obert, escrit en Go, desenvolupat per InfluxData (la mateixa empresa que InfluxDB). La seva feina és recollir mètriques de tot tipus de fonts, processar-les, i escriure-les a una o més destinacions. És l'equivalent a un "conseller delegat" de la cadena de dades: sap com parlar amb centenars de serveis diferents.

Telegraf es basa en **plugins**:

- **Inputs**: llegeixen dades de fonts externes (MQTT, HTTP, fitxers, execució de comandes, etc.).
- **Processors**: transformen les dades (renombrar camps, agregar, parsejar, etc.).
- **Aggregators**: combinen múltiples punts en un de sol (mitjana, màxim, etc.).
- **Outputs**: escriuen les dades a destinacions (InfluxDB, Kafka, fitxers, etc.).

Cada plugin és un petit programa independent que es configura al fitxer `telegraf.conf`. La potència de Telegraf ve d'aquesta modularitat: podem combinar inputs i outputs com si fossin peces de Lego.

Al BernatLab, Telegraf serà el **pont entre Mosquitto i InfluxDB**: escoltarà tots els missatges MQTT del broker i els escriurà com a punts a InfluxDB. Per fer-ho, usarem dos plugins:

- **Input MQTT**: subsciu a topics MQTT, rep els missatges, parseja el payload.
- **Output InfluxDB v2**: escriu els punts a InfluxDB 2.x.

A més, afegirem un **processor** per renombrar camps i tags, i un **aggregator** per sumar lectures quan calgui.

16.2 Instal·lació al BernatLab

Telegraf es desplega amb Docker. La imatge oficial és `telegraf:1.30` (o l'última estable).

Definició al `docker-compose.yml`

```
services:
  telegraf:
    image: telegraf:1.30
    container_name: telegraf
    restart: unless-stopped
    user: telegraf:998
    volumes:
      - ./telegraf.conf:/etc/telegraf/telegraf.conf:ro
      - /home/bernat/homelab/data/telegraf:/var/lib/telegraf
    environment:
      - INFLUX_TOKEN=${TELEGRAF_INFLUX_TOKEN}
      - MQTT_USERNAME=telegraf
      - MQTT_PASSWORD=${TELEGRAF_MQTT_PASSWORD}
```

Telegraf necessita accedir al fitxer de configuració en mode lectura, i a una carpeta per emmagatzemar estat intern (caches, etc.). Les credencials les passem com a variables d'entorn des del `.env`.

16.3 Configuració bàsica

El fitxer `telegraf.conf` és un TOML (una variant de INI). Té tres seccions principals: `[agent]`, `[[inputs.mqtt_consumer]]`, `[[outputs.influxdb_v2]]`.

Configuració de l'agent

```
[agent]
  interval = "30s"
  flush_interval = "30s"
  metric_batch_size = 1000
  metric_buffer_limit = 10000
  debug = false
  quiet = false
  hostname = "hortosona"
  omit_hostname = false
```

- **`interval`**: cada quan recollim dades de les fonts. Per a MQTT, no és rellevant (Telegraf reacciona als missatges), però és bona pràctica definir-lo.
- **`flush_interval`**: cada quan enviem les dades als outputs. 30 segons és un bon valor per defecte.
- **`metric_batch_size`**: mida màxima del lot de mètriques. 1000 és adequat.
- **`hostname`**: com s'identifica aquest agent a les mètriques. Aquest valor s'afegeix com a tag a cada punt.

Configuració de l'input MQTT

```
[[inputs.mqtt_consumer]]
  servers = ["tcp://100.115.134.76:1883"]
  username = "telegraf"
  password = "${MQTT_PASSWORD}"
  client_id = "telegraf-bernatlab"
  qos = 0
  clean_session = true
  topics = [
    "hort/+/+/+",
    "hort/+/estat",
  ]
  data_format = "json"
  json_string_fields = []
  tag_keys = ["zona", "sensor", "unitat"]
  json_timestamp_units = "1s"
  tagexclude = ["host", "topic"]
```

- **`servers`**: el broker MQTT. Al BernatLab és `tcp://100.115.134.76:1883`.
- **`username` i `password`**: les credencials del compte `telegraf`.
- **`topics`**: els patrons de topics a subscriure. Aquí volem tot `hort/+/+/+` (zona + tipus + identificador) i els topics d'estat.
- **`data_format = "json"`**: Telegraf parsejarà cada missatge com a JSON.

- **tag_keys**: quins camps del JSON es converteixen en tags (indexats) en lloc de camps. Aquí volem `zona`, `sensor`, i `unitat` com a tags.
- **json_timestamp_units**: unitat del timestamp al JSON. Si el JSON té un camp `ts` en segons Unix, posem `"1s"`. Si no, podem ometre aquest paràmetre i Telegraf assignarà el temps del servidor.
- **tagexclude**: tags que volem excloure del punt final (per exemple, `topic` que és informatiu però no útil per a consultes).

Configuració de l'output InfluxDB v2

```
[[outputs.influxdb_v2]]
  urls = ["http://influxdb:8086"]
  token = "${INFLUX_TOKEN}"
  org = "bernatlab"
  bucket = "hort-osona"
  timeout = "10s"
  user_agent = "telegraf-bernatlab"
  content_encoding = "gzip"
```

- **urls**: la URL d'InfluxDB. Com que és un servei del mateix `docker-compose.yml`, podem usar el nom del servei (`influxdb`).
- **token**: el token d'escriptura que hem creat per a Telegraf.
- **org i bucket**: l'organització i el bucket d'InfluxDB.
- **content_encoding = "gzip"**: Telegraf comprimirà les dades abans d'enviar-les, la qual cosa estalvia ample de banda.

Filtratge i processament

Podem afegir un processor per netejar les dades. Per exemple, per renombrar el camp `valor` a `value` (que és el nom convencional a InfluxDB) i afegir un tag de zona horària:

```
[[processors.rename]]
  [[processors.rename.replace]]
    field = "valor"
    dest = "value"

[[processors.converter]]
  [[processors.converter.tags]]
    string = ["zona", "sensor", "unitat"]
```

Això és opcional, però útil per mantenir la consistència.

16.4 Exemple: com es transforma un missatge

Imaginem que un sensor publica:

Topic: `hort/zona-tomateres/temperatura/aire`
 Payload: `{"valor": 23.5, "unitat": "graus_C", "ts": 1717823400}`

Telegraf ho processa i crea un punt InfluxDB:

Measurement: `hort/zona-tomateres/temperatura/aire`
 Tags: `zona=zona-tomateres, sensor=bme280, unitat=graus_C, host=hortosona`
 Fields: `valor=23.5`

Time: 1717823400 (or server time if ts is missing)

Aquest punt s'escriu a InfluxDB al bucket `hort-osona`. A partir d'aquí, podem consultar-lo amb Flux o visualitzar-lo amb Grafana.

16.5 Treballar amb múltiples tipus de mesura

Quan tenim molts sensors i moltes mesures, podem voler organitzar les dades de manera més neta. Hi ha dues estratègies:

Estratègia 1: una mesura per tipus

Tots els punts amb el camp `valor` que són temperatures van a la mesura `temperatura`; tots els que són humitat van a la mesura `humitat`, etc. Això s'aconsegueix amb un processor que renombra la mesura:

```
[[processors.regex]]
[[processors.regex.tags]]
  key = "topic"
  pattern = "^hort/[^/]+/([^\s/]+)/.*$"
  replacement = "${1}"
```

Aquest processor extreu el tipus de mesura (temperatura, humitat, ...) del topic i l'assigna com a tag. Després, podem renombrar-lo a `_measurement` perquè InfluxDB l'usi com a mesura:

```
[[processors.rename]]
[[processors.rename.replace]]
  tag = "topic_type"
  dest = "_measurement"
```

Estratègia 2: mantenir el topic com a mesura

Deixar que el topic sigui directament la mesura (per exemple, `hort/zona-tomateres/temperatura/aire`). Això és més simple, però pot generar un gran nombre de mesures.

Al BernatLab, començarem amb l'estratègia 1 (una mesura per tipus) perquè és més neta per a Grafana.

16.6 Agregacions a Telegraf

Telegraf pot agregar dades abans d'enviar-les. Per exemple, podem calcular la mitjana de cada 5 minuts:

```
[[aggregators.basicstats]]
  period = "300s"
  drop_original = false
  stats = ["mean", "min", "max", "stddev"]
```

Aquest aggregator crea nous camps:

- `valor_mean`: mitjana del període.
- `valor_min`: mínim.
- `valor_max`: màxim.
- `valor_stddev`: desviació estàndard.

Útil per reduir el volum de dades quan tenim sensors que publiquen molt sovint.

16.7 Provar la configuració

Un cop escrit el fitxer `telegraf.conf`, podem validar-lo:

```
docker compose exec telegraf telegraf --config /etc/telegraf/telegraf.conf --test
```

Aquesta ordre fa una passada de prova, llegint les dades durant uns segons i mostrant per pantalla el que enviaria a InfluxDB. Si veiem les mètriques correctes, la configuració és bona.

Alternativament, podem mirar els logs:

```
docker compose logs telegraf
```

Si tot va bé, veurem línies com:

```
2024-06-08T12:00:00Z I! Starting Telegraf 1.30.0
2024-06-08T12:00:00Z I! Loaded inputs: mqtt_consumer
2024-06-08T12:00:00Z I! Loaded outputs: influxdb_v2
2024-06-08T12:00:00Z I! Tags enabled: host=hortosona
2024-06-08T12:00:00Z I! [agent] Start: agent started
```

Si hi ha errors de connexió a MQTT o a InfluxDB, els veurem aquí.

16.8 Provar amb un simulador de sensor

Per validar tota la cadena, podem executar el simulador Python del Capítol 14 i comprovar que les dades apareixen a InfluxDB. Al Capítol 14 ja vàrem tenir:

```
# simula_bme280.py - script que publica a MQTT cada 60 segons
```

Executem-lo en una terminal, i en una altra mirem si InfluxDB rep les dades:

```
# Mira les mètriques al Data Explorer
# O amb la CLI:
influx query '
from(bucket: "hort-osona")
  |> range(start: -5m)
  |> filter(fn: (r) => r._measurement == "temperatura")
  |> last()
' --org bernatlab --token TOKEN
```

Si tot és correcte, veurem les últimes lectures.

16.9 Consideracions de rendiment

Telegraf pot gestionar volums molt alts de dades, però en una Raspberry Pi hem de ser curosos:

- **`flush_interval`**: 30 segons és un bon equilibri. Més curt = més càrrega, més llarg = més memòria.
- **`metric_buffer_limit`**: 10.000 mètriques. Si el buffer s'omple, Telegraf llençarà mètriques. Ajustar segons la càrrega.
- **Processors**: cada processor afegeix latència. No abusar-ne.
- **Compressió**: `gzip` estalvia ample de banda però consumeix una mica de CPU.

Al BernatLab, amb 5-10 sensors publicant cada minut, el rendiment no serà un problema. Però estarem atents als logs per si Telegraf comença a perdre mètriques.

16.10 Quan fallen les coses: estratègies de recuperació

Quan Telegraf no pot escriure a InfluxDB (perquè InfluxDB està caigut, per exemple), què passa?

- **Telegraf desa les mètriques a memòria** (fins al `metric_buffer_limit`).
- Si InfluxDB torna, Telegraf buida el buffer.
- Si el buffer s'omple, Telegraf llença les mètriques més antigues (configurable).

Això vol dir que en una caiguda curta, no perdem dades. En una caiguda llarga, podem perdre'n.

Per minimitzar la pèrdua de dades, podem:

1. **Configurar Uptime Kuma** per alertar quan InfluxDB està caigut.
2. **Augmentar el `metric_buffer_limit`** si tenim RAM disponible.
3. **Configurar Telegraf per escriure a disc** quan el buffer estigui ple (configuració avançada, no sempre recomanable).

16.11 Logs i depuració

Quan alguna cosa no funciona, els logs són la primera pista. Per augmentar la verbositat:

```
[agent]
  debug = true
  quiet = false
```

I per veure exactament quin missatge arriba i quin punt genera:

```
[[inputs.mqtt_consumer]]
  # ... altres opcions ...
  json_debug_fields = true
```

Això ens permet veure, per a cada missatge, quin punt s'ha generat i quins tags i camps té. Molt útil quan estem afinant la configuració.

16.12 Configuracions avançades

Processar payloads no-JSON

Si tenim sensors que publiquen en text pla (per exemple, `23.5`), podem canviar el format:

```
[[inputs.mqtt_consumer]]
  data_format = "value"
  data_type = "float"
```

Això crearà un punt amb el valor numèric directament.

Afegir tags automàtics

Podem afegir tags addicionals a tots els punts, com ara el nom del projecte:

```
[agent]
  hostname = "hortosona"
```

```
[[inputs.mqtt_consumer]]
# ... options ...
[[inputs.mqtt_consumer.tagpass]]
project = "bernatlab"
```

Això és útil per distingir dades de diferents projectes si en el futur en tenim més d'un.

Múltiples brokers

Si tenim més d'un broker MQTT, podem afegir múltiples inputs:

```
[[inputs.mqtt_consumer]]
servers = ["tcp://broker1:1883"]
name = "broker1"
# ...

[[inputs.mqtt_consumer]]
servers = ["tcp://broker2:1883"]
name = "broker2"
# ...
```

Això ens permet consolidar dades de diversos brokers en un sol bucket d'InfluxDB.

16.13 Còpies de seguretat

La configuració de Telegraf és al fitxer `telegraf.conf`, que ja està versionat amb Git (a `stacks/iot/`). No cal fer-ne còpies addicionals, tret dels logs si els volem analitzar.

El volum persistent de Telegraf (`/var/lib/telegraf`) conté caches que es poden regenerar. No cal copiar-lo.

16.14 Integració amb la resta del BernatLab

Un cop tenim Telegraf funcionant:

- Les dades dels sensors flueixen de MQTT a InfluxDB automàticament.
- Grafana pot consultar InfluxDB per visualitzar les dades.
- Node-RED pot subscriure's directament a MQTT (o consultar InfluxDB) per prendre decisions.
- L'API FastAPI pot consultar InfluxDB per servir dades a la web.

Telegraf és, per tant, el **cor de la cadena de dades**. Sense ell, les dades no arribarien a InfluxDB.

16.15 Esquema conceptual

Esquema Mermaid (text):

```
graph LR
    subgraph Sensors["Sensors"]
        S1["Sensor 1"]
        S2["Sensor 2"]
    end

    subgraph MQTT["MQTT Broker"]
        M["Mosquitto"]
    end

    subgraph Telegraf["Telegraf"]
        I["Input MQTT"]
        P["Processors"]
    end
```

```

    A["Aggregators"]
    O["Output InfluxDB"]
end

subgraph InfluxDB["InfluxDB"]
    B["Bucket hort-osona"]
end

S1 --> M
S2 --> M
M --> I
I --> P
P --> A
A --> O
O --> B

```

16.16 Errors habituals

Error 1: subscriure's a massa topics. Síntoma: Telegraf consumeix massa CPU, el buffer s'omple. Solució: ser específic amb els patrons de topics.

Error 2: no parsejar correctament el JSON. Síntoma: els punts arriben a InfluxDB amb camps buits o erronis. Solució: usar `--test` per validar la configuració.

Error 3: token d'InfluxDB incorrecte. Síntoma: errors d'autenticació als logs. Solució: revisar el `.env` i el token.

****Error 4: no excloure el tag `topic`**.** Síntoma: cada missatge té un tag `topic` únic, cosa que multiplica les series a InfluxDB. Solució: afegir `tagexclude = ["topic"]`.

Error 5: timestamps inconsistent. Síntoma: les dades apareixen amb dates incorrectes. Solució: usar `json_timestamp_units` correctament o deixar que InfluxDB assigni el temps.

16.17 Bones pràctiques

1. **`tagexclude = ["topic"]`** sempre, per evitar multiplicar les series.
2. **Un `topic_measurement` clar**, normalment basat en el tipus de dada.
3. **Flush_interval de 30 segons** com a bon equilibri.
4. **Validar amb `--test`** abans de posar en marxa.
5. **Monitorar amb Uptime Kuma** que Telegraf està corrent.
6. **Backups del bucket** d'InfluxDB (no de Telegraf directament).
7. **Documentar els tags** al README del projecte.
8. **Limitar el nombre de processors** per minimitzar la latència.
9. **Usar compressió gzip** per estalviar ample de banda.
10. **Revisar els logs periòdicament** per detectar anomalies.

16.18 Resum

Hem après què és Telegraf, com es configura amb inputs i outputs, com es connecta a Mosquitto i a InfluxDB al BernatLab, com es parsegen els missatges JSON, com s'agreguen dades i com es filtra el que arriba a InfluxDB. Hem vist exemples reals de configuració, hem après a provar-la amb `--test`, i hem après les bones pràctiques. En el proper capítol veurem Node-RED, l'eina de programació visual que ens permetrà processar les dades d'una manera molt flexible.

16.19 Exercicis pràctics

1. Desplega Telegraf al BernatLab amb la configuració que hem vist.
2. Executa el simulador Python del Capítol 14 durant 5 minuts.
3. Comprova al Data Explorer d'InfluxDB que les dades hi arriben.
4. Consulta les últimes 10 lectures de temperatura amb una query Flux.
5. Afegeix un processor a Telegraf que calculi la mitjana de cada 5 minuts.
6. Prova de canviar el `flush_interval` a 5 segons i observa com canvia el comportament.
7. Fes una prova amb `--test` per veure què passaria amb la configuració actual.
8. Documenta al README l'esquema de tags que genera Telegraf.

Comandes útils:

```
# Validar la configuració
docker compose exec telegraf telegraf --config /etc/telegraf/telegraf.conf --test

# Veure els logs
docker compose logs -f telegraf

# Comprovar que escriu a InfluxDB
influx query '
from(bucket: "hort-osona")
  |> range(start: -5m)
  |> filter(fn: (r) => r._measurement == "temperatura")
  |> last()
' --org bernatlab --token TOKEN
```

Paraules clau: **Telegraf**, **agent**, **plugin**, **input**, **output**, **processor**, **aggregator**, **MQTT**, **InfluxDB**, **Line Protocol**, **JSON**, **telegraf.conf**, **tagexclude**, **flush_interval**, **--test**, **validació**, **simulador**, **sensors**, **hort**, **mesura**, **tags**, **camps**, **agregació**, **rendiment**, **memòria**, **buffer**, **depuració**, **monitoratge**, **Uptime Kuma**, **README**.

Capítol 17 — Node-RED: programació visual

*“Node-RED és l'eina que fa que la programació sembli un joc de nens. Però que sembli un joc no vol dir que sigui trivial: la disciplina continua sent necessària.”**

17.1 Què és Node-RED

Node-RED és una eina de programació visual, basada en fluxos, desenvolupada originalment per IBM i ara mantinguda per la comunitat de codi obert. La seva interfície és una graella on arrosseguem **nodes** (blocs amb funcionalitats específiques) i els connectem amb **cables** per crear **fluxos** (flows).

La idea és senzilla: en lloc d'escriure codi Python o JavaScript, dibuixem la lògica. Cada node fa una cosa:

- Un node MQTT es subscriu a un topic i ens entrega els missatges.
- Un node InfluxDB consulta o escriu punts.
- Un node Telegram envia un missatge.
- Un node Function executa JavaScript personalitzat.

Connectem nodes amb cables per definir el flux de dades. Quan tot està connectat, el flux s'executa: cada vegada que un node rep un missatge, el processa i l'envia al següent node.

17.2 Per què Node-RED al BernatLab

Al BernatLab, Node-RED ens servirà per:

1. **Processar les dades dels sensors** un cop estan a InfluxDB. Per exemple, calcular una mitjana mòbil, detectar patrons, transformar unitats.
2. **Detectar anomalies** i generar alertes. Si la temperatura baixa de 2 °C, enviar un missatge a Telegram.
3. **Coordinar actuadors**. Si la humitat del sòl baixa del 30 %, obrir una vàlvula de reg.
4. **Crear APIs senzilles** amb un node HTTP in.
5. **Integrar serveis externs**: enviar dades a un webhook, rebre comandes de Telegram, etc.

Node-RED és molt adequat per a aquestes tasques perquè:

- La interfície visual fa que la lògica sigui entenedora d'un cop d'ull.
- Tenim centenars de nodes preconfigurats per a tot tipus de serveis.
- Permet incrustar JavaScript personalitzat als nodes Function quan calgui.
- Es pot desar el flux com a JSON, perfecte per a Git.

17.3 Instal·lació al BernatLab

Node-RED es desplega amb Docker. La imatge oficial és `nodered/node-red:3.1` (o l'última estable).

Definició al `docker-compose.yml`

```
services:
  nodered:
    image: nodered/node-red:3.1
    container_name: nodered
    restart: unless-stopped
    ports:
      - "1880:1880"
    volumes:
      - /home/bernat/homelab/data/nodered:/data
    environment:
      - TZ=Europe/Madrid
```

Cal un volum persistent per desar els fluxos i la configuració. El port 1880 és el port estàndard de Node-RED.

Primer accés

Un cop en marxa, podem accedir a la interfície web a <http://100.115.134.76:1880>. Veurem una graella buida amb una paleta de nodes a l'esquerra i informació a la dreta.

La interfície té tres zones principals:

- **Paleta** (esquerra): llista de nodes disponibles, organitzats per categories.
- **Espai de treball** (centre): la graella on dibuixem els fluxos.
- **Informació** (dreta): informació del node seleccionat, debug, configuració.

17.4 Conceptes bàsics

Nodes

Un **node** és un bloc amb una funcionalitat específica. Hi ha molts tipus:

- **Input**: comencen un flux. Per exemple, un node `mqtt in` que escolta un topic.
- **Output**: acaben un flux. Per exemple, un node `mqtt out` que publica un missatge.
- **Function**: processen dades. Per exemple, un node `function` que executa JavaScript.
- **Storage**: emmagatzemen dades. Per exemple, un node `influxdb out` que escriu un punt.

Cada node té:

- **Ports d'entrada** (a l'esquerra): reben missatges.
- **Ports de sortida** (a la dreta): envien missatges.
- **Configuració** (doble clic): propietats específiques.

Missatges

Quan un node rep o envia un missatge, ho fa en forma d'objecte JavaScript. L'estructura bàsica és:

```
{
  payload: ...,           // el contingut principal
  topic: "...",           // opcional, útil per MQTT
  _msgid: "uuid",         // identificador únic
  // ... camps personalitzats
}
```

La clau **payload** és la convenció: la majoria de nodes la fan servir per defecte.

Fluxos

Un **flux** és una agrupació de nodes connectats. Podem tenir múltiples fluxos a la mateixa instància de Node-RED, cadascun amb el seu propi nom i la seva pròpia finalitat.

Pestanya de flux

A la part superior de l'espai de treball tenim les **pestanyes de flux** (flow tabs). Cada pestanya és un flux independent. Això ens permet organitzar la lògica en parts clarament diferenciades.

17.5 Un primer flux: Hola Món

Comencem amb el clàssic "Hola Món". Crea un flux amb dos nodes:

1. Un node **inject** (categoria *input*).
2. Un node **debug** (categoria *output*).

Connecta'ls. Configura el node **inject** perquè dispari cada 5 segons amb el payload "Hola Món". Fes clic a **Deploy** (botó a la part superior dreta).

Ara, a la pestanya **Debug** (icona del panell dret), veuràs cada 5 segons un missatge amb el payload "Hola Món".

Aquest flux no fa res útil, però ensenya el patró bàsic: un node que origina dades, un node que les consumeix, i un cable que els connecta.

17.6 Nodes útils per al BernatLab

Vegem els nodes que farem servir més sovint.

MQTT

- **mqtt in**: subscriu a un topic MQTT.
- **mqtt out**: publica un missatge MQTT.

Configuració típica de **mqtt in**:

```
{
  "broker": "100.115.134.76:1883",
  "clientId": "nodered-bernatlab",
  "topic": "hort/+/+",
  "qos": "0",
  "name": "MQTT BernatLab"
}
```

Cal configurar el broker (adreça, port, credencials) fent doble clic a la secció "Broker".

InfluxDB

Hi ha un node de la paleta `node-red-contrib-influxdb` que permet consultar i escriure a InfluxDB.

Per instal·lar-lo, anem al menú **Manage palette** → **Install** i busquem `node-red-contrib-influxdb`.

Un cop instal·lat, podem usar els nodes:

- **influxdb in**: consulta dades.
- **influxdb out**: escriu punts.

Exemple de configuració de `influxdb in`:

```
{
  "bucket": "hort-osona",
  "query": "from(bucket: \"hort-osona\") |> range(start: -1h) |> filter(fn: (r) => r._measurement == \"temperature\")"
}
```

Telegram

Hi ha un node `node-red-contrib-telegrambot` que ens permet enviar i rebre missatges de Telegram.

Per instal·lar-lo, busquem `node-red-contrib-telegrambot` a la paleta.

Configuració típica: cal crear un bot de Telegram (com vam fer per a Uptime Kuma al Mòdul 1) i obtenir el token. Un cop configurat, podem enviar missatges amb un node `telegram sender`.

Function

El node **function** ens permet executar JavaScript personalitzat. El codi s'executa per a cada missatge que arriba al node, i retorna un o més missatges a la sortida.

Exemple: comptar quantes temperatures hem rebut:

```
let comptador = context.get("comptador") || 0;
comptador += 1;
context.set("comptador", comptador);

msg.payload = `Hem rebut ${comptador} lectures.`;
return msg;
```

Això és molt útil per a lògica personalitzada que no es pot expressar amb nodes visuals.

17.7 Debug i depuració

Node-RED té una eina de **debug** molt potent. Hi ha tres formes de depurar:

Pestanya Debug

A la dreta de la interfície, una pestanya "Debug" mostra tots els missatges enviats a nodes `debug`. Podem filtrar per node, per topic, per tipus de missatge.

Catch nodes

Si un flux llença un error, podem capturar-lo amb un **catch node** (categoria *function*). Connectem un catch node a un node que pot fallar, i ens avisa quan hi ha un error.

Status nodes

Els **status nodes** mostren l'estat d'un node concret. Per exemple, podem veure si un node MQTT està connectat o desconnectat.

17.8 Persistència i còpies de seguretat

Node-RED desa tots els fluxos a un fitxer anomenat `flows.json` dins del volum persistent (`/data`). Aquest fitxer és el cor del sistema: conté tota la lògica.

Per fer còpies de seguretat:

```
cp /home/bernat/homelab/data/nodered/flows.json \
  /home/bernat/homelab/backup/nodered-flows-$(date +%F).json
```

Per restaurar:

```
cp /home/bernat/homelab/backup/nodered-flows-2026-XX-XX.json \
  /home/bernat/homelab/data/nodered/flows.json
docker compose restart nodered
```

Alternativament, podem versionar el `flows.json` amb Git. És un fitxer de text, perfectament versionable. Les diferències entre versions es poden veure clarament.

17.9 Bones pràctiques

Organitzar els fluxos

Node-RED tendeix a créixer. Per evitar el caos:

- **Un flux per funcionalitat.** Per exemple, un flux "Alertes", un flux "Processament de dades", un flux "API".
- **Noms clars als nodes.** Un node anomenat "MQTT" no serveix; "Subscriu a hort/zona1/#" sí.
- **Comentaris als fluxos.** Podem afegir caixes de text descriptives al costat dels nodes.
- **Agrupar nodes relacionats.** Seleccionar múltiples nodes i agrupar-los.

Evitar la complexitat

Node-RED pot gestionar fluxos complexos, però cada complexitat afegida és un risc de fallada. Recomanacions:

- **Mantenir els fluxos petits i llegibles.** Si un flux té 30 nodes, cal dividir-lo.
- **Reutilitzar sub-fluxos.** Node-RED permet crear sub-fluxos (subflows) que encapsulen lògica complexa.
- **Documentar la lògica** amb comentaris.

Evitar nodes Function excessius

Els nodes `function` són útils, però si un flux té 10 nodes `function` seguits, potser és millor escriure un script Python independent. Node-RED és per a orquestració, no per a lògica de negoci complexa.

Control de versions

Com ja hem dit, versionar `flows.json` amb Git. Cada canvi important, un commit.

17.10 Configuració avançada

settings.js

El fitxer `settings.js` (a `/data/`) controla la configuració global de Node-RED. Algunes directives útils:

```
module.exports = {
  // Port HTTP
  uiPort: 1880,

  // Autenticació (recomanable!)
  adminAuth: {
    type: "credentials",
    users: [{
      username: "bernat",
      password: "$2a$08$..." // hash bcrypt
    }]
  },

  // HTTPS (opcional, ho fariem si exposéssim Node-RED a Internet)
  // https: { ... },

  // Timezone
  timezone: "Europe/Madrid",

  // Nivell de log
  logging: {
    console: {
      level: "info",
      metrics: false,
      audit: false
    }
  },

  // Directori de fluxos
  flowFile: "flows.json",

  // Configuració de nodes personalitzats
  // ...
}
```

Cal **activar l'autenticació** (`adminAuth`) si exposem Node-RED a una xarxa que no és totalment privada. Al BernatLab, amb Tailscale, és menys crític, però és bona pràctica.

Còpies de seguretat automàtiques

Podem programar un node `exec` que periòdicament faci una còpia de `flows.json` a un altre lloc. Per exemple:

```
cp /data/flows.json /backup/nodered-$(date +%F).json
```

I cridar-lo amb un node `cron` o un node `inject` configurat per executar-se cada dia.

17.11 Integració amb la resta del BernatLab

Un cop tenim Node-RED funcionant, podem:

- Subscriure'ns a MQTT per rebre dades dels sensors.
- Consultar InfluxDB per obtenir dades històriques.
- Enviar alertes a Telegram.
- Crear API endpoints amb `http in + http response`.
- Publicar comandes a MQTT per controlar actuadors.
- Cridar serveis externs via HTTP.

Al Capítol 18 veurem fluxos concrets que posen tot això en pràctica.

17.12 El primer flux útil: comptador de missatges

Connectem-nos a MQTT i comptem quantes lectures de temperatura hem rebut:

1. Un node `mqtt in` subscrit a `hort/+/temperatura/aire`.
2. Un node `function` que compta missatges.
3. Un node `debug` que mostra el comptador periòdicament.

Codi del node `function`:

```
let comptador = context.get("comptador") || 0;
comptador += 1;
context.set("comptador", comptador);

if (comptador % 10 === 0) {
  msg.payload = `Hem rebut ${comptador} lectures de temperatura.`;
  return msg;
} else {
  return null;
}
```

Aquest flux ens permet veure, cada 10 lectures, quantes hem rebut en total. Simple, però útil.

17.13 Codi JavaScript als nodes Function

Algunes pautes per escriure JavaScript als nodes Function:

- **`context`**: un magatzem de valors que persisteixen entre missatges. Útil per a estats, comptadors, etc.
- **`flow`**: un magatzem compartit per tots els nodes del mateix flux.
- **`global`**: un magatzem compartit per tota la instància.
- **`msg`**: l'objecte del missatge actual. Hem de retornar-lo (o un array) per enviar-lo a la sortida.

- **node.warn(msg)** i **node.error(msg)**: missatges d'avís i d'error que apareixen a la consola i a la pestanya debug.

Exemple: detectar temperatures extremes i generar un avís:

```
if (msg.payload > 35) {
  node.warn("Temperatura massa alta: " + msg.payload);
}
if (msg.payload < 5) {
  node.warn("Temperatura massa baixa: " + msg.payload);
}
return msg;
```

17.14 Rendiment

Node-RED és un programa JavaScript que s'executa a Node.js. En una Raspberry Pi 4 amb 4 GB de RAM, podem allotjar Node-RED còmodament, sempre que no tinguem centenars de fluxos actius.

Recomanacions:

- **Evitar nodes function** amb bucles infinits.
- **Limitar el nombre de nodes debug**: cada missatge enviat a debug es guarda a memòria.
- **Usar link nodes** per dividir fluxos llargs en parts separades.
- **No abusar de nodes delay**: cusen el missatge a la memòria.

17.15 Esquema conceptual

Esquema Mermaid (text):

```
graph TB
  subgraph NR["Node-RED"]
    IN1["mqtt in<br/>(hort/#)"]
    IN2["influxdb in<br/>(consulta)"]
    IN3["http in<br/>(API)"]
    F1["function<br/>(neteja)"]
    F2["function<br/>(alerta)"]
    F3["function<br/>(agregació)"]
    OUT1["mqtt out<br/>(publica)"]
    OUT2["telegram sender"]
    OUT3["influxdb out"]
    OUT4["http response"]
  end

  IN1 --> F1
  IN1 --> F2
  F1 --> F3
  F2 --> OUT2
  F3 --> IN2
  IN2 --> OUT3
  IN3 --> F3
  F3 --> OUT1
  IN3 --> OUT4
```

17.16 Errors habituals

Error 1: oblidar fer Deploy. Síntoma: els canvis no s'apliquen. Solució: clicar el botó **Deploy** a la part superior dreta.

Error 2: no connectar bé un node. Síntoma: el flux no rep missatges. Solució: comprovar que els cables estan connectats als ports correctes.

Error 3: credencials incorrectes al broker MQTT. Síntoma: el node `mqtt in` mostra "Disconnected". Solució: revisar l'usuari, la contrasenya i el port.

****Error 4: nodes Function que retornen `null` quan no toca**.** Síntoma: el flux s'atura. Solució: retornar sempre un `msg` o un array de `msg`.

Error 5: cicles infinits. Síntoma: Node-RED es penja o consumeix tota la CPU. Solució: dissenyar fluxos acíclics, o usar nodes `delay` per evitar bucles.

Error 6: no desar els fluxos. Síntoma: en reiniciar Node-RED, els fluxos es perden. Solució: fer Deploy sempre que fem canvis, i comprovar que el volum `/data` està muntat correctament.

17.17 Resum

Hem après què és Node-RED, com es programa visualment amb nodes i fluxos, quins nodes són útils per al BernatLab, com es connecta a MQTT, InfluxDB i Telegram, i com es depuren els fluxos. Hem vist un primer flux útil, hem après bones pràctiques d'organització i rendiment, i hem après a fer còpies de seguretat. En el proper capítol posarem tot això en pràctica amb exemples reals: neteja de dades, agregacions, detecció d'anomalies i alertes a Telegram.

17.18 Exercicis pràctics

1. Desplega Node-RED al BernatLab amb la configuració que hem vist.
2. Crea un flux "Hola Món" amb `inject` i `debug`. Deploy. Comprova que funciona.
3. Instal·la el node `node-red-contrib-influxdb` des de la paleta.
4. Crea un flux que es subscrigui a `hort/+/+` i compti missatges amb un node `function`.
5. Configura l'autenticació a `settings.js` per accedir a Node-RED.
6. Exporta els fluxos a JSON i guarda'ls a `~/homelab/backup/`.
7. Documenta al README del projecte quin és el patró de noms que fas servir per als nodes.

Comandes útils:

```
# Veure els logs
docker compose logs -f nodered

# Fer una còpia de seguretat
cp /home/bernat/homelab/data/nodered/flows.json \
  /home/bernat/homelab/backup/nodered-flows-$(date +%F).json

# Reiniciar Node-RED
docker compose restart nodered
```

Paraules clau: **Node-RED, flux, node, paleta, MQTT, InfluxDB, Telegram, Function, debug, deploy, context, flow, global, settings.js, adminAuth, còpia de seguretat, versionat, Git, HTTP, http in, http response, rendiment, organització, subflows, alertes, sensors, hort, BernatLab.**

Capítol 18 — Fluxos pràctics

*“La diferència entre un prototip i un sistema en producció és, sovint, un bon flux de Node-RED.”**

18.1 Què aprendrem

En aquest capítol construirem, pas a pas, **fluxos reals** que posen en pràctica tot el que hem après fins ara. Cada flux és una peça autònoma que podem integrar al BernatLab:

1. **Netejar dades:** convertir payloads, validar valors, descartar lectures impossibles.
2. **Detectar gelades:** alerta immediata quan la temperatura baixa de 2 °C.
3. **Detectar sòl sec:** alerta quan la humitat del sòl baixa del 30 %.
4. **Mitjana mòbil:** suavitzar lectures sorolloses amb una mitjana de les últimes N lectures.
5. **Controlar el reg:** activar una vàlvula de reg quan la humitat del sòl és baixa.
6. **Monitorar sensors inactius:** detectar sensors que no han publicat en X minuts.
7. **Publicar resums periòdics:** cada hora, enviar un resum a Telegram.

Cada flux ve amb:

- Descripció del problema que resol.
- Esquema visual (descripció dels nodes i les connexions).
- Codi dels nodes `function`.
- Configuració dels nodes MQTT i InfluxDB.
- Consideracions i extensions.

Comencem.

18.2 Flux 1: netejar i validar dades

Problema

Els sensors publiquen lectures que poden tenir errors: temperatures impossibles (200 °C, -100 °C), humitats fora de rang (> 100 %, < 0 %), valors NaN, etc. Volem netejar aquestes dades abans d'escriure-les a InfluxDB.

Solució

Un flux que:

1. Es subscriu a tots els temes de mesura.
2. Parseja el payload.
3. Valida els valors segons el tipus.
4. Si el valor és vàlid, l'envia a Telegram (perquè l'escriui a InfluxDB).

5. Si no, l'envia a debug i a un log d'errors.

Implementació

Nodes necessaris:

- **mqtt in**: subscriu a `hort/+/+/+`.
- **function**: valida i neteja.
- **mqtt out**: re-publica a un topic "net".
- **debug**: per veure els rebutjos.

Codi del node **function**:

```
// Validar i netejar el missatge
const msgOriginal = msg;

// Rang vàlid segons el tipus
const limits = {
  'graus_C': [-20, 50],
  '%': [0, 100],
  'hPa': [800, 1100],
  'lux': [0, 200000]
};

try {
  const payload = typeof msg.payload === 'string'
    ? JSON.parse(msg.payload)
    : msg.payload;

  const valor = payload.valor;
  const unitat = payload.unitat;

  // Validar
  if (typeof valor !== 'number' || isNaN(valor)) {
    node.warn("Valor no numèric: " + JSON.stringify(payload));
    return null;
  }

  if (!limits[unitat]) {
    node.warn("Unitat desconeguda: " + unitat);
    return null;
  }

  const [min, max] = limits[unitat];
  if (valor < min || valor > max) {
    node.warn(`Valor fora de rang: ${valor} ${unitat}`);
    return null;
  }

  // Re-publicar a un topic net
  msg.payload = payload;
  msg.topic = msg.topic + '/net';
  return msg;
} catch (e) {
  node.error("Error parsejant: " + e.message);
  return null;
}
```

Aquest codi:

- Parseja el payload JSON.
- Comprova que el valor sigui un número.
- Comprova que l'unitat sigui coneguda.
- Comprova que el valor estigui dins del rang esperat.
- Si tot és correcte, retorna el missatge. Si no, retorna `null` (descarta).

Una millora: usar un node `rbe` (Report By Exception) o `delay` per evitar que el mateix valor es publiqui massa sovint.

18.3 Flux 2: alerta de gelada

Problema

A l'Hort Osona hi ha plantes sensibles a les gelades (tomàquets, pebrots, albergínies). Si la temperatura baixa de 2 °C a la nit, volem rebre un missatge immediat a Telegram per poder actuar (cobrir les plantes, regar per crear una capa de gel protectora, etc.).

Solució

Un flux que:

1. Es subscriu a `hort/+/temperatura/aire`.
2. Comprova si la temperatura és inferior a 2 °C.
3. Si sí, envia un missatge a Telegram.
4. Per evitar spam, només envia una alerta cada 30 minuts com a mínim.

Implementació

Nodes:

- `mqtt in`: subscriu a `hort/+/temperatura/aire`.
- `function`: comprova la temperatura.
- `telegram sender`: envia el missatge.
- `delay`: limita la freqüència d'alertes.

Codi del node `function`:

```
const payload = typeof msg.payload === 'string'
  ? JSON.parse(msg.payload)
  : msg.payload;

const valor = payload.valor;
const zona = msg.topic.split('/')[1];

if (valor < 2) {
  msg.payload = `❄️ ALERTA GELADA\n\nZona: ${zona}\nTemperatura: ${valor}°C\nHora: ${new Date().toISOString()}`;
  return msg;
}

return null;
```


Aquest codi:

- Parseja el payload.
- Extreu la zona del topic.
- Si la temperatura és $< 2\text{ }^{\circ}\text{C}$, construeix un missatge d'alerta.
- Retorna el missatge; si no, retorna `null`.

Configuració del node `telegram sender`:

- **Bot**: el bot que hem configurat a Uptime Kuma o un de nou.
- **Chat ID**: el nostre chat ID.
- **Message**: `{{payload}}` (que serà el missatge del node function).

Per evitar spam, afegim un node `delay` configurat a 30 minuts entre missatges.

18.4 Flux 3: alerta de sòl sec

Problema

Si la humitat del sòl baixa del 30 %, les plantes poden patir estrès hídric. Volem rebre un avís.

Solució

Similar al flux de gelada, però amb el llindar del 30 % i el topic `hort/+/humitat/sol`.

Codi del node `function`:

```
const payload = typeof msg.payload === 'string'
  ? JSON.parse(msg.payload)
  : msg.payload;

const valor = payload.valor;
const zona = msg.topic.split('/')[1];

if (valor < 30) {
  msg.payload = `■ ALERTA SÒL SEC\n\nZona: ${zona}\nHumitat: ${valor}%\nHora: ${new Date().toISOString()}`;
  return msg;
}

return null;
```

18.5 Flux 4: mitjana mòbil

Problema

Alguns sensors (sobretot els barats) tenen soroll: la temperatura pot oscil·lar $\pm 1\text{ }^{\circ}\text{C}$ entre lectures consecutives. Volem una mitjana mòbil de les últimes 10 lectures per suavitzar el senyal.

Solució

Un flux que:

1. Es subscriu a `hort/+/temperatura/aire`.
2. Desa les últimes 10 lectures en memòria (context).

3. Calcula la mitjana.

4. Publica el valor suavitzat a un nou topic `hort/+/temperatura/aire/suavitzat`.

Implementació

Codi del node `function`:

```
const zona = msg.topic.split('/')[1];
const clau = `historial_${zona}`;

let historial = context.get(clau) || [];
const nouValor = (typeof msg.payload === 'string'
  ? JSON.parse(msg.payload)
  : msg.payload).valor;

historial.push(nouValor);
if (historial.length > 10) {
  historial.shift();
}

context.set(clau, historial);

const mitjana = historial.reduce((a, b) => a + b, 0) / historial.length;

msg.payload = JSON.stringify({
  valor: Math.round(mitjana * 100) / 100,
  unitat: "graus_C"
});
msg.topic = `hort/${zona}/temperatura/aire/suavitzat`;

return msg;
```

Aquest codi:

- Manté un historial de les últimes 10 lectures per zona.
- Calcula la mitjana.
- Publica el resultat a un nou topic.

Ara Telegraf (si ho configurem adequadament) pot subscriure's a aquest topic i guardar la versió suavitzada, o podem consultar InfluxDB directament.

18.6 Flux 5: vàlvula de reg (control d'actuadors)

Problema

Quan la humitat del sòl baixa del 30 %, volem obrir una vàlvula de reg durant 5 minuts, i després tancar-la automàticament.

Solució

Un flux que:

1. Es subscriu a `hort/+/humitat/sol`.
2. Comprova si la humitat és < 30 % i si la vàlvula està tancada.
3. Publica una comanda `ON` a `hort/control/reg`.

4. Espera 5 minuts.
5. Publica una comanda **OFF** al mateix topic.

Implementació

Codi del node `function` (lectura):

```
const payload = typeof msg.payload === 'string'
  ? JSON.parse(msg.payload)
  : msg.payload;

const valor = payload.valor;
const zona = msg.topic.split('/')[1];

// Comprovar l'estat actual de la vàlvula
const clauEstat = `valvula_${zona}`;
const estat = context.get(clauEstat) || 'OFF';

if (valor < 30 && estat === 'OFF') {
  context.set(clauEstat, 'ON');

  // Enviar comanda ON
  msg.payload = 'ON';
  msg.topic = `hort/control/reg/${zona}`;
  msg.accion = 'ON';
  return msg;
}

return null;
```

Codi del node `function` (tancament després de 5 min):

```
// Aquest node s'activa quan arriba el missatge
const clauEstat = `valvula_${msg.zona}`;
context.set(clauEstat, 'OFF');

msg.payload = 'OFF';
msg.topic = msg.topic.split('/').slice(0, -1).join('/');
return msg;
```

Aquest és un exemple simplificat. En un sistema real, caldria considerar:

- **Confirmació de la vàlvula:** un node MQTT que es subscriu a `hort/control/reg/status` per saber si la vàlvula s'ha obert realment.
- **Mecanismes de seguretat:** no regar massa, no regar si plou, etc.
- **Horaris:** només regar durant el dia, no a la nit.

Al BernatLab, podem implementar la vàlvula amb un actuador Sonoff, Shelly, o similar, controlat per MQTT. Però la implementació física la veurem al Mòdul 3.

18.7 Flux 6: monitorar sensors inactius

Problema

Si un sensor deixa de publicar (bateria esgotada, pèrdua de Wi-Fi, avaria), volem saber-ho. Podríem fer pings al sensor, però és més eficient mirar l'última vegada que ha publicat.

Solució

Un flux que:

1. Cada 5 minuts, comprova l'última publicació de cada sensor.
2. Si l'última publicació és de fa més de 10 minuts, envia una alerta.

Implementació

Nodes:

- **inject**: dispara cada 5 minuts.
- **influxdb in**: consulta l'última publicació per zona.
- **function**: comprova si ha passat massa temps.
- **telegram sender**: envia l'alerta.

Consulta InfluxDB:

```
from(bucket: "hort-osona")
  |> range(start: -1h)
  |> filter(fn: (r) => r._measurement == "temperatura")
  |> group(columns: ["zona"])
  |> last()
  |> keep(columns: ["_time", "zona", "_value"])
```

Codi del node **function**:

```
const ara = new Date();
const limitMinuts = 10;

for (const lectura of msg.payload) {
  const ultima = new Date(lectura._time);
  const minuts = (ara - ultima) / 1000 / 60;

  if (minuts > limitMinuts) {
    msg.payload = `■ SENSOR INACTIU\n\nZona: ${lectura.zona}\nÚltima lectura: ${ultima.toISOString()}`;
    return msg;
  }
}

return null;
```

Aquest flux ens avisa quan un sensor porta més de 10 minuts sense publicar.

18.8 Flux 7: resum diari a Telegram

Problema

Cada matí, volem un resum de les condicions de l'hort: temperatura mitjana, humitat mitjana, anomalies detectades.

Solució

Un flux que:

1. Es dispara cada dia a les 8:00.
2. Consulta InfluxDB per obtenir estadístiques del dia anterior.

3. Construeix un missatge amb el resum.
4. L'envia a Telegram.

Implementació

Nodes:

- **inject**: dispara a les 8:00 cada dia.
- **influxdb in** (múltiples): consultes per a temperatura, humitat, etc.
- **function**: construeix el resum.
- **telegram sender**: envia el missatge.

Codi del node **function**:

```
// Suposem que rebem múltiples consultes en una sola msg
// Cada msg.payload conté un array de resultats

const resum = {
  temperatura: null,
  humitat: null,
  anomalies: []
};

for (const item of msg.payload) {
  if (item._measurement === 'temperatura') {
    resum.temperatura = {
      mitjana: item.mitjana,
      max: item.maxim,
      min: item.minim
    };
  }
  if (item._measurement === 'humitat') {
    resum.humitat = {
      mitjana: item.mitjana,
      max: item.maxim,
      min: item.minim
    };
  }
}

let text = `■ RESUM DIARI HORT OSONA\n\n`;
text += `■■■ Temperatura: ${resum.temperatura?.mitjana?.toFixed(1)}°C (max ${resum.temperatura?.maxim?.toFixed(1)}, min ${resum.temperatura?.minim?.toFixed(1)})\n`;
text += `■ Humitat: ${resum.humitat?.mitjana?.toFixed(1)}% (max ${resum.humitat?.maxim?.toFixed(1)}, min ${resum.humitat?.minim?.toFixed(1)})\n`;

if (resum.anomalies.length > 0) {
  text += `\n■■■ Anomalies:\n` + resum.anomalies.join('\n');
}

msg.payload = text;
return msg;
```

Aquest flux ens permet començar el dia amb una visió general de l'estat de l'hort.

18.9 Patrons avançats

Patró: confirmació d'acció

Quan publiquem una comanda a un actuator, volem saber si s'ha executat. Patró:

1. Publicar comanda a `hort/control/actuador`.
2. Subscriure's a `hort/control/actuador/status`.
3. Si rebem confirmació dins de X segons, tot bé.
4. Si no, alerta.

Patró: debounce

Per evitar alertes duplicades per un sol esdeveniment, podem usar un node `delay` o un node `rbe` (Report By Exception).

Per exemple, si la temperatura baixa de 2 °C durant 5 minuts consecutius, no volem 5 alertes. Volem 1 alerta al principi, i 1 altra quan es recuperi.

Patró: dead letter

Quan un flux llença un error, podem capturar-lo amb un node `catch` i enviar-lo a un topic d'errors. Per exemple, `bernatlab/errors`. Això ens permet tenir un log centralitzat d'errors.

Patró: heartbeat

Cada N minuts, publiquem un missatge a `bernatlab/heartbeat`. Si Uptime Kuma o un altre sistema detecta que aquest heartbeat ha parat, sabem que Node-RED té problemes.

18.10 Subflows: components reutilitzables

Si tenim un patró que es repeteix, podem crear un **subflow** (subflux) que encapsuli la lògica. Per exemple, un subflow "Alerta si valor < llindar" podria rebre dos paràmetres (llindar, missatge) i aplicar la lògica de detecció i enviament.

Per crear un subflow:

1. Seleccionem els nodes que volem encapsular.
2. Anem a **Subflows** → **Create subflow from selection**.
3. Definim els paràmetres d'entrada.
4. Usem el subflow com si fos un node més.

18.11 Debug avançat

Per depurar fluxos complexos, podem usar:

- **Node debug** amb `complete msg object`: mostra tots els camps del missatge, no només el payload.
- **Node catch**: captura errors i els envia a un node de log.
- **Node status**: mostra canvis d'estat dels nodes (per exemple, quan un MQTT broker es connecta o desconnecta).

Exemple de configuració de debug:

```
{
```

```
"active": true,  
"tosidebar": true,  
"console": false,  
"complete": "true",  
"targetType": "msg",  
"statusVal": "",  
"statusType": "auto"  
}
```

Això mostra el missatge complet a la pestanya debug, sense imprimir-lo a la consola.

18.12 Optimització i rendiment

Quan els fluxos creixen, Node-RED pot alentir-se. Algunes optimitzacions:

- **Limitar els nodes `debug`**: cada missatge enviat a debug es guarda a memòria.
- **Usar `link nodes`** en lloc de cables llargs.
- **Processar dades en lots** quan sigui possible.
- **Cachejar consultes a InfluxDB** si es fan moltes vegades amb els mateixos paràmetres.
- **Evitar nodes `function` amb codi massa complex**: millor encapsular en mòduls.

18.13 Proves d'integració

Quan construïm un flux, hem de provar-lo de punta a punta. El procediment és:

1. **Provar amb dades simulades**: usar el simulador Python del Capítol 14.
2. **Validar el comportament**: rebre una alerta, veure una gràfica, etc.
3. **Provar casos extrems**: què passa si arriba un valor null? I un valor molt gran?
4. **Documentar el flux**: afegir comentaris, guardar una captura de pantalla, desar el JSON.

18.14 Resum

Hem après a construir set fluxos pràctics que cobreixen els casos d'ús més habituals al BernatLab: neteja de dades, alertes per gelada i sòl sec, mitjana mòbil, control d'actuadors, monitoratge de sensors, resums diaris. Hem vist patrons avançats com la confirmació d'accions, el debounce, el dead letter i el heartbeat. Hem après a organitzar la lògica amb subflows, a depurar amb els nodes adequats, i a optimitzar el rendiment. En el proper capítol veurem Grafana, l'eina de visualització que ens permetrà veure totes aquestes dades en gràfiques.

18.15 Exercicis pràctics

1. Implementa el flux 1 (neteja de dades) al teu Node-RED.
2. Implementa el flux 2 (alerta de gelada) i prova'l amb el simulador Python.
3. Implementa el flux 4 (mitjana mòbil) i observa la diferència amb les lectures originals.
4. Implementa el flux 6 (monitorar sensors inactius) i comprova que detecta bé quan pares el simulador.

5. Implementa el flux 7 (resum diari) i configura'l perquè s'envii a les 8:00.
6. Crea un subflow "Alerta si valor < llindar" que puguis reutilitzar en diferents parts del sistema.
7. Exporta tots els fluxos a JSON i guarda'ls a `~/home lab/backup/`.

Paraules clau: **Node-RED, flux pràctic, neteja de dades, validació, alerta de gelada, alerta de sòl sec, mitjana mòbil, vàlvula de reg, sensor inactiu, resum diari, subflow, debounce, heartbeat, dead letter, confirmació, optimització, debug, catch, rbe, delay, InfluxDB, MQTT, Telegram, sensors, hort, BernatLab, Hort Osona.**

Capítol 19 — Grafana: visualitzar les dades

""Una gràfica val més que mil números. I una alerta visual val més que mil correus.""

19.1 Què és Grafana

Grafana és una plataforma de codi obert per visualitzar, analitzar i monitorar dades. Originalment pensada per a mètriques de sistemes, s'ha convertit en l'eina de referència per a qualsevol cosa que es pugui representar en gràfiques, taules, gauges, heatmaps, i un llarg etcètera.

Grafana es connecta a **moltes fonts de dades** (InfluxDB, Prometheus, MySQL, PostgreSQL, Elasticsearch, Loki, fins i tot APIs HTTP) i ens permet crear **dashboards** combinant panells (visualitzacions individuals). Cada panell té la seva pròpia consulta, configuració visual, i opcionalment alertes.

Al BernatLab, Grafana serà la finestra visual del sistema Hort Osona. Aquí veurem:

- L'evolució de la temperatura al llarg del temps.
- La comparació entre zones.
- Les alertes visuals (un termòmetre que es posa vermell si la temperatura baixa de 2 °C).
- L'estat dels sensors (quins estan actius, quants porten més de X minuts sense publicar).
- Estadístiques agregades (mitjanes diàries, màxims mensuals, etc.).

19.2 Per què Grafana i no la UI d'InfluxDB

InfluxDB 2.x té la seva pròpia interfície web amb gràfiques bàsiques, però Grafana és molt més potent:

- **Més tipus de visualitzacions:** gauges, heatmaps, bar charts, pie charts, world maps, etc.
- **Més fonts de dades:** podem combinar InfluxDB amb altres fonts (Prometheus, MySQL) en un sol dashboard.
- **Templating:** podem crear variables que canvien el contingut del dashboard (per exemple, triar quina zona veure).
- **Alertes visuals:** podem configurar que un panell canviï de color segons el valor.
- **Compartició pública:** podem fer dashboards públics amb URLs específiques.
- **Anotacions:** podem afegir marques a les gràfiques (per exemple, "vàlvula de reg oberta" o "alerta de gelada").

19.3 Instal·lació al BernatLab

Grafana es desplega amb Docker. La imatge oficial és `grafana/grafana:11.0` (o l'última estable).

Definició al docker-compose.yml

```
services:
  grafana:
    image: grafana/grafana:11.0
    container_name: grafana
    restart: unless-stopped
    ports:
      - "3000:3000"      # interfície web
    volumes:
      - /home/bernat/homelab/data/grafana:/var/lib/grafana
    environment:
      - GF_SECURITY_ADMIN_USER=bernat
      - GF_SECURITY_ADMIN_PASSWORD=${GRAFANA_ADMIN_PASSWORD}
      - GF_USERS_ALLOW_SIGN_UP=false
      - GF_INSTALL_PLUGINS=
      - GF_SERVER_ROOT_URL=http://100.115.134.76:3000
      - GF_ANALYTICS_REPORTING_ENABLED=false
```

Cal un volum persistent per desar dashboards, fonts de dades, alertes i configuracions. Les credencials les passem com a variables d'entorn.

Primer accés

Un cop en marxa, accedim a <http://100.115.134.76:3000>. Ens demanarà usuari (**bernat**) i contrasenya. El primer cop, Grafana ens portarà a la pantalla de benvinguda.

19.4 Configurar la font de dades: InfluxDB

El primer pas és connectar Grafana a InfluxDB. Anem a **Configuration** → **Data sources** → **Add data source** i seleccionem **InfluxDB**.

Configuració:

- **Name:** `InfluxDB-HortOsona` (o el nom que preferim).
- **Query language:** `Flux` (per InfluxDB 2.x).
- **URL:** `http://influxdb:8086` (el nom del servei dins del `docker-compose.yml`).
- **Organization:** `bernatlab`.
- **Token:** el token d'accés per a Grafana (el que hem creat al Capítol 15).
- **Default bucket:** `hort-osona`.

Fes clic a **Save & test**. Si tot és correcte, veureu "Data source is working".

19.5 Crear el primer panell

Ara crearem un panell per visualitzar la temperatura de les últimes 24 hores.

1. Anem a **Create** → **Dashboard** → **Add new panel**.
2. A la consulta, seleccionem la font de dades `InfluxDB-HortOsona`.
3. A la query, escrivim:

```
from(bucket: "hort-osona")
  |> range(start: v.timeRangeStart, stop: v.timeRangeStop)
  |> filter(fn: (r) => r._measurement == "temperatura")
```

```
|> filter(fn: (r) => r._field == "valor")
|> aggregateWindow(every: v.windowPeriod, fn: mean, createEmpty: false)
|> yield(name: "mean")
```

Aquesta consulta agafa les dades del rang temporal seleccionat, les filtra per mesura **temperatura** i camp **valor**, i n'agrega la mitjana per finestres temporals.

1. A la dreta, configurem: - **Panel title**: "Temperatura". - **Unit**: Celsius. - **Visualization**: Time series.
2. Fem clic a **Apply**.

Veurem una gràfica amb l'evolució de la temperatura al llarg del temps. Si tenim dades al InfluxDB, apareixeran aquí.

19.6 Tipus de panells

Grafana ofereix molts tipus de panells. Els més útils per al BernatLab:

Time series

Línies al llarg del temps. Perfecte per temperatures, humitats, etc.

Stat

Un valor únic amb format gran. Útil per mostrar l'últim valor d'una mesura, l'estat d'un sensor, etc.

Gauge

Un indicador circular amb un rang. Útil per mostrar una mesura dins d'un rang (per exemple, humitat del 0 al 100 %).

Bar chart

Barres verticals o horitzontals. Útil per comparar valors entre zones.

Table

Una taula. Útil per mostrar llistes de dades o resums.

Heatmap

Una matriu de colors que mostra la densitat. Útil per veure patrons en dades de molts sensors.

19.7 Dashboard d'Hort Osona: estructura

Crearem un dashboard amb aquests panells:

1. **Temperatura actual**: un **stat** amb l'últim valor de temperatura per zona.
2. **Humitat actual**: un **stat** amb l'últim valor d'humitat per zona.
3. **Evolució de la temperatura**: un **time series** amb la temperatura de les últimes 24 h.
4. **Evolució de la humitat del sòl**: un **time series** amb la humitat del sòl.
5. **Lluminositat**: un **time series** amb la lluminositat.
6. **Estat dels sensors**: una taula amb l'última publicació de cada sensor.

7. **Alertes recents**: una taula amb les alertes de les últimes 24 h (si les tenim guardades).

Podem organitzar-los en files i columnes, i configurar el refresc automàtic (per exemple, cada 30 segons).

19.8 Variables: filtres dinàmics

Grafana permet crear **variables** que canvien el contingut dels panells. Per exemple, podem crear una variable `zona` que ens permet triar quina zona veure:

1. Anem a **Dashboard settings** → **Variables** → **New variable**.

2. **Name**: `zona`.

3. **Type**: `Query`.

4. **Data source**: `InfluxDB-HortOsona`.

5. **Query**:

```
import "influxdata/influxdb/schema"
schema.tagValues(bucket: "hort-osona", tag: "zona")
```

1. **Multi-value**: `true`.

2. **Include All option**: `true`.

Ara, a la part superior del dashboard, apareixerà un selector de zona. Els panells que usin la variable `$zona` es filtraran automàticament.

Per usar la variable a les queries:

```
from(bucket: "hort-osona")
  |> range(start: v.timeRangeStart, stop: v.timeRangeStop)
  |> filter(fn: (r) => r._measurement == "temperatura")
  |> filter(fn: (r) => r.zona =~ /^${zona:regex}$/)
  |> filter(fn: (r) => r._field == "valor")
  |> aggregateWindow(every: v.windowPeriod, fn: mean, createEmpty: false)
```

Aquí, `${zona:regex}` converteix la variable en una expressió regular. Si la variable permet múltiples valors, Grafana els combina amb `|`.

19.9 Alertes a Grafana

Grafana permet configurar **alertes visuals** que canvien el color d'un panell segons el valor. Per exemple, podem fer que el panell de temperatura es posi vermell si baixa de 2 °C.

1. Al panell de temperatura, anem a la secció **Thresholds**.

2. Afegim un llindar: - **Mode**: `Absolute`. - **Value**: `2`. - **Color**: vermell.

3. Afegim un altre: - **Value**: `0`. - **Color**: blau (per gelades severes).

Ara, el panell canviarà de color segons la temperatura. Això és una alerta visual molt intuïtiva.

A més de les alertes visuals, Grafana pot enviar alertes reals (per correu, webhook, Telegram, etc.) quan un panell compleix una condició. Per configurar alertes:

1. Al panell, anem a **Alert** → **Create alert rule**.

2. Definim la condició: `WHEN last() OF A IS BELOW 2`.

3. Definim la carpeta d'avaluació: cada 1 minut.

4. Afegim un contact point: Telegram (configurat prèviament).

Grafana avaluarà la condició periòdicament i enviarà un missatge a Telegram quan es compleixi.

19.10 Contact points: configurar Telegram

Per rebre alertes per Telegram, cal configurar un contact point.

1. Anem a **Alerting** → **Contact points** → **New contact point**.

2. **Name**: Telegram Bernat.

3. **Type**: Telegram.

4. **Telegram bot token**: el token del nostre bot.

5. **Telegram chat ID**: el nostre chat ID.

6. **Save**.

Ara, quan creem una regla d'alerta, podem triar aquest contact point com a destinatari.

19.11 Plantilles d'alerta

Quan Grafana envia una alerta, el missatge es pot personalitzar amb plantilles. Per exemple:

■■ ALERTA HORT OSONA

Temperatura massa baixa: `{{ $values.A }}` °C

Zona: `{{ $labels.zona }}`

Hora: `{{ now }}`

Podem definir la plantilla a la configuració de la regla d'alerta, secció **Message**.

19.12 Compartició i accés públic

Si volem, podem fer un dashboard accessible sense autenticació, compartint-lo amb un enllaç. Per fer-ho:

1. Anem a **Dashboard settings** → **Sharing**.

2. Activem **Public dashboard**.

3. Triem un nom d'enllaç (per exemple, `hort-osona-public`).

4. Configurem quin temps es pot consultar (per defecte, 30 dies).

Ara qualsevol pot accedir al dashboard a

`http://100.115.134.76:3000/public-dashboards/hort-osona-public`.

Això és útil si volem ensenyar les dades a algú sense donar-li accés a tota la interfície de Grafana.

19.13 Exportar i importar dashboards

Quan tenim un dashboard ben configurat, podem **exportar-lo a JSON** per:

- Guardar-lo com a còpia de seguretat.
- Compartir-lo amb altres.
- Versionar-lo amb Git.

Anem a **Dashboard settings** → **JSON model** → **Copy to clipboard** o **Export**.

Per **importar**:

1. Anem a **Create** → **Import**.
2. Enganxem el JSON o pujem un fitxer.

Això ens permet moure dashboards entre instàncies de Grafana.

19.14 Provar el dashboard

Amb el simulador Python del Capítol 14 publicant, podem validar que Grafana mostra les dades correctament. Si no apareixen, cal revisar:

- La consulta Flux: és correcta?
- El rang temporal: estem mirant les últimes 24 h?
- Les dades són a InfluxDB? (Comprovar amb `influx query`)

També podem crear un panell "raw" que mostri les dades sense agregacions, per veure exactament què hi ha.

19.15 Rendiment

En una Raspberry Pi 4, hem de ser curosos amb el nombre de panells i la complexitat de les consultes:

- **Limitar el nombre de panells per dashboard**: 8-12 és un bon màxim.
- **Evitar consultes molt complexes** que triguin més de 5 segons.
- **Configurar el `cache`** correctament per no repetir consultes idèntiques.
- **Limitar el rang temporal per defecte**: les últimes 24 h en comptes de "tots els temps".

Si Grafana es torna lent, podem:

- Augmentar la memòria assignada (`GF_DATABASE_MEMORY_CACHE`).
- Optimitzar les consultes a InfluxDB.
- Reduir el nombre de panells.

19.16 Integració amb la resta del BernatLab

Grafana s'integra amb:

- **InfluxDB**: la font de dades principal.
- **Uptime Kuma**: podem afegir un panell que mostri l'estat dels serveis (via una API).

- **Homepage:** podem afegir una targeta amb l'enllaç al dashboard.
- **Telegram:** contact point per a alertes.

A més, podem fer que Grafana cridi webhooks quan passa alguna cosa. Per exemple, quan es detecta una gelada, Grafana pot trucar a un webhook de Node-RED, que al seu torn pot activar actuadors.

19.17 Esquema conceptual

Esquema Mermaid (text):

```
graph TB
    subgraph Dades["Dades"]
        I["InfluxDB<br/>(hort-osona)"]
    end

    subgraph Grafana["Grafana"]
        DS["Data Source<br/>InfluxDB-HortOsona"]
        DASH["Dashboard Hort Osona"]
        P1["Panell temperatura"]
        P2["Panell humitat"]
        P3["Panell estat sensors"]
        ALERT["Alertes visuals"]
        CP["Contact points<br/>(Telegram)"]
    end

    subgraph Usuaris["Usuaris"]
        U["Bernat (privat)"]
        PUB["Públic"]
        TG["Telegram"]
    end

    I --> DS
    DS --> DASH
    DASH --> P1
    DASH --> P2
    DASH --> P3
    P1 --> ALERT
    P2 --> ALERT
    P3 --> ALERT
    ALERT --> CP
    DASH --> U
    DASH --> PUB
    CP --> TG
```

19.18 Errors habituals

Error 1: token incorrecte. Síntoma: la font de dades no es connecta. Solució: revisar el token al `.env`.

Error 2: consulta Flux massa lenta. Síntoma: els panells trigem molt a carregar. Solució: simplificar la consulta, agregar abans.

Error 3: no configurar el refresc automàtic. Síntoma: les dades no s'actualitzen. Solució: configurar el refresh interval del dashboard.

Error 4: alertes que no s'envien. Síntoma: les condicions es compleixen però no rebem missatges. Solució: revisar el contact point i la regla d'alerta.

Error 5: gràfiques buides. Síntoma: no apareixen dades. Solució: comprovar que hi ha dades a InfluxDB, revisar la consulta, comprovar el rang temporal.

19.19 Bones pràctiques

1. **Exportar els dashboards a JSON** i versionar-los amb Git.
2. **Usar variables** per fer els dashboards reutilitzables.
3. **Configurar alertes visuals** (thresholds) a tots els panells crítics.
4. **Configurar alertes reals** per correu o Telegram als panells crítics.
5. **Limitar el nombre de panells** per dashboard.
6. **Documentar** què fa cada panell i què signifiquen les alertes.
7. **Provar les alertes** forçant condicions (per exemple, amb el simulador Python).
8. **Monitorar l'estat de Grafana** amb Uptime Kuma.
9. **Fer còpies de seguretat** periòdiques (exportar a JSON).
10. **Limitar l'accés** amb autenticació i, si cal, compartició pública selectiva.

19.20 Resum

Hem après què és Grafana, com es connecta a InfluxDB, com es creen panells i dashboards, com es configuren variables i alertes, i com es comparteixen resultats. Hem vist com crear un dashboard complet per a Hort Osona, amb gràfiques de temperatura, humitat, lluminositat i estat dels sensors. En el proper capítol veurem com exposar aquestes dades al món a través d'una API REST, perquè la web pública Hort Osona les pugui consumir.

19.21 Exercicis pràctics

1. Desplega Grafana al BernatLab amb la configuració que hem vist.
2. Configura la font de dades InfluxDB amb el token adequat.
3. Crea un dashboard amb almenys 4 panells: temperatura actual, temperatura 24h, humitat actual, humitat 24h.
4. Afegeix una variable `zona` que permeti filtrar els panells.
5. Configura un contact point de Telegram.
6. Crea una alerta visual al panell de temperatura (color vermell si $< 2\text{ °C}$).
7. Crea una regla d'alerta real que envii un missatge a Telegram quan la temperatura baixa de 2 °C .
8. Exporta el dashboard a JSON i guarda'l a `~/home/lab/stacks/grafana/dashboards/`.

Comandes útils:

```
# Veure els logs
docker compose logs -f grafana
```

```
# Reiniciar
docker compose restart grafana
```

```
# Exportar el dashboard (des de la UI)
# Dashboard settings → JSON model → Copy to clipboard
```


Paraules clau: **Grafana, dashboard, panell, time series, stat, gauge, bar chart, heatmap, table, variable, templating, alerta visual, threshold, alerta real, contact point, Telegram, JSON, export, import, InfluxDB, Flux, Font de dades, dades, gràfica, visualització, sensors, hort, BernatLab, Hort Osona, compartició pública, rendiment, monitoratge, Uptime Kuma.**

Capítol 20 — API pública: servir les dades al món

"Una API ben dissenyada és com un bon mosaic: cada peça té el seu lloc, i el conjunt és més que la suma de les parts."

20.1 Per què una API

Al BernatLab tenim les dades a InfluxDB, ben organitzades, amb un dashboard bonic a Grafana. Però ens falta una peça clau: **una manera de servir aquestes dades al món exterior**, en un format estandarditzat i documentat.

Les dades serveixen per molt més que visualitzar-les a Grafana. Volem:

- Que la **web pública Hort Osona** mostri gràfiques en temps quasi-real.
- Que una **aplicació mòbil** (que potser construirem en el futur) consumeixi les dades.
- Que altres **sistemes externs** integrin les dades d'Hort Osona.
- Que **desenvolupadors** puguin experimentar amb les dades sense haver de configurar InfluxDB.

La millor manera de fer tot això és amb una **API REST** que:

- Accepti peticions HTTP estàndard.
- Retorni dades en format JSON.
- Estigui documentada amb **OpenAPI** (Swagger).
- Tingui autenticació per API keys.
- Estigui versionada per a futures evolucions.

20.2 Quina tecnologia: FastAPI

Hi ha moltes opcions per construir una API en Python:

- **Flask**: senzill, flexible, però cal afegir moltes coses a mà.
- **Django REST Framework**: potent, però excessiu per al nostre cas.
- **FastAPI**: modern, ràpid, documentació automàtica, async.

Al BernatLab farem servir **FastAPI** perquè:

- Genera documentació OpenAPI automàticament.
- Suporta async/await de forma nativa.
- Valida les dades amb Pydantic.
- Té un rendiment comparable a Node.js o Go.
- És fàcil d'aprendre.

20.3 Estructura de l'API

L'API del BernatLab tindrà aquests endpoints principals:

GET /	→ informació general
GET /zones	→ llista de zones
GET /zones/{zona}/latest	→ últimes lectures d'una zona
GET /zones/{zona}/measurements/{tipus}	→ últimes N lectures d'un tipus
GET /measurements	→ últimes lectures de totes les zones
GET /stats	→ estadístiques agregades
GET /sensors/status	→ estat dels sensors (última publicació)
GET /health	→ estat del servei

Cada endpoint retornarà JSON. Alguns exemples de respostes:

****/zones****:

```
{
  "zones": ["zona-tomateres", "zona-enciams", "zona-pebrots"]
}
```

****/zones/zona-tomateres/latest****:

```
{
  "zona": "zona-tomateres",
  "last_update": "2026-07-08T12:34:56Z",
  "temperatura": {
    "valor": 23.5,
    "unitat": "graus_C"
  },
  "humitat": {
    "valor": 60,
    "unitat": "%"
  },
  "pressio": {
    "valor": 1013,
    "unitat": "hPa"
  }
}
```

20.4 Implementació

L'API serà un contenidor Docker amb Python 3.12 i FastAPI.

Estructura del projecte

```
/home/bernat/homelab/stacks/api/
■■■■ Dockerfile
■■■■ pyproject.toml
■■■■ requirements.txt
■■■■ app/
■   ■■■■ __init__.py
■   ■■■■ main.py
■   ■■■■ config.py
■   ■■■■ auth.py
■   ■■■■ influx.py
■   ■■■■ models.py
■   ■■■■ routers/
■       ■■■■ __init__.py
■       ■■■■ zones.py
■       ■■■■ measurements.py
■       ■■■■ stats.py
■       ■■■■ sensors.py
■       ■■■■ health.py
■■■■ .env.example
```

Dockerfile

```
FROM python:3.12-slim

WORKDIR /app

# Dependències del sistema
RUN apt-get update && apt-get install -y --no-install-recommends \
    gcc \
    && rm -rf /var/lib/apt/lists/*

# Dependències Python
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Codi
COPY app/ ./app/

# Usuari no root
RUN useradd -m -u 1000 apiuser
USER apiuser

EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

requirements.txt

```
fastapi==0.115.0
uvicorn[standard]==0.32.0
influxdb-client==1.43.0
pydantic==2.9.0
python-dotenv==1.0.1
pydantic-settings==2.5.2
```

Configuració (config.py)

```
from pydantic_settings import BaseSettings
from functools import lru_cache

class Settings(BaseSettings):
    # API
    api_title: str = "BernatLab API"
    api_version: str = "1.0.0"
    api_description: str = "API pública per a Hort Osona"

    # InfluxDB
    influx_url: str = "http://influxdb:8086"
    influx_token: str
    influx_org: str = "bernatlab"
    influx_bucket: str = "hort-osona"

    # Seguretat
    api_keys: str = "" # llista separada per comes

    # CORS
    cors_origins: str = "https://bernatmora.github.io"

    class Config:
        env_file = ".env"
        env_prefix = "API_"

@lru_cache
def get_settings():
    return Settings()
```

Client d'InfluxDB (influx.py)

```
from influxdb_client import InfluxDBClient
from .config import get_settings

_client: InfluxDBClient | None = None

def get_influx_client() -> InfluxDBClient:
    global _client
    if _client is None:
        settings = get_settings()
        _client = InfluxDBClient(
            url=settings.influx_url,
            token=settings.influx_token,
            org=settings.influx_org,
        )
    return _client

def get_query_api():
    return get_influx_client().query_api()
```

Autenticació (auth.py)

```
from fastapi import Security, HTTPException, status
from fastapi.security import APIKeyHeader
from .config import get_settings

api_key_header = APIKeyHeader(name="X-API-Key", auto_error=False)

def get_api_key(api_key: str = Security(api_key_header)) -> str:
    settings = get_settings()
    claus_valides = settings.api_keys.split(",")

    if api_key in claus_valides:
        return api_key

    raise HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="API key invàlida o absent",
    )
```

Model de dades (models.py)

```
from pydantic import BaseModel, Field
from datetime import datetime
from typing import Optional

class Mesura(BaseModel):
    valor: float
    unitat: str

class UltimesLectures(BaseModel):
    zona: str
    last_update: datetime
    temperatura: Optional[Mesura] = None
    humitat: Optional[Mesura] = None
    pressio: Optional[Mesura] = None
    lluminositat: Optional[Mesura] = None

class ZonaInfo(BaseModel):
    zona: str
    last_update: Optional[datetime] = None
    sensor_count: int = 0

class SensorStatus(BaseModel):
    zona: str
    last_publish: Optional[datetime]
    status: str # "online" | "stale" | "offline"
```

Router principal (main.py)

```

from fastapi import FastAPI, Depends
from fastapi.middleware.cors import CORSMiddleware
from .config import get_settings
from .routers import zones, measurements, stats, sensors, health

settings = get_settings()

app = FastAPI(
    title=settings.api_title,
    version=settings.api_version,
    description=settings.api_description,
    docs_url="/docs",
    redoc_url="/redoc",
)

# CORS
origins = settings.cors_origins.split(",")
app.add_middleware(
    CORSMiddleware,
    allow_origins=origins,
    allow_credentials=True,
    allow_methods=["GET"],
    allow_headers=["*"],
)

# Routers
app.include_router(health.router, tags=["health"])
app.include_router(
    zones.router,
    prefix="/zones",
    tags=["zones"],
    dependencies=[Depends(get_api_key)],
)
app.include_router(
    measurements.router,
    prefix="/zones",
    tags=["measurements"],
    dependencies=[Depends(get_api_key)],
)
app.include_router(
    stats.router,
    prefix="/stats",
    tags=["stats"],
    dependencies=[Depends(get_api_key)],
)
app.include_router(
    sensors.router,
    prefix="/sensors",
    tags=["sensors"],
    dependencies=[Depends(get_api_key)],
)

@app.get("/")
def root():
    return {
        "name": settings.api_title,
        "version": settings.api_version,
        "description": settings.api_description,
        "docs": "/docs",
    }

```

Endpoint /zones (routers/zones.py)

```
from fastapi import APIRouter, HTTPException
from ..influx import get_query_api
from ..models import ZonaInfo
import os

router = APIRouter()

@router.get("", response_model=list[ZonaInfo])
def llistar_zones():
    """Llista totes les zones amb sensors actius."""
    query_api = get_query_api()
    query = '''
import "influxdata/influxdb/schema"
schema.tagValues(bucket: "hort-osona", tag: "zona")
'''
    result = query_api.query(query)
    zones = [table.values[0] for table in result]
    return [{"zona": z, "last_update": None, "sensor_count": 1} for z in zones]
```


Endpoint /zones/{zona}/latest (routers/measurements.py)

```

from fastapi import APIRouter, HTTPException
from datetime import datetime
from ..influx import get_query_api
from ..models import UltimesLectures, Mesura

router = APIRouter()

@router.get("/{zona}/latest", response_model=UltimesLectures)
def ultimes_lectures(zona: str):
    """Retorna les últimes lectures de tots els sensors d'una zona."""
    query_api = get_query_api()
    query = f'''
    from(bucket: "hort-osona")
      |> range(start: -1h)
      |> filter(fn: (r) => r.zona == "{zona}")
      |> last()
    '''
    result = query_api.query(query)

    if not result:
        raise HTTPException(status_code=404, detail="Zona no trobada")

    lectures = {}
    last_update = None

    for table in result:
        for record in table.records:
            tipus = record.get_measurement()
            valor = record.get_value()
            unitat = record.values.get("unitat", "")
            time = record.get_time()

            if tipus not in lectures:
                lectures[tipus] = Mesura(valor=valor, unitat=unitat)
            if last_update is None or time > last_update:
                last_update = time

    return UltimesLectures(
        zona=zona,
        last_update=last_update,
        **lectures
    )

```

docker-compose.yml

```

services:
  api:
    build: ./stacks/api
    container_name: bernatlab-api
    restart: unless-stopped
    ports:
      - "8000:8000"
    environment:
      - API_INFLUX_URL=http://influxdb:8086
      - API_INFLUX_TOKEN=${API_INFLUX_TOKEN}
      - API_INFLUX_ORG=bernatlab
      - API_INFLUX_BUCKET=hort-osona
      - API_API_KEYS=${API_KEYS}
      - API_CORS_ORIGINS=https://bernatmora.github.io
    depends_on:
      - influxdb

```

20.5 Documentació automàtica

FastAPI genera automàticament la documentació OpenAPI. Un cop en marxa, podem accedir a:

- <http://100.115.134.76:8000/docs>: interfície Swagger UI.
- <http://100.115.134.76:8000/redoc>: interfície ReDoc.
- <http://100.115.134.76:8000/openapi.json>: especificació OpenAPI en JSON.

Aquesta documentació és navegable, podem provar les crides directament, i s'actualitza automàticament quan afegim nous endpoints.

20.6 Seguretat: API keys

L'API està protegida per una o més **API keys**. Cada client que vulgui accedir-hi ha d'incloure la capçalera **X-API-Key** amb una clau vàlida.

Al BernatLab, podem generar múltiples claus:

- **WEB_KEY**: per a la web pública Hort Osona.
- **MOBILE_KEY**: per a una futura aplicació mòbil.
- **DEV_KEY**: per a desenvolupament.

Cada clau es pot revocar individualment si cal.

Generar una clau forta

```
python3 -c "import secrets; print(secrets.token_urlsafe(32))"
```

Això ens dona una cadena aleatòria de 43 caràcters, perfecta com a API key.

20.7 CORS: permetre l'accés des de la web

Si volem que la web pública Hort Osona (allotjada a bernatmora.github.io) pugui consumir l'API, hem de configurar **CORS** (Cross-Origin Resource Sharing). Al codi, hem afegit:

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=["https://bernatmora.github.io"],
    allow_credentials=True,
    allow_methods=["GET"],
    allow_headers=["*"],
)
```

Això permet que el navegador accepti les respostes de l'API quan la petició ve de la web.

Compte: la Raspberry és a una xarxa privada (Tailscale), però la web pública és a Internet. La solució és exposar l'API a través d'un **proxy invers** (un nginx, un Cloudflare Tunnel) o fer que el navegador accedeixi directament a la IP Tailscale. En aquest últim cas, l'usuari ha de tenir Tailscale instal·lat. Una solució intermèdia és fer un **Cloudflare Tunnel** que exposi l'API de manera segura sense obrir ports.

20.8 Caching: optimitzar el rendiment

L'API rebrà peticions freqüents (cada vegada que la web es carrega). Per optimitzar el rendiment, podem afegir **caching**:

```
from fastapi_cache import FastAPICache
from fastapi_cache.backends.inmemory import InMemoryBackend
from fastapi_cache.decorator import cache

@app.get("/zones/{zona}/latest")
@cache(expire=30) # cache durant 30 segons
async def ultimes_lectures(zona: str):
    ...
```

Això fa que les respostes es guardin en memòria durant 30 segons, evitant crides repetides a InfluxDB. Si la web es refresca cada 30 segons, InfluxDB només rebrà una petició cada 30 segons per usuari.

20.9 Tests

Una bona API té tests. Podem escriure tests amb **pytest**:

```
from fastapi.testclient import TestClient
from app.main import app

client = TestClient(app)

def test_root():
    response = client.get("/")
    assert response.status_code == 200
    assert "BernatLab" in response.json()["name"]

def test_zones():
    response = client.get("/zones", headers={"X-API-Key": "test"})
    assert response.status_code == 200
    assert isinstance(response.json(), list)

def test_zona_latest():
    response = client.get(
        "/zones/zona-tomateres/latest",
        headers={"X-API-Key": "test"}
    )
    assert response.status_code == 200
    data = response.json()
    assert data["zona"] == "zona-tomateres"
```

Els tests els podem executar automàticament en cada desplegament.

20.10 Monitoratge de l'API

L'API s'ha de poder monitorar amb Uptime Kuma. Un monitor de tipus **HTTP(s)** que comprovi l'endpoint `/health` cada minut:

- URL: `http://100.115.134.76:8000/health`
- Interval: 60 segons

- **Condicció:** codi 2xx

Si l'API cau, Uptime Kuma ens avisarà per Telegram.

A més, podem afegir logs estructurats per analitzar quines crides es fan, amb quina freqüència, i quines fallen.

20.11 Desplegament i actualització

L'API es desplega com un contenidor Docker. Per actualitzar-la:

```
cd /home/bernat/homelab/stacks/api
git pull
docker compose build api
docker compose up -d api
```

Si hi ha canvis en les dependències, caldrà `docker compose build --no-cache api`.

20.12 Limitacions i millores futures

L'API actual és una base sòlida, però hi ha millores que es poden fer:

- **Paginació:** retornar moltes mesures pot ser lent. Cal afegir paginació.
- **Filtres avançats:** poder filtrar per dates, per tipus de mesura, etc.
- **Rate limiting:** limitar el nombre de peticions per IP.
- **Més endpoints:** alertes, històrics, comparacions entre zones, etc.
- **WebSockets:** per rebre dades en temps real sense fer polling.
- **GraphQL:** una alternativa a REST que permet consultes més flexibles.

Al BernatLab, anirem afegint funcionalitats a mesura que les necessitem, sense complicar excessivament la base.

20.13 Esquema conceptual

Esquema Mermaid (text):

```
graph TB
    subgraph Clients["Clients"]
        WEB["Web Hort Osona"]
        APP["App mòbil"]
        DEV["Desenvolupador"]
    end

    subgraph API["BernatLab API (FastAPI)"]
        ROOT[" / "]
        ZONES[" /zones "]
        MEAS[" /zones/{zona}/latest "]
        STATS[" /stats "]
        HEALTH[" /health "]
    end

    subgraph InfluxDB["InfluxDB"]
        B["Bucket hort-osona"]
    end

    subgraph Auth["Seguretat"]
        KEY["API Keys"]
        CORS["CORS"]
    end
```

```
end

WEB -->|X-API-Key| MEAS
APP -->|X-API-Key| ZONES
DEV -->|X-API-Key| STATS
MEAS --> B
ZONES --> B
STATS --> B
KEY --> API
CORS --> API
UPK["Uptime Kuma"] -->|/health| API
```

20.14 Errors habituals

Error 1: API key oblidada. Síntoma: rebem un 401. Solució: afegir la capçalera `X-API-Key` a totes les peticions.

Error 2: CORS bloquejant les peticions del navegador. Síntoma: la web no pot accedir a l'API. Solució: afegir l'origen a la llista de CORS.

Error 3: token d'InfluxDB incorrecte. Síntoma: l'API retorna errors 500. Solució: revisar el token al `.env`.

Error 4: consultes massa lentes. Síntoma: l'API triga molt a respondre. Solució: agregar les consultes a InfluxDB, afegir caching.

Error 5: no documentar els canvis. Síntoma: la documentació no correspon a la implementació. Solució: actualitzar el README quan es fan canvis.

20.15 Bones pràctiques

1. **Documentar cada endpoint** amb docstrings descriptius.
2. **Validar totes les entrades** amb Pydantic.
3. **Usar API keys** des del primer moment.
4. **Configurar CORS** adequadament.
5. **Afegir caching** per a consultes freqüents.
6. **Monitorar l'API** amb Uptime Kuma.
7. **Versionar l'API** (per exemple, `/v1/zones`).
8. **Fer tests** amb pytest.
9. **Documentar l'API** al README del projecte.
10. **Limitar el que retorna l'API** (no exposar dades internes).

20.16 Resum

Hem après què és una API REST, per què serveix, com construir-la amb FastAPI, com connectar-la a InfluxDB, com protegir-la amb API keys, com documentar-la amb OpenAPI, i com integrar-la amb la web pública. Hem vist exemples reals d'endpoints i configuracions. En el proper capítol veurem com la web Hort Osona consumeix aquesta API per mostrar dades en temps quasi-real.

20.17 Exercicis pràctics

1. Desplega l'API al BernatLab amb la configuració que hem vist.
2. Accedeix a <http://100.115.134.76:8000/docs> i explora la documentació.
3. Fes una crida amb `curl` a `/zones/zona-tomateres/latest`.
4. Genera una API key forta i afegeix-la al `.env`.
5. Prova de fer una crida sense API key. Hauries de rebre un 401.
6. Escriu un test amb `pytest` que comprovi l'endpoint `/health`.
7. Configura un monitor a Uptime Kuma per a l'API.
8. Documenta l'API al README del projecte.

Comandes útils:

```
# Generar una API key
python3 -c "import secrets; print(secrets.token_urlsafe(32))"

# Cridar l'API
curl -H "X-API-Key: CLAU" http://100.115.134.76:8000/zones
curl -H "X-API-Key: CLAU" http://100.115.134.76:8000/zones/zona-tomateres/latest

# Veure els logs
docker compose logs -f api

# Executar tests
docker compose exec api pytest
```

Paraules clau: **API, REST, FastAPI, OpenAPI, Swagger, JSON, endpoint, API key, CORS, autenticació, Pydantic, Uvicorn, async, InfluxDB, cache, monitoratge, Uptime Kuma, tests, pytest, versionat, documentació, web pública, Hort Osona, BernatLab, sensors, dades, peticions, respostes, codi HTTP, 200, 401, 500, headers, dependències, Docker, contenidor.**

Capítol 21 — Integració amb Hort Osona

""Una web estàtica pot ser bonica, però una web que respira amb dades reals és una altra cosa.""

21.1 On som

Al llarg d'aquest mòdul hem construït tota la cadena de dades:

- MQTT rep les dades dels sensors.
- Telegraf les mou a InfluxDB.
- Node-RED les processa i envia alertes.
- Grafana ensenya gràfiques.
- L'API FastAPI les serveix a qui les demani.

Ara toca tancar el cercle: **la web pública Hort Osona ha de consumir l'API per mostrar dades en temps quasi-real.**

La web actual (bernadmora.github.io/hort-osona) és una PWA (Progressive Web App) allotjada a GitHub Pages, feta amb HTML, CSS i JavaScript vanilla. Té 80+ documents, 9 categories, 8 pàgines funcionals. Ara li afegirem una secció nova: "**Dades en directe**", que mostrarà:

- Les últimes lectures de cada zona.
- Gràfiques històriques.
- L'estat dels sensors.

Això serà un procés incremental: primer una versió bàsica que consumeix l'API, després anirem afegint funcionalitats.

21.2 El problema de la connectivitat

Aquí topem amb un dels reptes del BernatLab: la Raspberry és a una xarxa privada (Tailscale), però la web pública és accessible des de qualsevol lloc del món. Com connectem les dues coses?

Tres opcions:

Opció 1: exposar l'API a través de Tailscale

Si els visitants de la web tenen Tailscale instal·lat i estan autenticats a la nostra xarxa, poden accedir directament a `http://100.115.134.76:8000`. Però això és molt limitat: la majoria de visitants no tindran Tailscale.

Opció 2: fer l'API accessible des d'Internet

Podem exposar el port 8000 a Internet a través del router. Però això és molt perillós: estarem exposant l'API a tothom, i malgrat l'API key, és un risc de seguretat innecessari.

Opció 3: usar un túnel (Cloudflare Tunnel, ngrok, etc.)

Un túnel crea un punt de connexió segur entre la nostra màquina i un servidor intermedi. La nostra API queda exposada a través d'un domini propi (per exemple, `api.bernatlab.cat`) sense necessitat d'obrir ports al router.

Al BernatLab, farem servir **Cloudflare Tunnel**, que és gratuït, fàcil de configurar, i segur.

21.3 Configurar Cloudflare Tunnel

Requisits

- Un domini gestionat per Cloudflare (pot ser gratuït).
- Un compte a Cloudflare.
- La imatge Docker `cloudflare/cloudflared`.

Pas 1: crear el túnel

A la consola de Cloudflare, anem a **Zero Trust** → **Networks** → **Tunnels** → **Create a tunnel**. Triem el tipus **Cloudflared** i donem un nom al túnel (per exemple, `bernatlab-api`).

Cloudflare ens donarà una comanda per executar al nostre servidor. La forma és:

```
cloudflared service install TOKEN
```

On `TOKEN` és un token llarg que identifica el túnel.

Pas 2: configurar el domini

A la configuració del túnel, afegim una ruta:

- **Subdomain:** `api`
- **Domain:** `bernatlab.cat`
- **Service:** `http://api:8000` (el nom del servei dins del `docker-compose.yml`)

Ara, qualsevol petició a `https://api.bernatlab.cat/` arriba al nostre servei API.

Pas 3: afegir cloudflared al docker-compose.yml

```
services:
  cloudflared:
    image: cloudflare/cloudflared:latest
    container_name: cloudflared
    restart: unless-stopped
    command: tunnel run
    environment:
      - TUNNEL_TOKEN=${CLOUDFLARE_TUNNEL_TOKEN}
    depends_on:
      - api
```

On `CLOUDFLARE_TUNNEL_TOKEN` és el token que hem obtingut al pas 1.

Avantatges

- **Cap port obert al router:** la Raspberry és inaccessible directament des d'Internet.

- **Xifratge de punta a punta:** Cloudflare xifra la connexió fins al nostre servidor.
- **Protecció DDoS gratuïta:** Cloudflare filtra atacs.
- **Analytics:** podem veure quantes peticions arriben, des d'on, etc.

Desavantatge

- Cal tenir un domini. Però un `.cat` o `.tk` pot costar només 5-10 € l'any.

21.4 Modificar la web Hort Osona

Ara que l'API és accessible des d'Internet (a través del túnel), podem modificar la web perquè la consumeixi.

Estructura de la nova secció

A la web Hort Osona, afegirem una nova pàgina: `directe.html`. Aquesta pàgina tindrà:

1. **Una capçalera** amb l'estat general (l'hora de l'última actualització, el nombre de sensors actius).
2. **Una graella de targetes**, una per zona, amb l'última lectura de cada sensor.
3. **Una gràfica** amb l'evolució de la temperatura al llarg del temps.
4. **Un peu** amb informació de contacte i enllaços.

Codi HTML bàsic

```
<!DOCTYPE html>
<html lang="ca">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Hort Osona - Dades en directe</title>
  <link rel="stylesheet" href="css/directe.css">
</head>
<body>
  <header>
    <h1>Hort Osona</h1>
    <nav>
      <a href="/">Inici</a>
      <a href="/hort/">L'hort</a>
      <a href="/directe/" class="active">Dades en directe</a>
    </nav>
  </header>

  <main>
    <section class="estat-general">
      <h2>Estat general</h2>
      <p>Última actualització: <span id="last-update">--</span></p>
      <p>Sensors actius: <span id="sensors-actius">--</span></p>
    </section>

    <section class="zones">
      <h2>Zones</h2>
      <div id="zones-grid" class="grid">
        <!-- Les targetes es generen dinàmicament -->
      </div>
    </section>

    <section class="grafica">
      <h2>Evolució de la temperatura (24 h)</h2>
      <canvas id="temperatura-chart"></canvas>
    </section>
  </main>

  <footer>
    <p>Dades proporcionades pel <a href="https://bernatlab.cat">BernatLab</a></p>
  </footer>

  <script src="https://cdn.jsdelivr.net/npm/chart.js@4.4"></script>
  <script src="js/directe.js"></script>
</body>
</html>
```

JavaScript per consumir l'API

```
// directe.js
const API_BASE = 'https://api.bernatlab.cat';
const API_KEY = 'CLAU_PUBLICA'; // compte: això serà visible

async function carregarDades() {
  try {
    // 1. Llista de zones
    const zonesResp = await fetch(`${API_BASE}/zones`, {
      headers: { 'X-API-Key': API_KEY }
    });
    const zones = await zonesResp.json();

    // 2. Per cada zona, obtenir últimes lectures
    const lectures = await Promise.all(
      zones.map(z => fetch(`${API_BASE}/zones/${z.zona}/latest`, {
        headers: { 'X-API-Key': API_KEY }
      })).then(r => r.json())
    );

    // 3. Renderitzar
    renderitzarZones(lectures);

    // 4. Actualitzar estat general
    document.getElementById('last-update').textContent =
      new Date().toLocaleString('ca-ES');
    document.getElementById('sensors-actius').textContent = zones.length;
  } catch (e) {
    console.error('Error carregant dades:', e);
  }
}

function renderitzarZones(lectures) {
  const grid = document.getElementById('zones-grid');
  grid.innerHTML = lectures.map(z => `
    <div class="zona-card">
      <h3>${z.zona}</h3>
      ${z.temperatura ? `
        <div class="mesura">
          <span class="valor">${z.temperatura.valor}°C</span>
          <span class="unitat">${z.temperatura.unitat}</span>
        </div>
      ` : ''}
      ${z.humitat ? `
        <div class="mesura">
          <span class="valor">${z.humitat.valor}%</span>
          <span class="unitat">${z.humitat.unitat}</span>
        </div>
      ` : ''}
      <p class="last-update">Actualitzat: ${new Date(z.last_update).toLocaleString('ca-ES')}</p>
    </div>
  `).join('');
}

async function carregarGrafica() {
  const resp = await fetch(`${API_BASE}/zones/zona-tomateres/measurements/temperatura?limit=100`, {
    headers: { 'X-API-Key': API_KEY }
  });
  const dades = await resp.json();

  const ctx = document.getElementById('temperatura-chart');
  new Chart(ctx, {
```

```

    type: 'line',
    data: {
      labels: dates.map(d => new Date(d.time).toLocaleTimeString('ca-ES')),
      datasets: [{
        label: 'Temperatura (°C)',
        data: dates.map(d => d.value),
        borderColor: '#1f3a5f',
        tension: 0.1
      }]
    },
    options: {
      responsive: true,
      plugins: {
        title: {
          display: true,
          text: 'Temperatura zona-tomateres'
        }
      }
    }
  });
}

// Inicialitzar
carregarDades();
carregarGrafica();

// Refrescar cada 60 segons
setInterval(carregarDades, 60000);

```

21.5 Seguretat: la clau pública

Aquí hi ha un problema: la clau d'API és visible al JavaScript del client, la qual cosa vol dir que qualsevol pot veure-la i fer-la servir.

Solucions

Opció 1: clau de només lectura amb limitacions

Creem una clau d'API específica per a la web, que només pot llegir ([read-only](#)) i té un **rate limit** molt baix. Per exemple, 60 peticions per minut. Així, si algú roba la clau, no pot fer gaire mal.

Opció 2: autenticació basada en domini

L'API pot validar que les peticions vinguin de [bernatmora.github.io](#) mirant la capçalera [Referer](#). Això no és gaire robust (es pot falsificar), però és una capa addicional.

Opció 3: subscripció per correu

Els visitants es subscriuen al web amb el seu correu, i reben un enllaç amb un token únic. Per a cada sessió, es genera un token temporal.

Opció 4: usar un intermediari

Una Cloudflare Worker actua com a intermediari: la web la crida, la Worker valida el domini, i la Worker crida l'API. La clau de l'API no és mai visible al client.

Al BernatLab, **combinarem l'opció 1 i l'opció 4**: una clau read-only amb rate limit, i una Cloudflare Worker com a intermediari (si cal).

21.6 CORS: configurar bé l'origen

A l'API, hem de permetre l'origen de la web:

```
allow_origins=["https://bernadmora.github.io"]
```

Si volem permetre també el domini propi, l'afegim:

```
allow_origins=[  
  "https://bernadmora.github.io",  
  "https://hortosona.bernadlab.cat"  
]
```

Compte: no posar mai `allow_origins=["*"]` en producció, perquè això permetria a qualsevol lloc accedir a l'API.

21.7 Caching al client

Per minimitzar el nombre de peticions, podem afegir caching al client:

```
// Cache simple en memòria  
const cache = new Map();  
const CACHE_TTL = 60000; // 1 minut  
  
async function fetchCached(url, options) {  
  const ara = Date.now();  
  const cached = cache.get(url);  
  
  if (cached && (ara - cached.timestamp) < CACHE_TTL) {  
    return cached.data;  
  }  
  
  const resp = await fetch(url, options);  
  const data = await resp.json();  
  cache.set(url, { data, timestamp: ara });  
  return data;  
}
```

Això redueix el nombre de peticions quan l'usuari navega per la web ràpidament.

21.8 Service Worker: PWA offline

Com que la web és una PWA, podem afegir un Service Worker que cachei les dades per a quan l'usuari està offline:

```
// sw.js  
const CACHE_NAME = 'hort-sona-v1';  
const urlsToCache = [  
  '/',  
  '/directe/',  
  '/css/directe.css',  
  '/js/directe.js'  
];  
  
self.addEventListener('install', event => {  
  event.waitUntil(  
    caches.open(CACHE_NAME)  
      .then(cache => cache.addAll(urlsToCache))  
  );  
});
```

```
self.addEventListener('fetch', event => {
  event.respondWith(
    caches.match(event.request)
      .then(response => response || fetch(event.request))
  );
});
```

Això permet que la web continuï funcionant (amb dades obsoletes) quan l'usuari perd la connexió.

21.9 Rendiment: optimitzar la càrrega

Per minimitzar el temps de càrrega:

1. **Comprimir les imatges:** usar formats moderns (WebP, AVIF).
2. **Minificar el CSS i JavaScript:** eliminar espais i comentaris.
3. ****Carregar els scripts amb `defer` o `async`**:** per no bloquejar la renderització.
4. **Usar CDNs per a llibreries externes:** Chart.js, per exemple.
5. **Limitar les dades retornades per l'API:** només el que la pàgina necessita.

21.10 Proves d'integració

Quan tot estigui enllestit, hem de provar:

1. **La web carrega correctament** amb dades reals.
2. **L'API respon** amb les dades esperades.
3. **El túnel Cloudflare funciona.**
4. **El CORS està ben configurat.**
5. **Les gràfiques es renderitzen.**
6. **El refresc periòdic funciona.**
7. **L'API key és vàlida.**
8. **El rate limit funciona** (per exemple, fent 100 peticions en 1 minut).

21.11 Publicació

Quan tot estigui provat, podem fer commit i push a GitHub:

```
cd hort-osona
git add directe.html css/directe.css js/directe.js sw.js
git commit -m "Afegeix secció Dades en directe"
git push origin main
```

GitHub Pages actualitzarà la web automàticament en pocs segons.

21.12 Esquema d'integració

Esquema Mermaid (text):

```
graph TB
  subgraph Sensors["Sensors al terreny"]
```

```

    S["ESP32 + BME280"]
end

subgraph BernatLab["BernatLab (Raspberry Pi)"]
    MOSQ["Mosquitto"]
    TELE["Telegraf"]
    INFL["InfluxDB"]
    API["API FastAPI"]
end

subgraph Cloudflare["Cloudflare"]
    TUNNEL["Tunnel"]
end

subgraph Web["Web pública"]
    GH["GitHub Pages<br/>(bernatmora.github.io)"]
    JS["JavaScript<br/>(fetch API)"]
end

subgraph Usuari["Usuari"]
    U["Visitant"]
end

S --> MOSQ
MOSQ --> TELE
TELE --> INFL
INFL --> API
API --> TUNNEL
TUNNEL --> GH
GH --> JS
JS --> U
U -->|navega| GH

```

21.13 Bones pràctiques

1. **Clau read-only amb rate limit** per a l'accés des de la web.
2. **CORS estricte**: només els orígens necessaris.
3. **Caching al client** per reduir peticions.
4. **Service Worker** per a offline.
5. **Comprimir imatges** i minificar codi.
6. **Provar en múltiples dispositius**: mòbil, tablet, PC.
7. **Documentar al README** com afegir noves zones o gràfiques.
8. **Monitorar l'API** amb Uptime Kuma.
9. **Versionar la web** amb Git, igual que el BernatLab.
10. **Fer commits petits i freqüents**, amb missatges clars.

21.14 Limitacions i millores

El que tenim ara és una primera versió funcional. Però hi ha moltes millores possibles:

- **Més gràfiques**: humitat, llum, etc.
- **Comparació entre zones**.
- **Mapa de l'hort** amb les zones marcades.
- **Alertes visuals** quan alguna cosa va malament.

- **Exportació de dades** (CSV, JSON).
- **Historial** amb calendari.
- **Integració amb calendaris** (sembra, collita).
- **Múltiples idiomes**.

Aquestes millores les farem gradualment, validant cada canvi amb el món real.

21.15 Resum

Hem après com connectar la web pública Hort Osona amb l'API del BernatLab, usant Cloudflare Tunnel com a intermediari segur. Hem vist el codi HTML, CSS i JavaScript necessari per mostrar les dades, hem après a configurar el CORS, a protegir la clau d'API, i a optimitzar el rendiment. En el proper i últim capítol del Mòdul 2 veurem l'operativa: còpies de seguretat, retenció, alerting avançat i quan caldrà pujar de hardware.

21.16 Exercicis pràctics

1. Configura Cloudflare Tunnel per exposar l'API.
2. Crea una nova pàgina `directe.html` a la web Hort Osona.
3. Escribe el JavaScript per consumir l'API i mostrar les dades.
4. Afegeix una gràfica de temperatura amb Chart.js.
5. Configura el CORS a l'API per permetre el domini de la web.
6. Prova la web localment amb `python -m http.server 8000`.
7. Fes commit i push a GitHub.
8. Verifica que la web pública mostra les dades correctament.
9. Documenta al README del projecte Hort Osona com funciona la integració.

Comandes útils:

```
# Servir la web localment per provar
cd hort-osona
python3 -m http.server 8000

# Comprovar l'API
curl -H "X-API-Key: CLAU" https://api.bernatlab.cat/zones

# Fer commit
git add .
git commit -m "Afegeix Dades en directe"
git push
```

Paraules clau: **web pública, Hort Osona, integració, API, Cloudflare Tunnel, Tailscale, CORS, rate limit, API key read-only, Chart.js, JavaScript, HTML, CSS, PWA, Service Worker, caching, GitHub Pages, Git, commit, push, monitoratge, Uptime Kuma, gràfica, zona, sensors, dades, BernatLab, domini, túnel, intermediari, seguretat, optimització, imatges, minificació, offline.**

Capítol 22 — Operativa: còpies, alertes, escalat

"Un sistema en producció no es diferencia d'un prototip pel que fa, sinó pel que passa quan alguna cosa falla."

22.1 Què hem construït

En aquest mòdul hem afegit al BernatLab:

- **Mosquitto**: broker MQTT amb autenticació i ACLs.
- **InfluxDB**: base de dades de sèries temporals.
- **Telegraf**: agent de recollida de dades.
- **Node-RED**: eina de programació visual.
- **Grafana**: visualització.
- **API FastAPI**: servei REST per a clients externs.
- **Cloudflare Tunnel**: exposició segura de l'API.
- **Integració amb la web Hort Osona**.

Tot plegat, **sis serveis nous** i una integració amb una web pública. El sistema, ara com ara, és força complex. I com més complex, més important és la operativa diària.

En aquest capítol veurem les tasques que hem de fer regularment per mantenir el sistema sa: còpies de seguretat, monitoratge avançat, gestió de la retenció, i quan caldrà considerar pujar de hardware.

22.2 Còpies de seguretat

La regla d'or: **si no podem restaurar el sistema en una hora, no estem preparats**.

Què hem de copiar?

Dades d'InfluxDB

InfluxDB conté tota la història de mesures. Si la perdem, perdem tot el coneixement acumulat sobre l'hort.

Còpia amb la CLI d'InfluxDB:

```
influx backup /home/bernat/homelab/backup/influxdb-$(date +%F) \  
  --bucket hort-osona \  
  --org bernatlab \  
  --token TOKEN
```

Aquesta ordre crea una còpia completa del bucket. Per restaurar:

```
influx restore /home/bernat/homelab/backup/influxdb-2026-XX-XX \  
  --bucket hort-osona \  
  --org bernatlab \  
  --token TOKEN
```

Compte: les còpies es fan amb el servei **aturat** per garantir la consistència.

Configuracions

Tots els fitxers de configuració viuen a `/home/bernat/home lab/stacks/`, que ja està versionat amb Git. Tanmateix, podem fer còpies addicionals:

- `telegraf.conf`
- `mosquitto.conf`, `passwordfile`, `aclfile`
- `nodered/flows.json`
- `grafana/dashboards/*.json`
- `api/.env`

Volums de dades

Els volums de dades contenen:

- InfluxDB: les dades històriques (ja cobert per la còpia d'InfluxDB).
- Grafana: dashboards, alertes, configuracions.
- Node-RED: fluxos.
- Mosquitto: missatges retained.

Per a còpies completes, podem aturar tots els serveis i copiar `/home/bernat/home lab/data/sencer`.

Freqüència

- **Diàriament**: còpia d'InfluxDB.
- **Setmanalment**: còpia de les configuracions no versionades.
- **Mensualment**: còpia completa del sistema, emmagatzemada fora de la Raspberry.

On guardar les còpies

Tres opcions:

1. **Un altre disc de la Raspberry**: ràpid, però no protegeix contra fallada de hardware.
2. **Un PC de la xarxa**: bona opció, amb `rsync` o `scp`.
3. **El núvol**: la millor opció per a còpies de seguretat de veritat. Backblaze B2, Mega.nz, o un repositori Git privat.

Al BernatLab, farem còpies al núvol amb un script que combina `influx backup` amb `rc lone` (que suporta múltiples proveïdors de núvol).

22.3 Retenció de dades

InfluxDB ens permet definir **polítiques de retenció** per a cada bucket. Al Capítol 15 ja vàrem parlar d'això, però ara veurem com gestionar-ho de forma dinàmica.

Estratègia de retenció

Al BernatLab, fem servir tres buckets:

- **hort-osona** (1 any): dades originals, alta resolució.
- **hort-osona-1h** (5 anys): dades agregades a resolució horària.
- **hort-osona-1d** (10 anys): dades agregades a resolució diària.

Les agregacions les fan **tasks** periòdiques que calculen la mitjana, màxim i mínim per a finestres temporals.

Quanta memòria/disc ocupa?

Podem estimar:

- $10 \text{ sensors} \times 5 \text{ mesures cadascun} \times 1 \text{ punt per minut} \times 60 \text{ minuts} \times 24 \text{ hores} \times 365 \text{ dies} = \mathbf{262 \text{ milions de punts per any.}}$
- Cada punt ocupa uns 100 bytes amb índexs.
- Total: uns **26 GB per any**.

Això és molt per a una microSD de 32 GB. Per això cal:

- **Reduir la freqüència de publicació** si no necessitem 1 punt per minut.
- **Agregar dades** en buckets separats.
- **Usar un SSD USB** (pròximament).

Monitoratge de l'ús de disc

Un script que ens avisa quan el disc s'omple:

```
#!/bin/bash
# comprovar_espai.sh
US=$(df -h / | tail -1 | awk '{print $5}' | sed 's/%//')
if [ $US -gt 80 ]; then
    curl -X POST "https://api.telegram.org/bot$BOT_TOKEN/sendMessage" \
        -d "chat_id=$CHAT_ID" \
        -d "text=🚨 Disc del BernatLab al ${US}% d'ús"
fi
```

Podem programar aquest script amb **cron** perquè s'executi cada dia.

22.4 Monitoratge avançat

Uptime Kuma ens avisa quan un servei cau. Però podem anar més enllà:

Monitorar mètriques internes

Cada servei pot exposar mètriques que Uptime Kuma pot consultar:

- **InfluxDB**: té un endpoint `/metrics` en format Prometheus.
- **Node-RED**: pot exposar mètriques amb un node personalitzat.
- **Grafana**: té el seu propi sistema d'alertes.

- **API:** podem afegir un endpoint `/metrics` amb `prometheus_client`.

Això ens permet monitorar coses com:

- Quantes consultes per segon rep l'API.
- Quanta memòria consumeix Node-RED.
- Quantes insercions per segon fa Telegraf.

Alertes de capacitat

A més dels serveis caiguts, podem alertar quan:

- El disc supera el 80 % d'ús.
- La RAM supera el 90 % d'ús.
- La CPU supera el 80 % durant més de 5 minuts.
- La temperatura de la CPU supera els 70 °C.

Això es pot fer amb un script que consulti `/proc` i enviï alertes per Telegram.

Alertes de dades

També podem alertar quan les dades no arriben:

- Cap publicació MQTT durant 10 minuts → possible problema de xarxa o de sensors.
- InfluxDB no ha rebut cap punt en 30 minuts → problema amb Telegraf.
- Un sensor concret porta més d'1 hora sense publicar → possible bateria esgotada.

Aquestes alertes les podem configurar a Node-RED o Grafana.

22.5 Logs centralitzats

Quan tenim molts serveis, revisar logs pot ser un mal de cap. Una solució és centralitzar-los.

Opció simple: volums compartits

Tots els serveis escriuen els seus logs a `/home/bernat/homelab/logs/`, organitzat per servei:

```
volumes:  
- /home/bernat/homelab/logs/mosquitto:/mosquitto/log  
- /home/bernat/homelab/logs/influxdb:/var/log/influxdb  
- /home/bernat/homelab/logs/grafana:/var/log/grafana
```

Podem revisar-los tots amb `tail -f` o eines com `lnav`.

Opció avançada: Loki

Loki és un sistema de logs centralitzat, similar a Prometheus però per a logs. Grafana s'hi integra nativament.

Però això és potser excessiu per a un homelab. Per ara, els volums compartits són suficients.

22.6 Actualitzacions

Mantenir el sistema actualitzat és fonamental, però cada actualització és un risc.

Estratègia

1. **Llegir les notes de versió** abans d'actualitzar res.
2. **Fer una còpia de seguretat** abans d'actualitzar.
3. **Actualitzar un servei a la vegada.**
4. **Provar** que tot funciona.
5. **Si alguna cosa falla**, poder tornar enrere.

Procediment estàndard

```
cd /home/bernat/homelab/stacks/iot
docker compose pull
docker compose up -d
# Esperar 5 minuts
docker compose ps
docker compose logs --tail=50
```

Si tot és correcte, podem continuar amb el següent servei.

Quan NO actualitzar

- Si el sistema està en ple funcionament i no tenim temps per depurar.
- Si les notes de versió mencionen canvis que poden trencar la configuració.
- Si tenim poc espai de disc per fer còpies.

22.7 Rendiment: quan cal pujar de hardware

Amb 4 GB de RAM i una microSD, el BernatLab té un sostell. Quan el sistema creix, pot ser que:

- InfluxDB es torna lent.
- Grafana trigui massa a carregar.
- Els contenidors es reiniciïn per manca de memòria.
- La microSD s'ompli ràpidament.

Quan cal més RAM

Si veiem **killed** als logs (Linux mata processos quan no hi ha prou RAM), és hora d'ampliar. Les solucions:

- **Passar a 8 GB de RAM** (la Raspberry Pi 4 admet fins a 8 GB).
- **Afegir swap a un SSD USB** (per crear "RAM virtual" en un disc).

Quan cal un SSD

La microSD és el coll d'ampolla per a bases de dades. Si InfluxDB va lent i tenim moltes dades, és hora de moure les dades a un SSD.

La Raspberry Pi 4 pot arrencar des d'un SSD USB. El procediment:

1. Connectar el SSD a un port USB 3.0.
2. Instal·lar Raspberry Pi OS al SSD (o copiar la microSD).
3. Configurar la BIOS per arrencar des d'USB.
4. Moure les dades.

Això és un projecte per si sol, però és al full de ruta del BernatLab.

Quan cal un segon servidor

Si arribem al punt que la Raspberry no dona per a més, podem considerar:

- Una Raspberry Pi 5 (més potent, ~120 €).
- Un mini PC amb Intel N100 (~200 €).
- Un servidor dedicat al núvol (VPS).

Per ara, amb la Raspberry 4 i un SSD, podem arribar força lluny.

22.8 Seguretat contínua

La seguretat no és un estat, és un procés. Cal revisar periòdicament:

Auditar accessos

- Qui s'ha connectat per SSH? (`journalctl -u ssh --since "1 month ago"`)
- Quins clients MQTT s'han connectat? (Logs de Mosquitto)
- Quins accessos a l'API s'han fet? (Logs de l'API)

Auditar tokens

- Tots els tokens d'InfluxDB són necessaris?
- Les API keys s'usen, o estan abandonades?
- Hi ha credencials al codi que s'haurien de moure al `.env`?

Auditar ports

`ss -tulpn`

Quins ports tenim oberts? Tots són necessaris? Algú ha afegit un servei que no toca?

Auditar permisos

- Els usuaris tenen només els permisos que necessiten?
- Hi ha fitxers amb permisos massa oberts? (`chmod 777` és sempre sospitós)

Auditar actualitzacions

- Tots els serveis estan a l'última versió?
- Hi ha vulnerabilitats conegudes? (`docker scan` o `trivy`)

22.9 Documentació contínua

Cada canvi al sistema ha de quedar documentat. Al BernatLab, tenim:

- **README.md**: descripció general, com començar.
- **CHANGELOG.md**: registre cronològic de canvis.
- **docs/**: documentació addicional per temes.
- **Git**: historial de canvis a la configuració.
- **Aquest manual**: el coneixement aprofundit.

Quan fem un canvi important, hem d'actualitzar el `CHANGELOG.md`:

```
# CHANGELOG — BernatLab
```

```
## 2026-08-15
```

- Afegit sensor BME280 a la zona de pebrots
- Configurat monitor d'Uptime Kuma per a l'API
- Actualitzat InfluxDB a 2.7.5

```
## 2026-08-01
```

- Integració amb la web Hort Osona completa
- Publicació del dashboard de Grafana

22.10 Procediments davant d'incidents

Quan alguna cosa falla, hem de tenir un protocol clar:

Pas 1: detectar

Uptime Kuma ens avisa per Telegram.

Pas 2: identificar

Quin servei falla? Què diuen els logs?

```
docker compose logs servei
```

Pas 3: contenir

Si el servei no es recupera amb un reinici, podem aturar-lo temporalment per evitar danys:

```
docker compose stop servei
```

Pas 4: resoldre

Buscar la solució: llegir documentació, buscar a Internet, provar.

Pas 5: recuperar

Tornar a aixecar el servei:

```
docker compose up -d servei
```

Pas 6: aprendre

Un cop resolt, escriure al CHANGELOG què ha passat i com s'ha resolt. Si cal, afegir un monitor per detectar-ho abans la pròxima vegada.

22.11 Quan rebre ajuda

A vegades, tot i els nostres esforços, no podem resoldre un problema. En aquests casos:

- **Documentació oficial:** la primera parada.
- **Forums i comunitats:** Reddit, StackOverflow, fòrums específics.
- **GitHub Issues:** si el problema és d'un servei específic, potser ja està reportat.
- **ChatGPT, Claude, altres IAs:** eines útils per a depurar.

I, per descomptat, **jo (Hermes)** soc aquí per ajudar-te amb el BernatLab quan calgui.

22.12 Full de ruta operativa

Aquí tens una llista de tasques operatives que hauries de fer regularment:

Diàriament

- ☐ Comprovar Uptime Kuma (o rebre les alertes per Telegram).
- ☐ Mirar si hi ha alertes de sensors inactius.

Setmanalment

- ☐ Còpia de seguretat d'InfluxDB.
- ☐ Revisar els logs dels serveis.
- ☐ Comprovar l'ús de disc i memòria.
- ☐ Actualitzar contenidors si n'hi ha de nous.

Mensualment

- ☐ Còpia de seguretat completa al núvol.
- ☐ Auditar accessos i tokens.
- ☐ Revisar el CHANGELOG i actualitzar-lo si cal.
- ☐ Comprovar l'estat del hardware (temperatura, salut de la microSD/SSD).

Anualment

- ☐ Planificar actualitzacions de hardware.
- ☐ Revisar l'arquitectura del sistema i simplificar si cal.
- ☐ Fer un balanç: què ha funcionat, què no.

22.13 Reflexió final

Hem cobert molta terra en aquest mòdul. Hem passat d'un servidor amb tres serveis bàsics a un sistema complet de captura, emmagatzemament, processament, visualització i exposició de dades d'un hort. Això és una fita important.

Però recorda: la millor tecnologia és la que s'usa. No té sentit tenir el sistema més sofisticat del món si després no el mirem, no l'entendem, no l'aprofitem. Un sistema senzill que funciona és infinitament millor que un sistema complex que abandonem.

Si en algun moment el BernatLab es torna massa complicat de mantenir, **simplifica**. Lleva el que no usis. Documenta el que conservis. Fes que cada peça compti.

22.14 Resum

Hem après les tasques operatives essencials per mantenir el BernatLab sa: còpies de seguretat, retenció de dades, monitoratge avançat, logs centralitzats, actualitzacions, rendiment, seguretat contínua, documentació, i procediments davant d'incidents. Hem vist quan cal pujar de hardware i hem establert una full de ruta operativa diària, setmanal, mensual i anual. En el proper mòdul (M3) tractarem la part de ràdio: sensors LoRa SX1262, xarxes de llarg abast, i la integració amb el sistema MQTT. En el M4 parlarem d'IA local amb Ollama i RAG sobre les dades.

22.15 Exercicis pràctics

1. Escriu un script `backup.sh` que faci còpia d'InfluxDB i la pugi al núvol amb `rclone`.
2. Programa aquest script amb `cron` perquè s'executi cada dia a les 3:00 AM.
3. Configura un monitor a Uptime Kuma per a l'API i per a Grafana.
4. Mira l'ús de disc actual: `df -h`. Quant queda lliure? Quant ocupa InfluxDB?
5. Escriu al CHANGELOG un resum de tot el que hem fet en aquest mòdul.
6. Planifica una auditoria de seguretat: quins tokens, quins usuaris, quins ports.
7. Documenta al README del projecte el procediment davant d'incidents.
8. Fes una còpia de seguretat completa del sistema i guarda-la fora de la Raspberry.

Comandes útils:

```
# Còpia d'InfluxDB
influx backup /backup/influxdb-$(date +%F) --token TOKEN
```

```
# Ús de disc
df -h
du -sh /home/bernat/homelab/data/*
```

```
# Monitorar serveis
docker ps
docker stats --no-stream
```

```
# Veure logs
docker compose logs -f --tail=100
```

```
# Auditoria
ss -tulpn
```

```
journalctl -u ssh --since "1 month ago"
```

Paraules clau: **operativa, còpia de seguretat, backup, retenció, monitoratge, alertes, logs, actualitzacions, rendiment, RAM, SSD, seguretat, auditar, tokens, ports, permisos, documentació, CHANGELOG, incidents, procediment, full de ruta, operativa diària, operativa setmanal, operativa mensual, operativa anual, simplificar, sistema, BernatLab, sensors, dades, hort, Mòdul 3, Mòdul 4, LoRa, IA, Ollama, RAG, cron, rclone, Uptime Kuma, Telegram, df, docker stats, journalctl, ss, talls, hardware, microSD, Raspberry Pi 5, N100, VPS.**